

Memory Hierarchy: Basics

Hung-Wei Tseng



Disney · PIXAR INSIDE OUT

GET DISNEY+

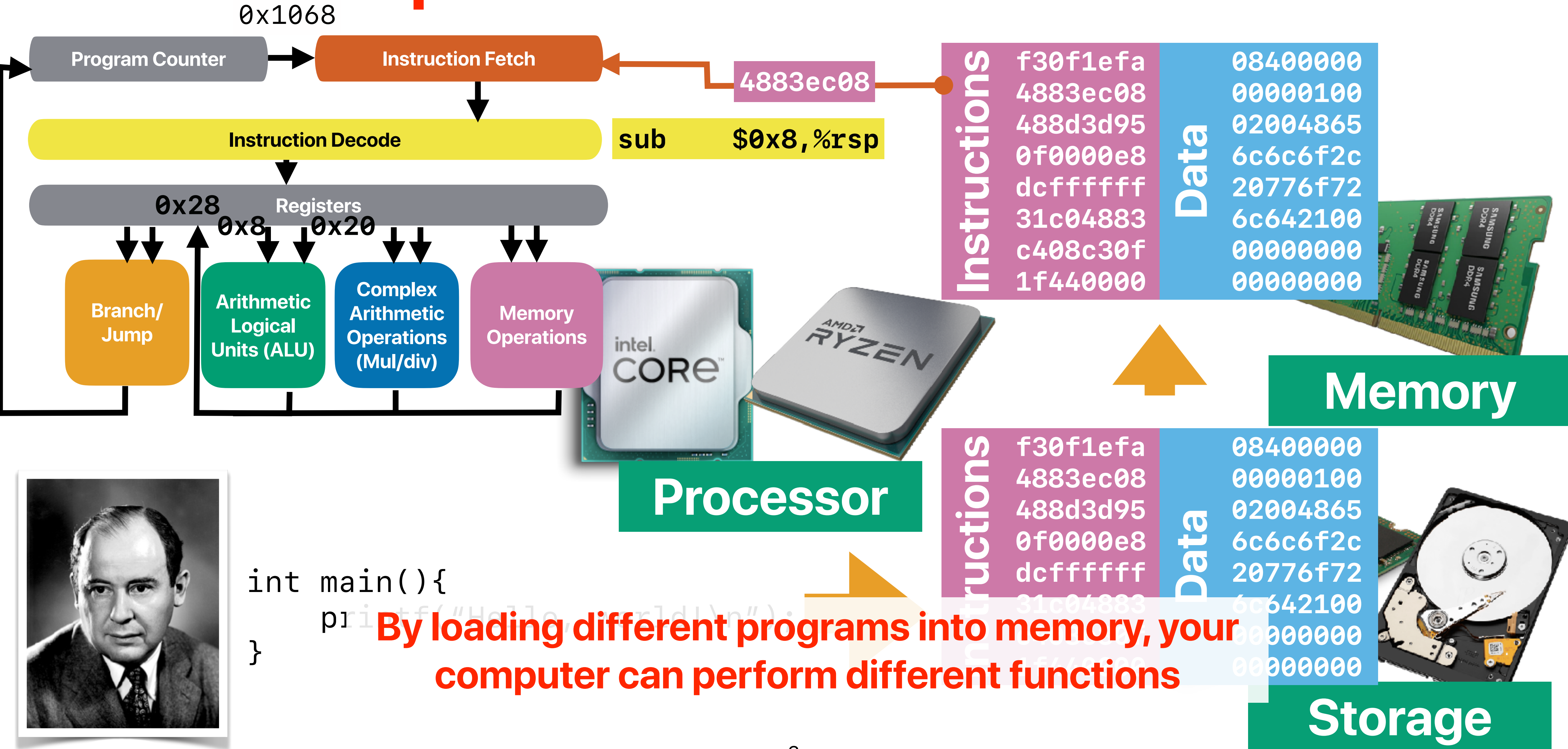
▶ TRAILER

PG 2015 • 1h 35m • Coming of age, Family, Animation

When 11-year-old Riley moves to a new city, her Emotions team up to help her through the transition. Joy, Fear, Anger, Disgust and Sadness work together, but when Joy and Sadness get lost, they must journey through unfamiliar places to get back home.



Recap: von Neumann architecture



Recap: Amdahl's Law

- Definition of "Speedup of Y over X" or say Y is n times faster than X:

$$speedup_{Y_over_X} = n = \frac{Execution\ Time_X}{Execution\ Time_Y}$$

- Amdahl's Law — $Speedup_{enhanced}(f, s) = \frac{1}{(1-f) + \frac{f}{s}}$

- Corollary 1 — each optimization has an upper bound $Speedup_{max}(f, \infty) = \frac{1}{(1-f)}$

- Corollary 2 — make the common case (the most time consuming case) fast!

$$Speedup_{max}(f_1, \infty) = \frac{1}{(1-f_1)}$$

$$Speedup_{max}(f_2, \infty) = \frac{1}{(1-f_2)}$$

$$Speedup_{max}(f_3, \infty) = \frac{1}{(1-f_3)}$$

$$Speedup_{max}(f_4, \infty) = \frac{1}{(1-f_4)}$$

- Corollary 3 — Optimization has a moving target

- Corollary 4 — Exploiting more parallelism from a program is the key to performance gain in modern architectures

$$Speedup_{parallel}(f_{parallelizable}, \infty) = \frac{1}{(1-f_{parallelizable})}$$

- Corollary 5 — Single-core performance still matters

$$Speedup_{parallel}(f_{parallelizable}, \infty) = \frac{1}{(1-f_{parallelizable})}$$

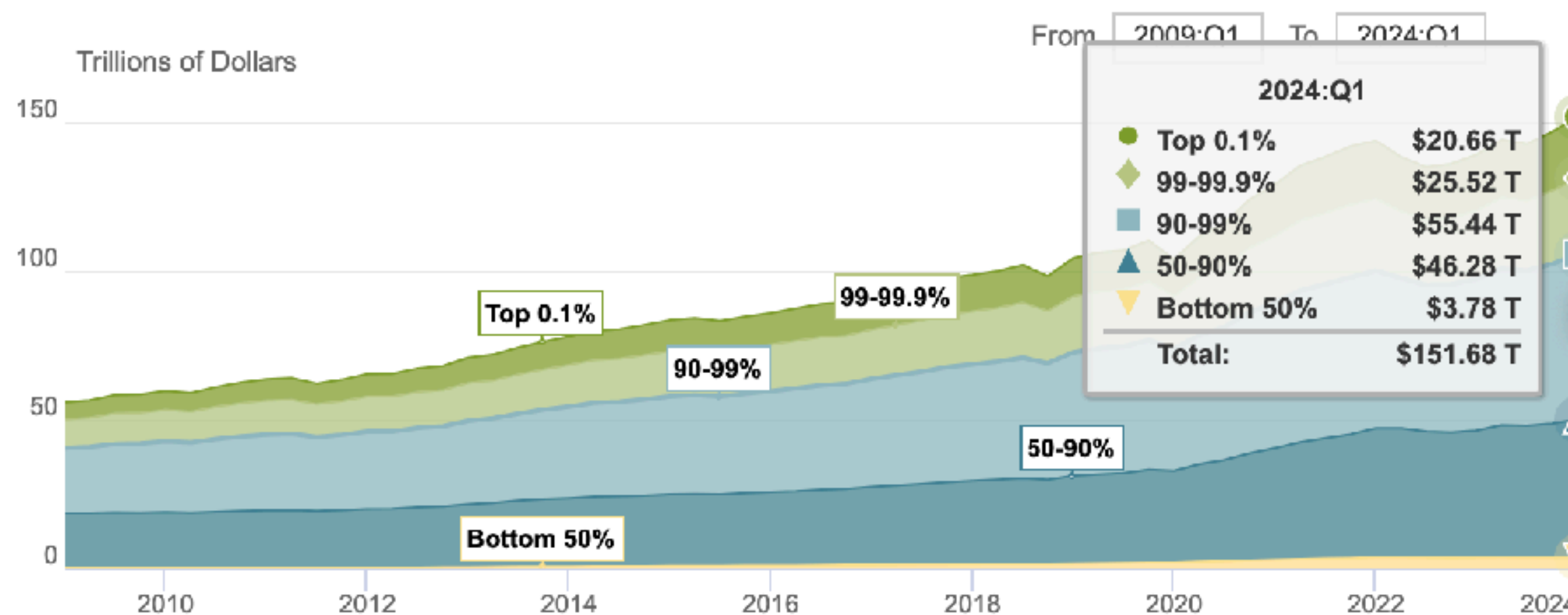
- Corollary 6 — Don't hurt the non-common case too much

$$Speedup_{enhanced}(f, s, r) = \frac{1}{(1-f) + perf(r) + \frac{f}{s}}$$

What percentage of “English words” from the dictionary do you think someone has to know for their everyday life?



Wealth by wealth percentile group



Top 10% own 67% of the wealth in the U.S.

Microsoft's CEO: 80-20 Rule Applies To Bugs, Not Just Features

BY PAULA ROONEY ▶

OCTOBER 03, 2002, 03:56 PM EDT

Microsoft

One common adage in the IT industry is that 80 percent of all end users generally use only 20 percent of a software application's features.

In recent months, Microsoft has learned that 80 percent of the errors and crashes in Windows and Office are caused by 20 percent of the entire pool of bugs detected, and that more than 50 percent of the headaches derive from a mere 1 percent of all flawed code.

In an e-mail update sent out broadly to enterprise customers on Oct. 2, Microsoft CEO Steve Ballmer highlighted the company's Trustworthy Computing initiative, an effort to improve the reliability and security of its products. For one thing, there will be faster bug-fixing as a result of an error-reporting facility embedded in Office and Windows. Another tool will be part of the forthcoming Windows XP.

80% of users use only 20% of features

[In the world](#)[For students](#)[Reddit](#)

AI Overview

[Learn more](#)

Native English speakers use around **2,000–3,000** words in daily life, which account for about 80% of their communication. These words include common verbs like "eat," "sleep," "work," "talk," and "walk," as well as pronouns like "I," "you," "he," and "she," and basic nouns like "house," "car," "food," and "water". The most commonly used words in English are "the," "be," and "to".

Eton Institute · LinkedIn · 1y

How many words are in the English language (95/5 RULE)?

How Many Words Do Native Speakers Use in Daily Life? Native speakers of...

GeeksforGeeks

Daily Used English Words: List of 100+ Most Common Words

May 6, 2024 — Words used in daily life often include common verbs like...

[Show more](#)

With **2,500 to 3,000 words**, you can understand **90%** of everyday English conversations, English newspaper and magazine articles, and English used in the workplace.



ef.edu

<http://www.ef.edu/english-vocabulary/top-3000-words>

3000 most common words in English

AI Overview

The number of words in an English dictionary depends on the dictionary whether it includes obsolete words, combinations, and phrases:

Oxford English Dictionary

The second edition of this 20-volume dictionary includes **171,476** words in current use, 47,156 obsolete words, and around 9,500 derivative words. It also includes combinations and derivatives, and 169,000 phrases and combinations, for a total of over 600,000 word forms. The dictionary is updated annually to include new words and new meanings for existing words.

Webster's Third New International Dictionary

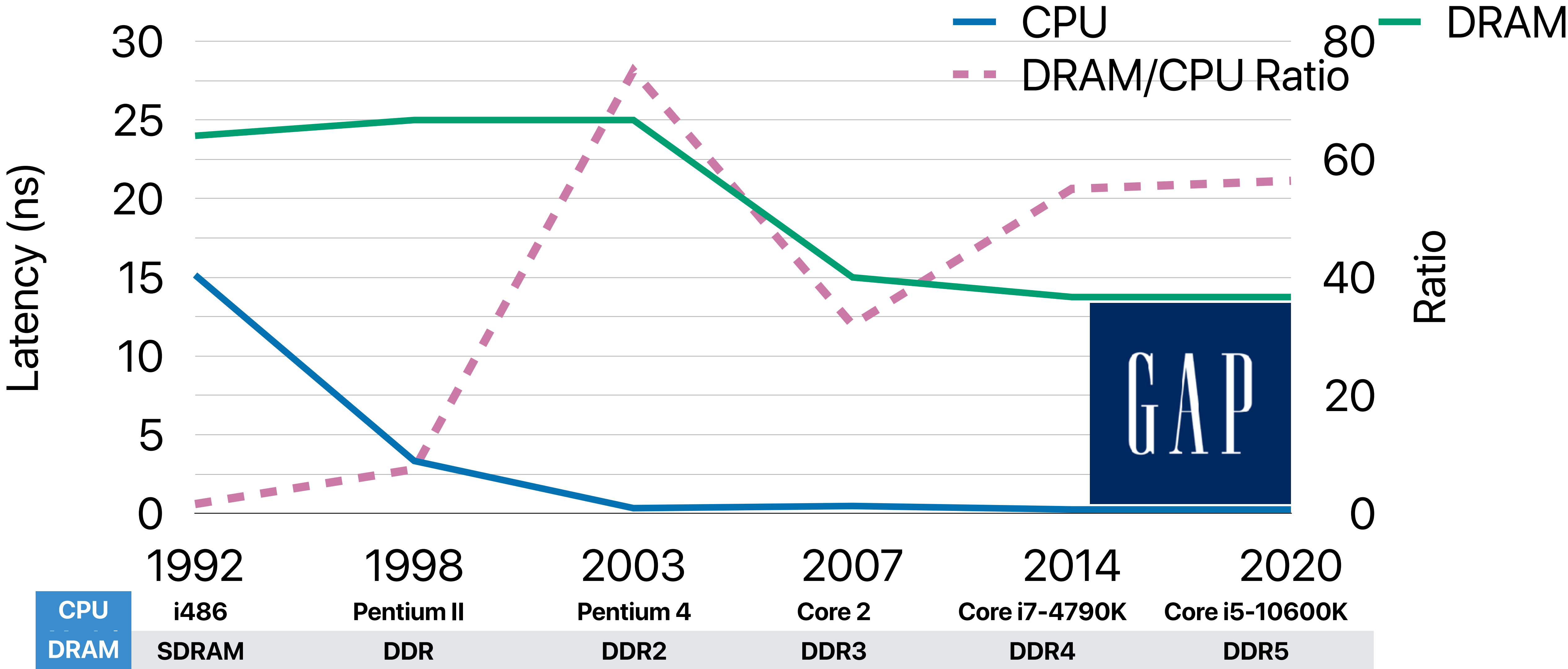
This dictionary, along with its 1993 Addenda Section, includes around 470,000 entries.

You only need to know 2% English words to understand 90% of conversations

Outline

- The memory-wall problem
- The “predictable” code behavior
- Designing a cache that captures the predictability

The "latency" gap between CPU and DRAM



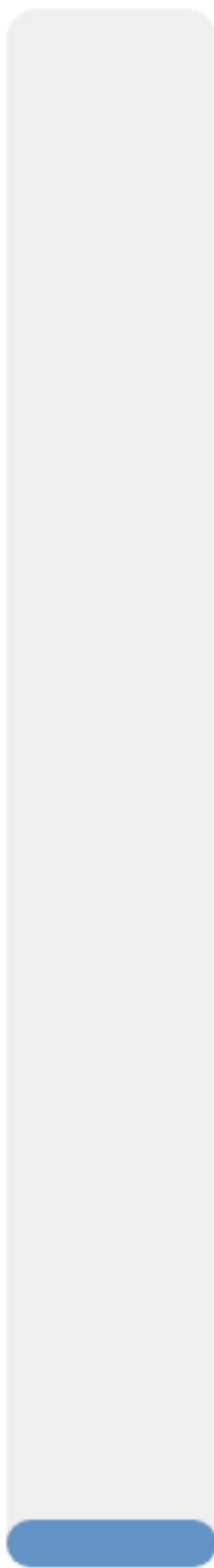


The impact of "slow" memory

- Assume that we have a processor running @ 4 GHz and a program with 20% of load/store instructions. If the instruction has no memory access and the processor already fetches the instruction, the CPI is just 1. Now, consider we have DDR5. The program is well-optimized so precharge is never necessary — the memory access latency is 13.75 ns. What's the average CPI (pick the closest one)?
 - A. 5
 - B. 10
 - C. 30
 - D. 50
 - E. 70

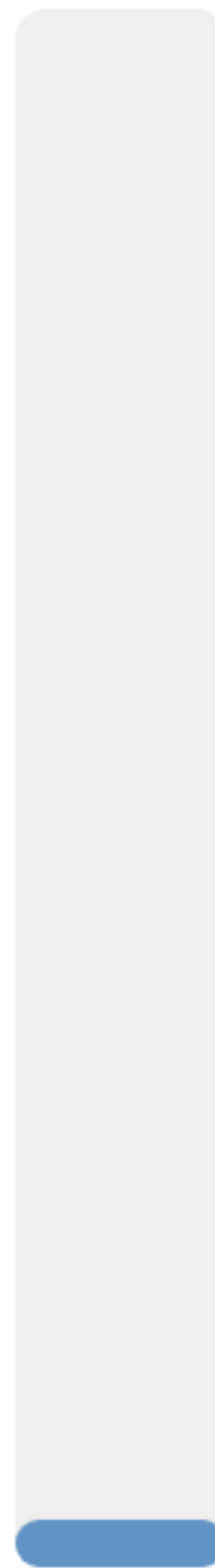
A screenshot of a Pollev poll interface. It shows five horizontal input fields labeled A, B, C, D, and E, stacked vertically. The fields are currently empty, suggesting a multiple-choice or open-ended poll question.

0



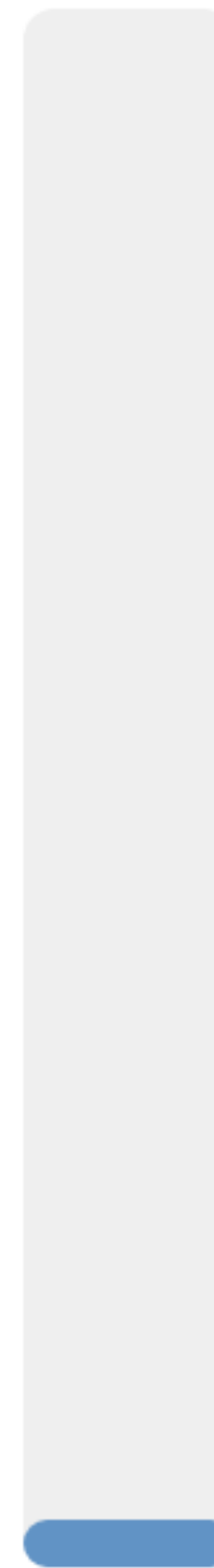
A

0



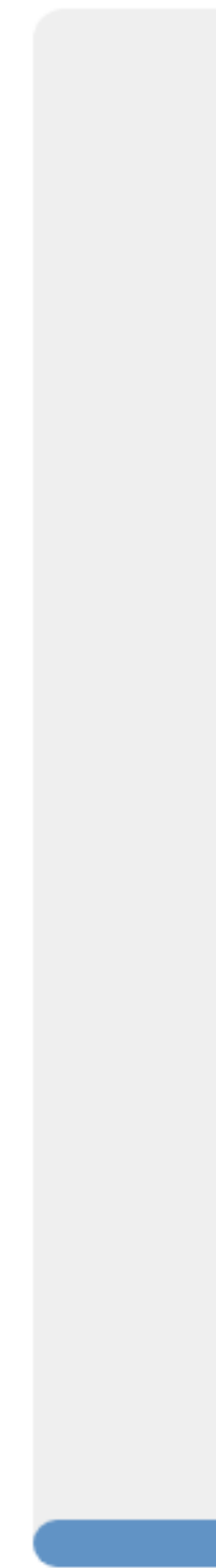
B

0



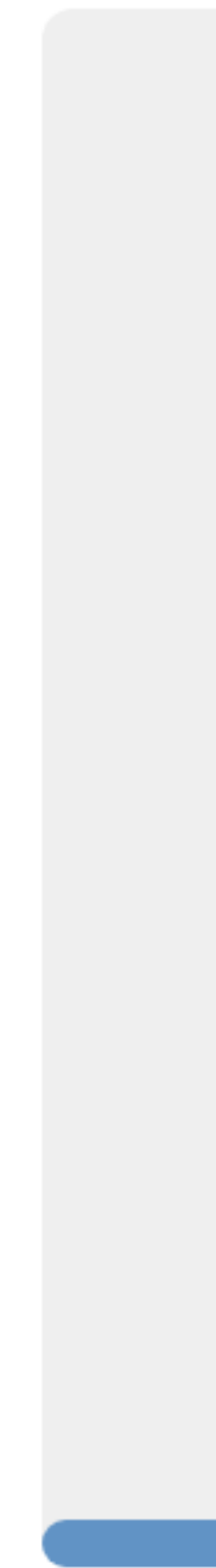
C

0



D

0



E

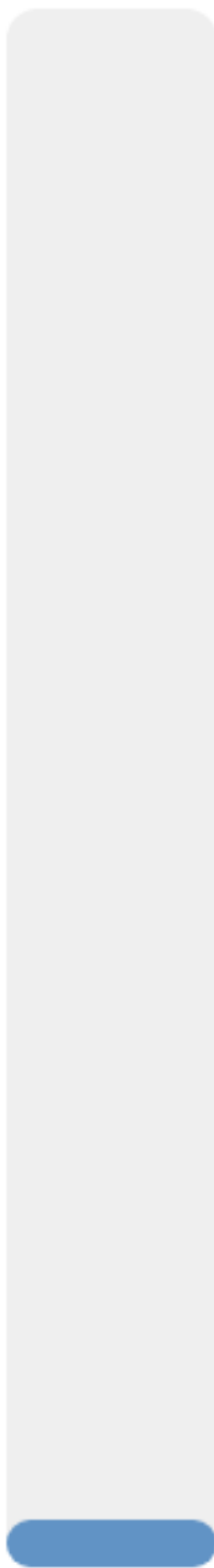


The impact of "slow" memory

- Assume that we have a processor running @ 4 GHz and a program with 20% of load/store instructions. If the instruction has no memory access and the processor already fetches the instruction, the CPI is just 1. Now, consider we have DDR5. The program is well-optimized so precharge is never necessary — the memory access latency is 13.75 ns. What's the average CPI (pick the closest one)?
 - A. 5
 - B. 10
 - C. 30
 - D. 50
 - E. 70

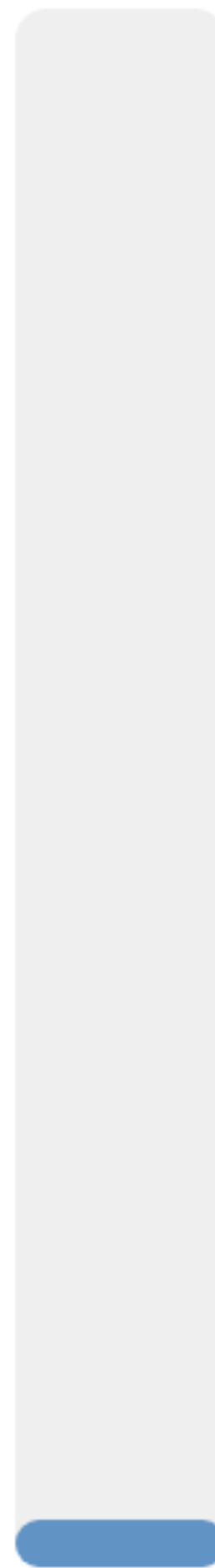
A screenshot of a Pollev poll interface. It shows five horizontal input fields labeled A, B, C, D, and E, stacked vertically. The fields are currently empty, suggesting a multiple-choice question where users select an answer.

0



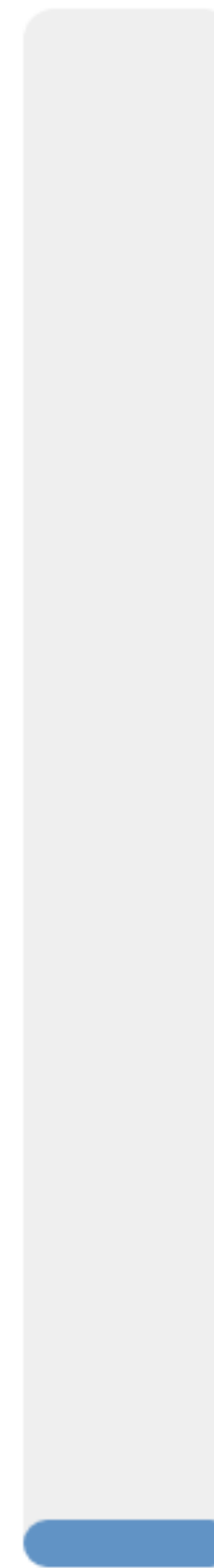
A

0



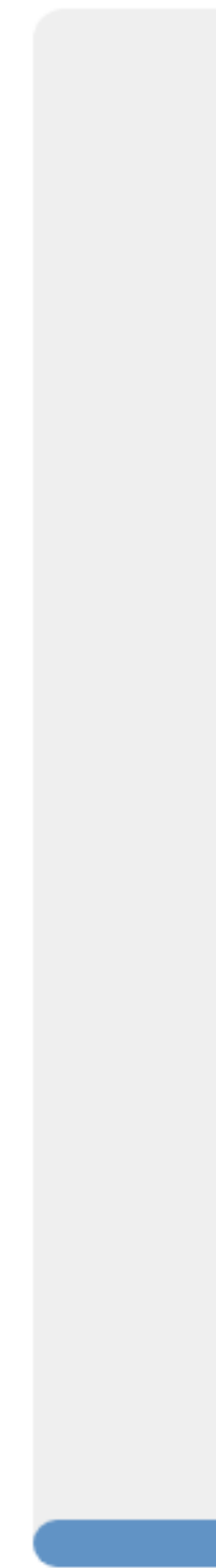
B

0



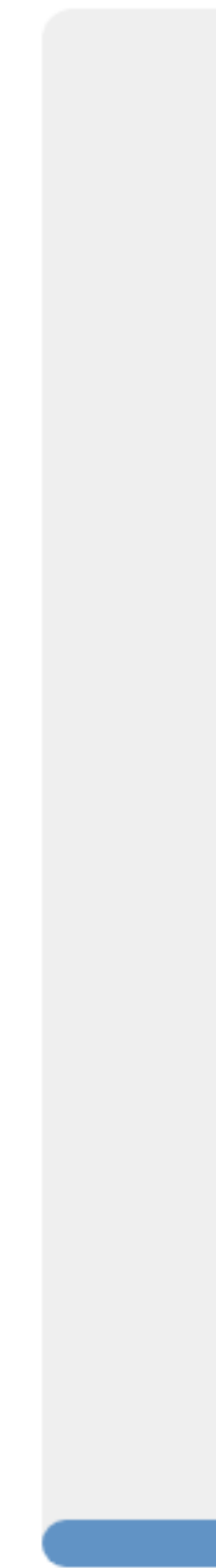
C

0



D

0



E

The impact of "slow" memory

- Assume that we have a processor running @ 4 GHz and a program with 20% of load/store instructions. If the instruction has no memory access and the processor already fetches the instruction, the CPI is just 1. Now, consider we have DDR5. The program is well-optimized so precharge is never necessary — the memory access latency is 13.75 ns. What's the average CPI (pick the closest one)?

A. 5

B. 10

C. 30

D. 50

E. 70

$$CPU \text{ cycle time} = \frac{1}{4 \times 10^9} = 0.25ns$$

$$Each \text{ DRAM access} = \frac{13.75}{0.25} = 55 \text{ cycles}$$

$$CPI_{average} = 1 + 100\% \times 55 + 20\% \times 55 = 67 \text{ cycles}$$

Don't forget, instructions are also from "memory"

$$\frac{66}{67} = 98.5\% \text{ of time, we're dealing with memory accesses!}$$

20% is under-estimating ...

Instruction class	MIPS examples	HLL correspondence	Frequency	
			Integer	Ft. pt.
Arithmetic	add, sub, addi	Operations in assignment statements	16%	48%
Data transfer	lw, sw, lb, lbu, lh, lhu, sb, lui	References to data structures, such as arrays	35%	36%
Logical	and, or, nor, andi, ori, sll, srl	Operations in assignment statements	12%	4%
Conditional branch	beq, bne, slt, slti, sltiu	If statements and loops	34%	8%
Jump	j, jr, jal	Procedure calls, returns, and case/switch statements	2%	0%

FIGURE 2.48 MIPS instruction classes, examples, correspondence to high-level program language constructs, and percentage of MIPS instructions executed by category for the average integer and floating point SPEC CPU2006 benchmarks.

Figure 3.24 in Chapter 3 shows average percentage of the individual MIPS instructions executed.

Recap: Speedup and Amdahl's Law?

- Definition of "Speedup of Y over X" or say Y is n times faster than X: $speedup_{Y_over_X} = n = \frac{Execution\ Time_X}{Execution\ Time_Y}$

- Amdahl's Law — $Speedup_{enhanced}(f, s) = \frac{1}{(1-f) + \frac{f}{s}}$

$$Speedup_{max}(f, \infty) = \frac{1}{(1-f)}$$

- Corollary 1 — each optimization has an upper bound

- **Corollary 2 — make the common case (the most time consuming case) fast!**

$$Speedup_{max}(f_1, \infty) = \frac{1}{(1-f_1)}$$

$$Speedup_{max}(f_2, \infty) = \frac{1}{(1-f_2)}$$

$$Speedup_{max}(f_3, \infty) = \frac{1}{(1-f_3)}$$

$$Speedup_{max}(f_4, \infty) = \frac{1}{(1-f_4)}$$

- Corollary 3: Optimization has a moving target
- Corollary 4: Exploiting more parallelism from a program is the key to performance gain in modern architectures
- Corollary 5: Single-core performance still matters

$$Speedup_{parallel}(f_{parallelizable}, \infty) = \frac{1}{(1-f_{parallelizable})}$$

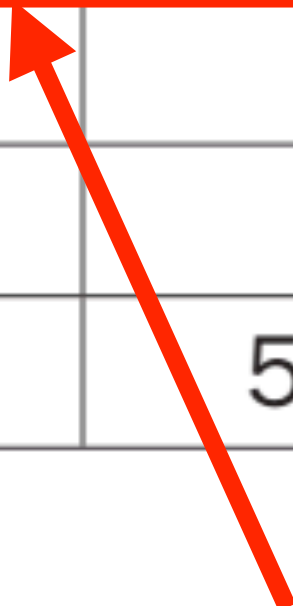
$$Speedup_{parallel}(f_{parallelizable}, \infty) = \frac{1}{(1-f_{parallelizable})}$$

Take-aways: inside out our memory hierarchy

- Memory access time is the most critical performance problem
 - One memory operation is as expensive as 50 arithmetic operations
 - Processor has to fetch instructions from memory
 - We have an average of 33% of data memory access instructions!

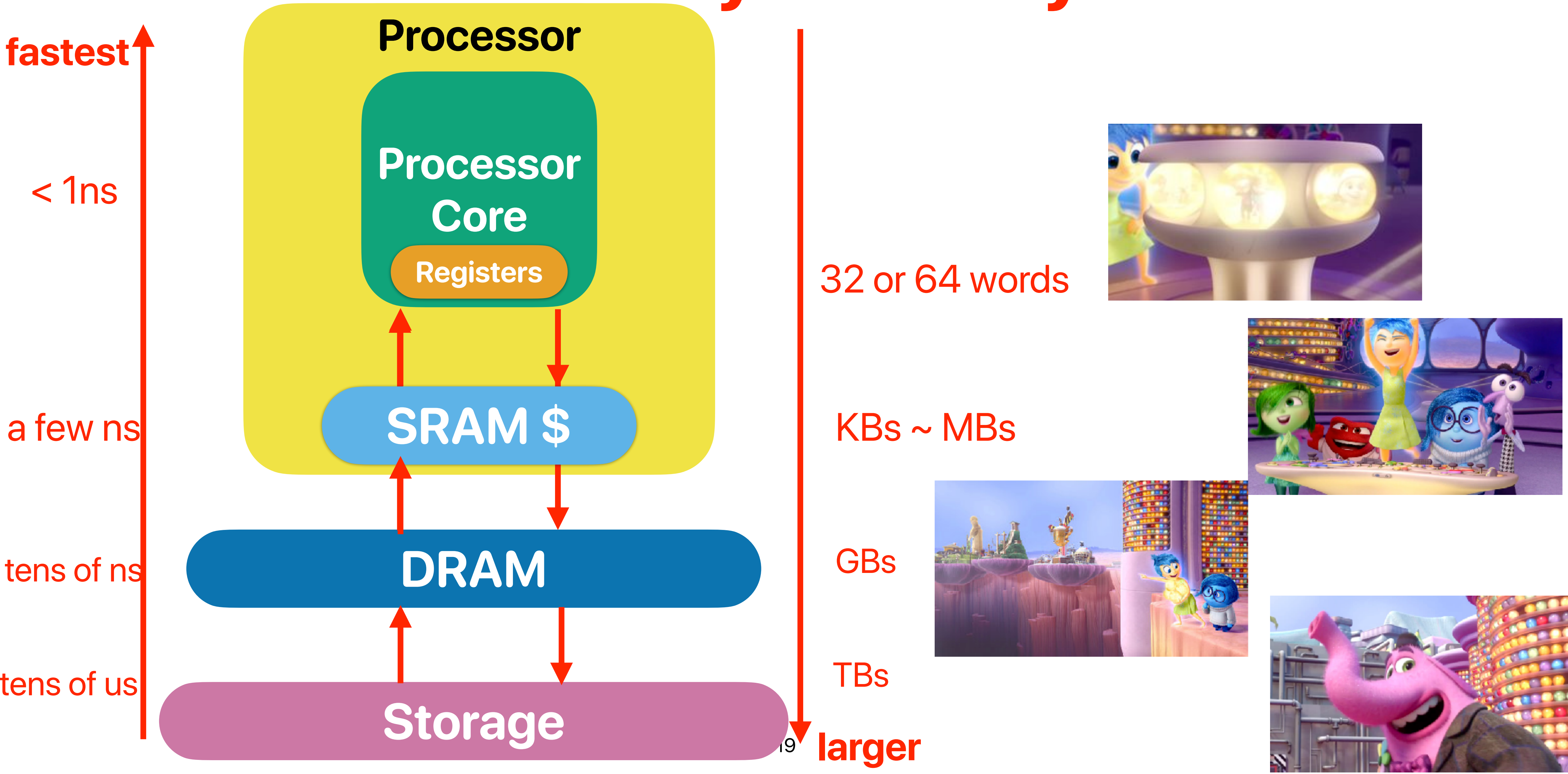
Alternatives?

Memory technology	Typical access time	\$ per GiB in 2012
SRAM semiconductor memory	0.5–2.5 ns	\$500–\$1000
DRAM semiconductor memory	50–70 ns	\$10–\$20
Flash semiconductor memory	5,000–50,000 ns	\$0.75–\$1.00
Magnetic disk	5,000,000–20,000,000 ns	\$0.05–\$0.10



Fast, but expensive \$\$\$

Memory Hierarchy





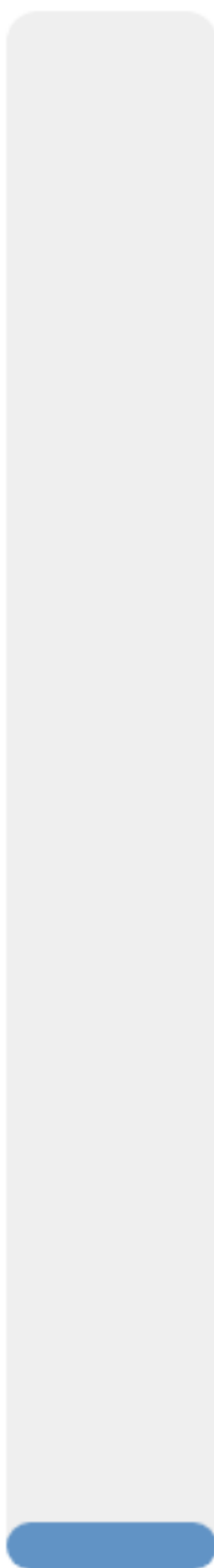
How can "memory hierarchy" help in performance?

- Assume that we have a processor running @ 4 GHz and a program with 20% of load/store instructions. If the instruction has no memory access, the CPI is just 1. Now, in addition to we DDR5, whose latency 13.75 ns, we also got an SRAM cache with latency of just at 0.5 ns and can capture **90%** of the desired data/instructions. what's the average CPI (pick the closest one)?

- A. 5
- B. 10
- C. 30
- D. 50
- E. 70

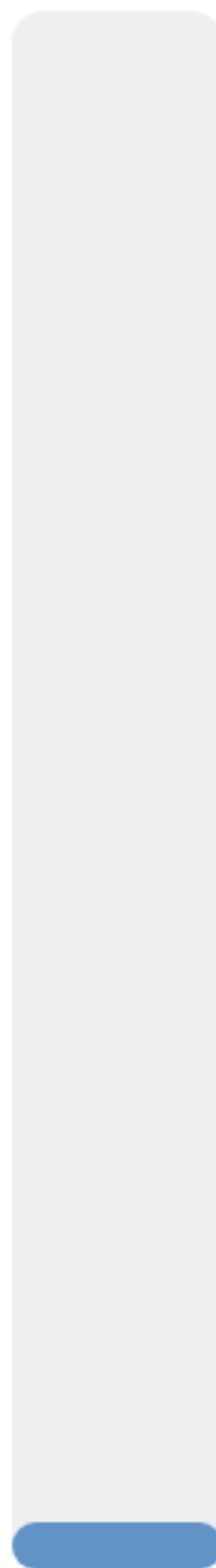
A screenshot of a Pollev poll interface. It shows five horizontal input fields, each with a letter label (A, B, C, D, E) on the left. The fields are currently empty, suggesting a multiple-choice poll where users select an answer.

0



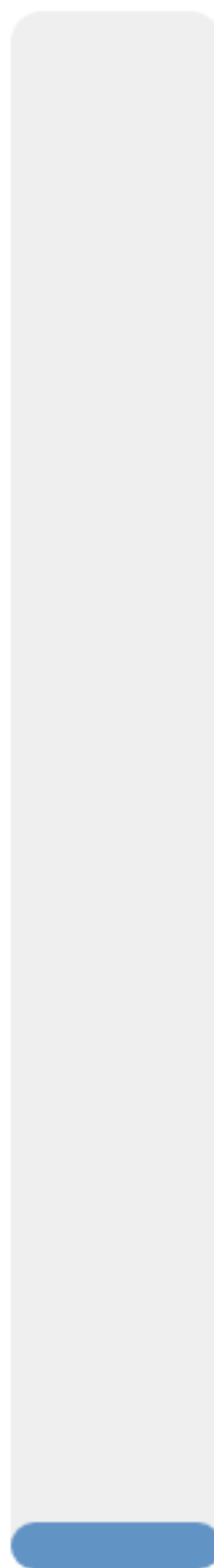
A

0



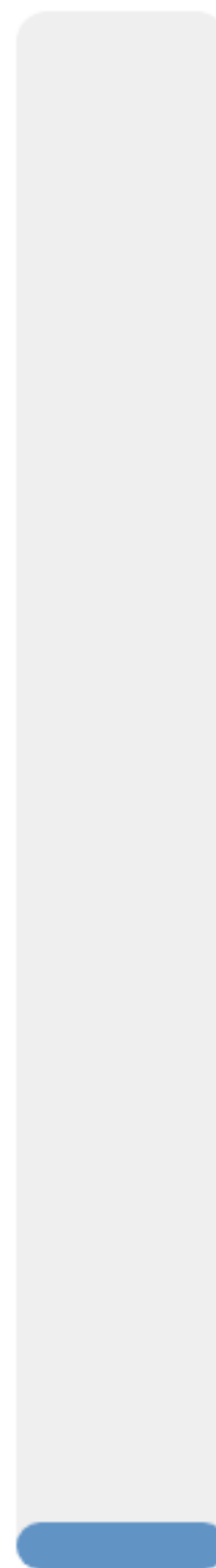
B

0



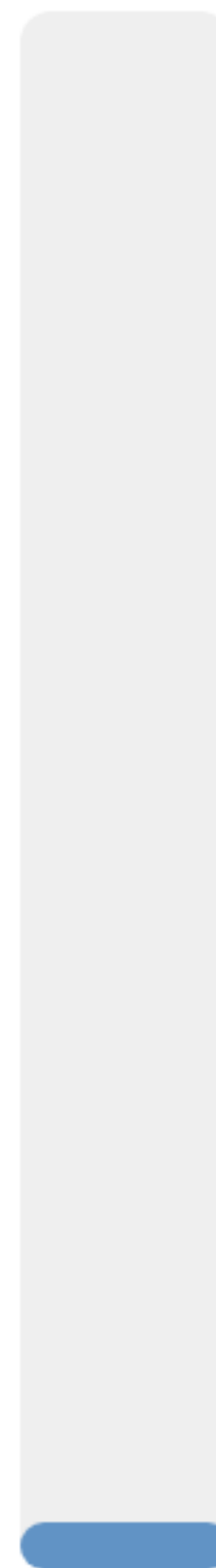
C

0



D

0



E



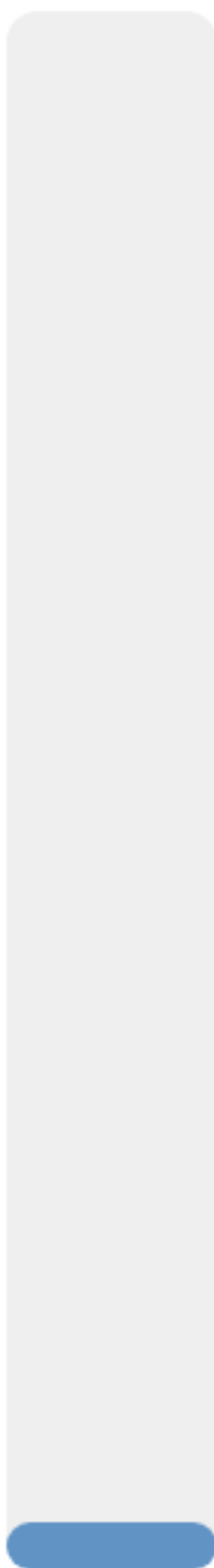
How can "memory hierarchy" help in performance?

- Assume that we have a processor running @ 4 GHz and a program with 20% of load/store instructions. If the instruction has no memory access, the CPI is just 1. Now, in addition to we DDR5, whose latency 13.75 ns, we also got an SRAM cache with latency of just at 0.5 ns and can capture **90%** of the desired data/instructions. what's the average CPI (pick the closest one)?

- A. 5
- B. 10
- C. 30
- D. 50
- E. 70

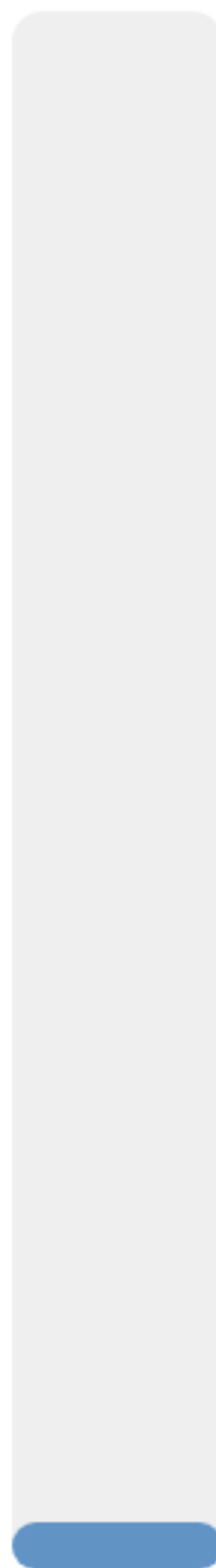
A screenshot of a Pollev poll interface. It shows five horizontal input fields labeled A, B, C, D, and E, stacked vertically. The fields are currently empty, suggesting a multiple-choice or open-ended poll question.

0



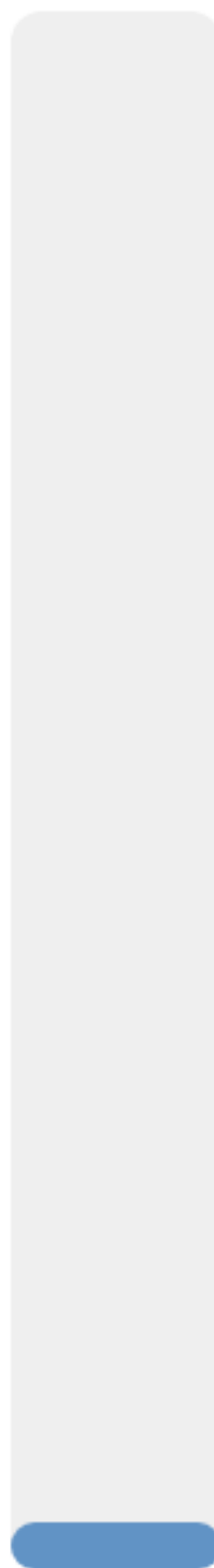
A

0



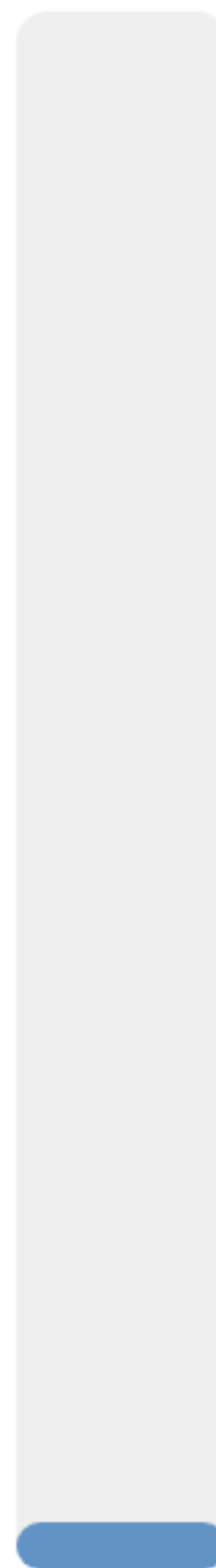
B

0



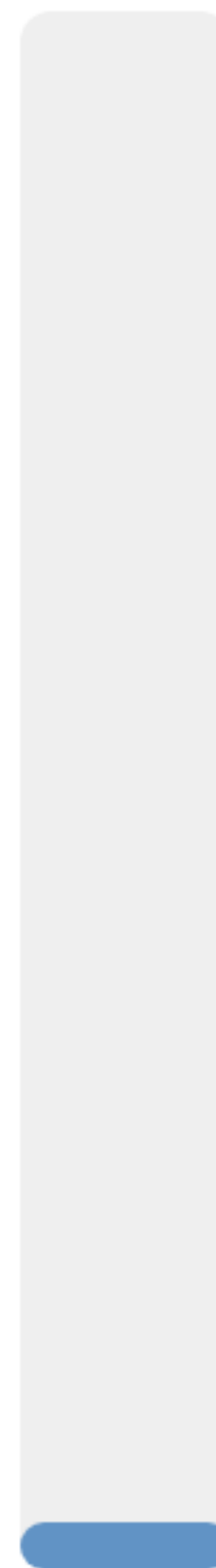
C

0



D

0



E

How can "memory hierarchy" help in performance?

- Assume that we have a processor running @ 4 GHz and a program with 20% of load/store instructions. If the instruction has no memory access, the CPI is just 1. Now, in addition to we DDR5, whose latency 13.75 ns, we also got an SRAM cache with latency of just at 0.5 ns and can capture **90%** of the desired data/instructions. what's the average CPI (pick the closest one)?

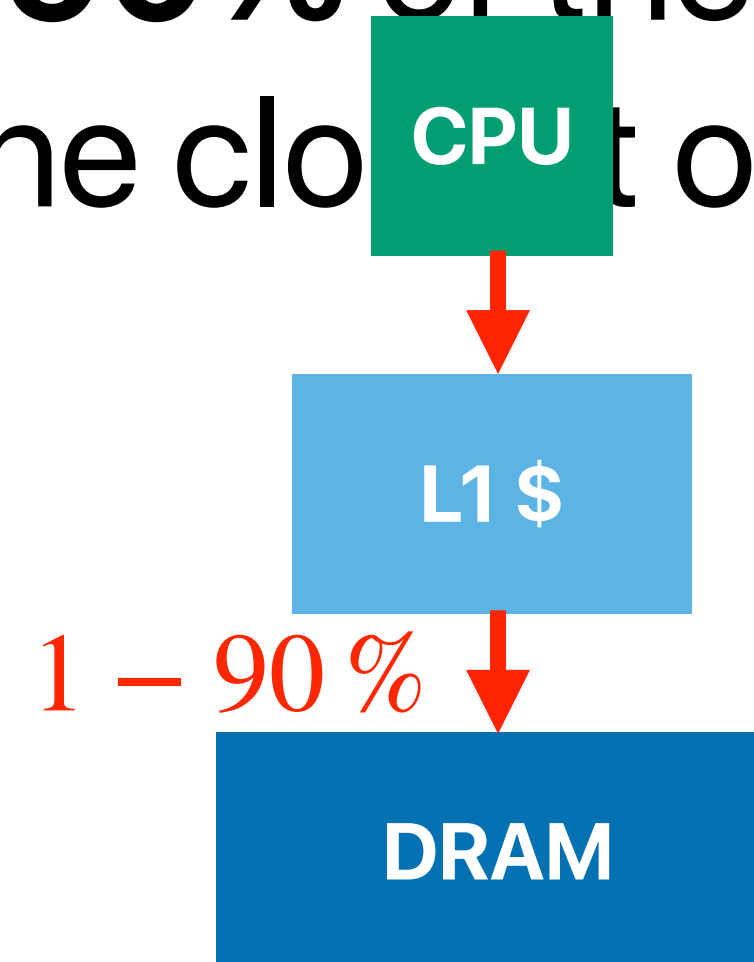
A. 5

B. 10

C. 30

D. 50

E. 70



$$\text{CPU cycle time} = \frac{1}{4 \times 10^9} = 0.25 \text{ ns}$$

$$\text{Each \$ access} = \frac{0.5}{0.25} = 2 \text{ cycles}$$

$$\text{Each DRAM access} = \frac{13.75}{0.25} = 55 \text{ cycles}$$

$$CPI_{\text{average}} = 1 + 100\% \times [2 + (1 - 90\%) \times 55] + 20\% \times [2 + (1 - 90\%) \times 55] = 10 \text{ cycles}$$



L1? L2? L3?

CPU-Z - ID : vfljgr

CPU Mainboard Memory SPD Graphics Bench About

Processor

Name	AMD Ryzen 7 7700X		
Code Name	Raphael	Max TDP	105 W
Package	Socket AM5 (LGA1718)		
Technology	5 nm	Core Voltage	1.288 V



Specification

AMD Ryzen 7 7700X 8-Core Processor			
Family	F	Model	1
Ext. Family	19	Ext. Model	61
Stepping	2	Revision	RPL-B2

Instructions

MMX(+), SSE, SSE2, SSE3, SSSE3, SSE4.1, SSE4.2, SSE4A, x86-64, AMD-V, AES, AVX, AVX2, AVX512, FMA3, SHA

Clocks (Core #0)

Core Speed	5188.99 MHz
Multiplier	x 52.0 (4 - 55.5)
Bus Speed	99.79 MHz
Rated FSB	

Cache

L1 Data	8 x 32 KB
L1 Inst.	8 x 32 KB
Level 2	8 x 1024 KB
Level 3	32 MBytes

Selection Socket #1 Cores 8 Threads 16

CPU-Z Ver. 2.09.0.x64 Tools Validate Close

CPU-Z - ID : pk15b

CPU Mainboard Memory SPD Graphics Bench About

Processor

Name	Intel Core i7 14700K		
Code Name	Raptor Lake	Max TDP	125 W
Package	Socket 1700 LGA		
Technology	10 nm	Core Voltage	1.412 V



Specification

Intel(R) Core(TM) i7-14700K			
Family	6	Model	7
Ext. Family	6	Ext. Model	B7
Stepping	1	Revision	B0

Instructions

MMX, SSE, SSE2, SSE3, SSSE3, SSE4.1, SSE4.2, EM64T, VT-x, AES, AVX, AVX2, FMA3, SHA

Clocks (Core #0)

Core Speed	5287.07 MHz
Multiplier	x 53.0 (8 - 55)
Bus Speed	99.76 MHz
Rated FSB	

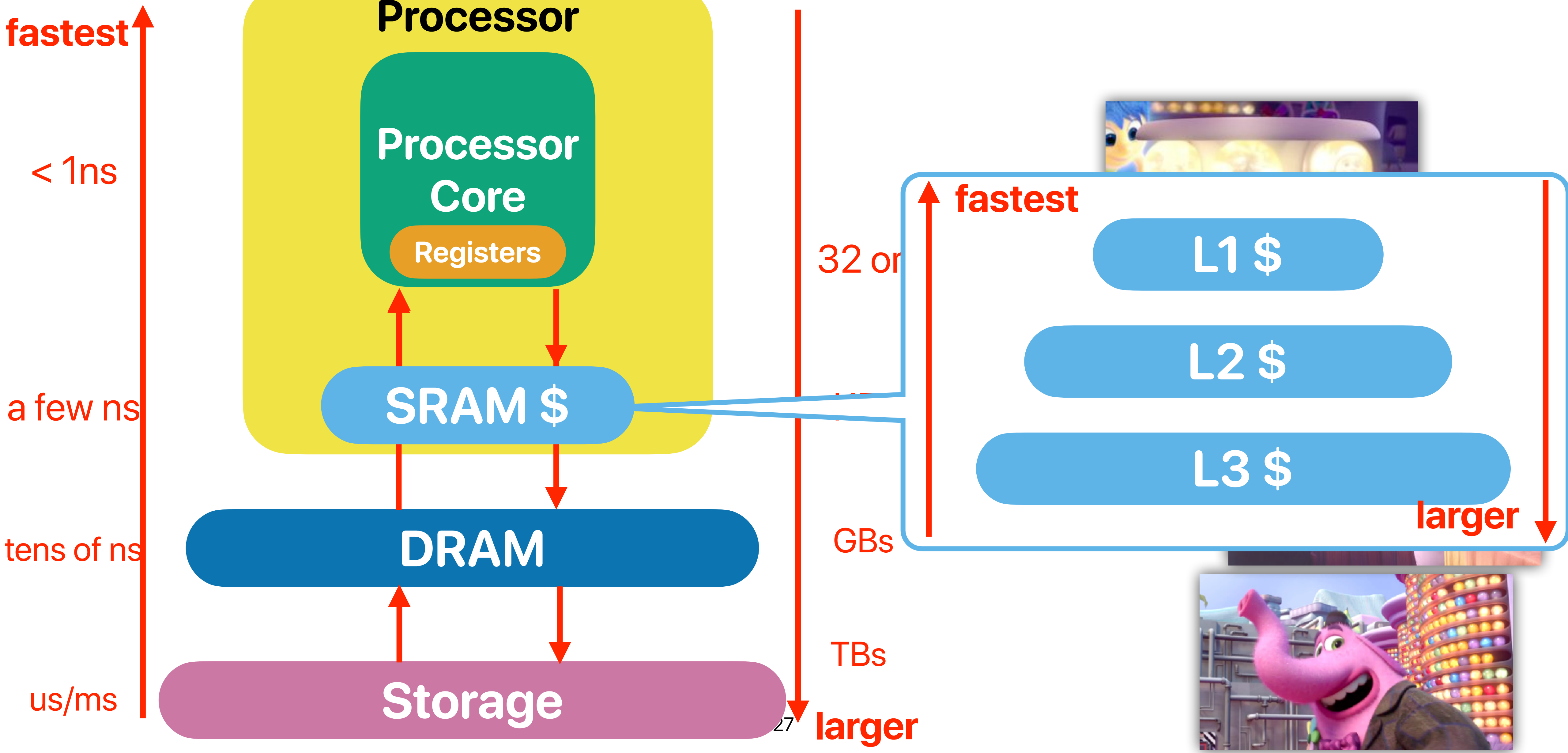
Cache

L1 Data	8 x 48 KB + 12 x 32 KB
L1 Inst.	8 x 32 KB + 12 x 64 KB
Level 2	8 x 2 MB + 3 x 4 MB
Level 3	33 MBytes

Selection Socket #1 Cores 8 + 12 Threads 28

CPU-Z Ver. 2.08.0.x64 Tools Validate Close

Memory Hierarchy





How can a deeper memory hierarchy help in performance?

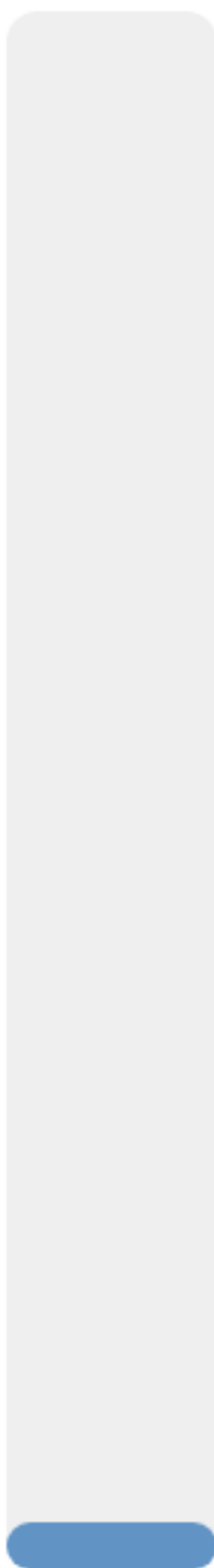
- Assume that we have a processor running @ 4 GHz and a program with 20% of load/store instructions. If the instruction has no memory access, the CPI is just 1. Now, in addition to we DDR5, whose latency 13.75 ns, we also got a 2-level SRAM caches with
 - it's 1st-level one at latency of 0.5ns and can capture 90% of the desired data/instructions.
 - the 2nd-level at latency of 5 ns and can capture 60% of the desired data/instructions

What's the average CPI (pick the closest one)?

- A. 5
- B. 10
- C. 30
- D. 50
- E. 70

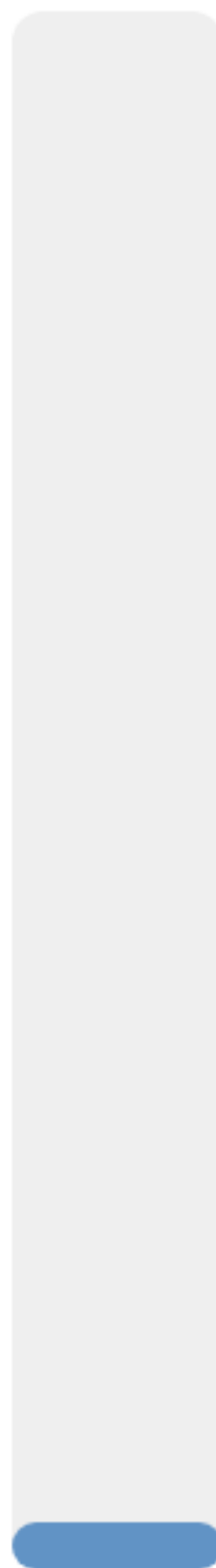
A screenshot of a Pollev poll interface. It shows five empty input boxes, each with a letter (A, B, C, D, E) to its left. The interface is clean and modern, with a light blue header and a white background for the input area.

0



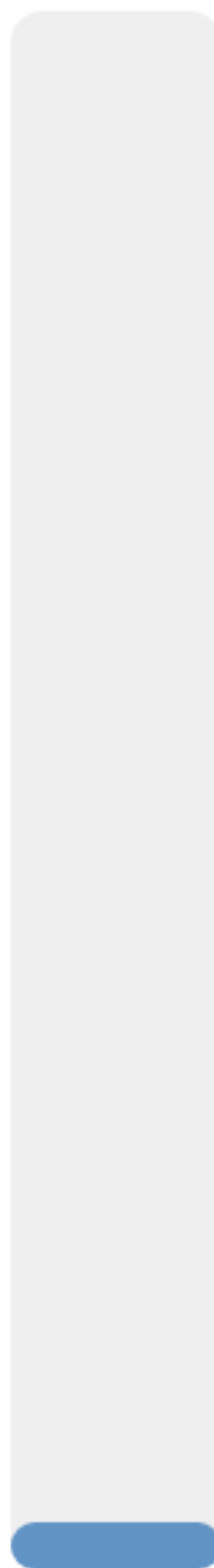
A

0



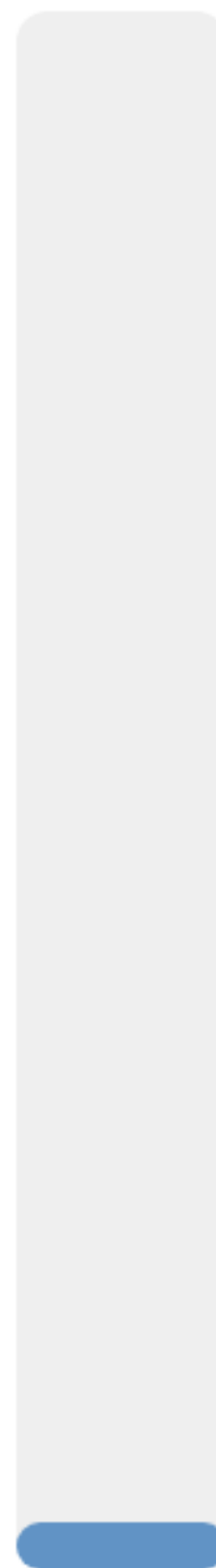
B

0



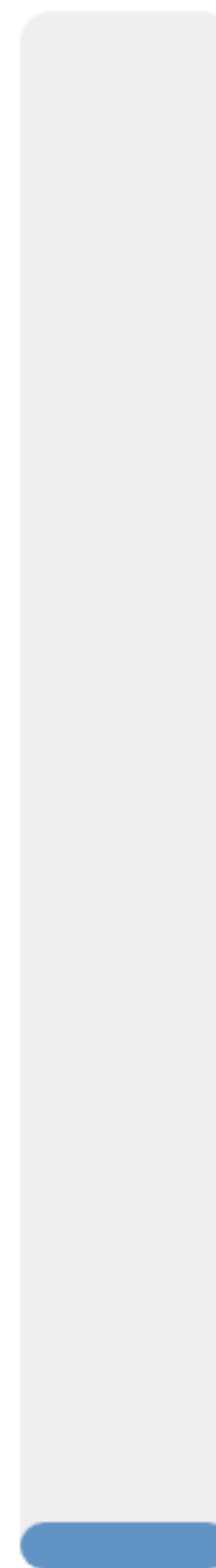
C

0



D

0



E



How can a deeper memory hierarchy help in performance?

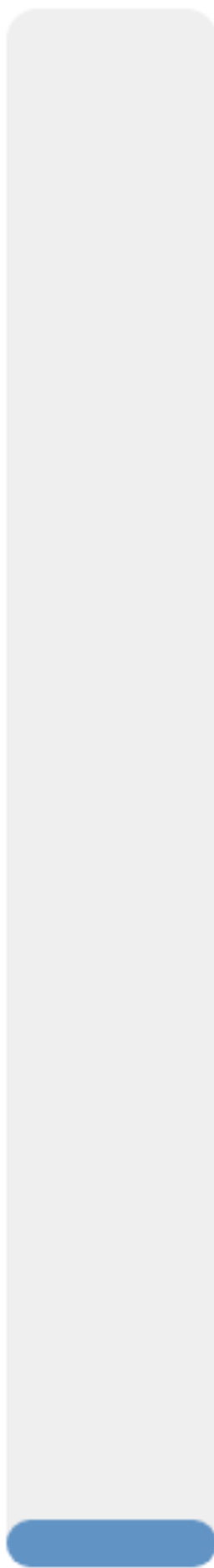
- Assume that we have a processor running @ 4 GHz and a program with 20% of load/store instructions. If the instruction has no memory access, the CPI is just 1. Now, in addition to we DDR5, whose latency 13.75 ns, we also got a 2-level SRAM caches with
 - it's 1st-level one at latency of 0.5ns and can capture 90% of the desired data/instructions.
 - the 2nd-level at latency of 5 ns and can capture 60% of the desired data/instructions

What's the average CPI (pick the closest one)?

- A. 5
- B. 10
- C. 30
- D. 50
- E. 70

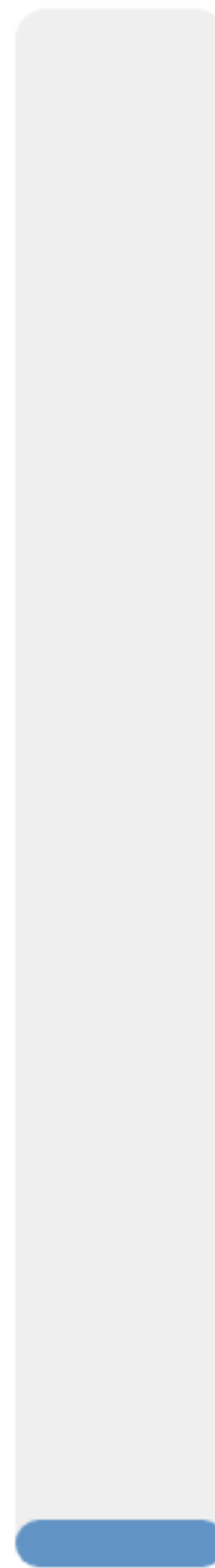
A screenshot of a Pollev poll interface. It shows five horizontal input fields, each with a letter label (A, B, C, D, E) on the left. The fields are currently empty, indicating that no answers have been submitted yet. The interface is clean and modern, with a light blue header and a white background for the poll area.

0



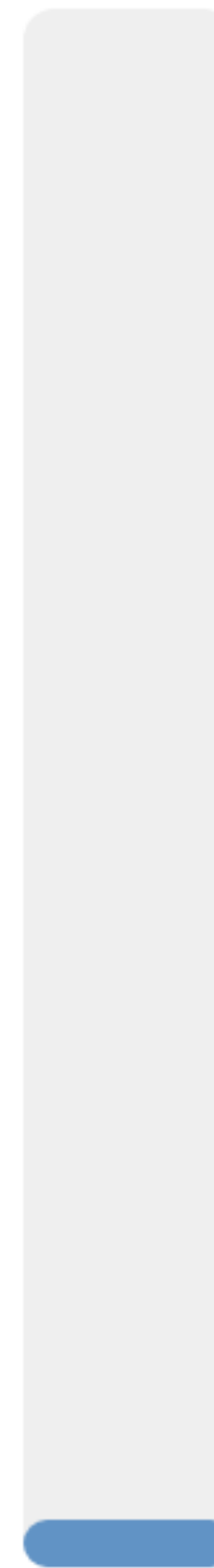
A

0



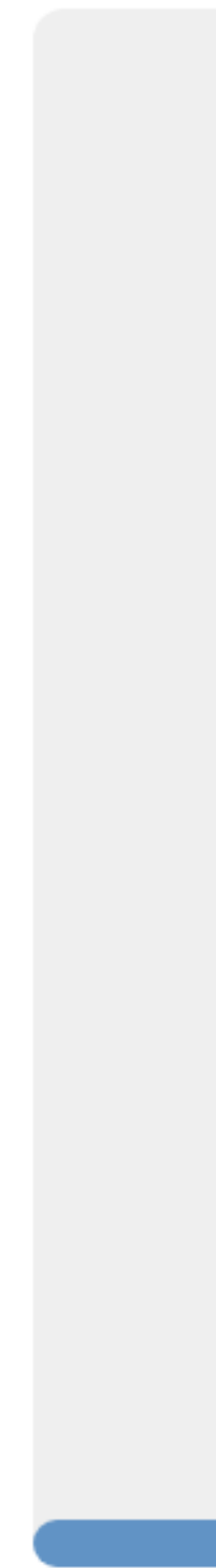
B

0



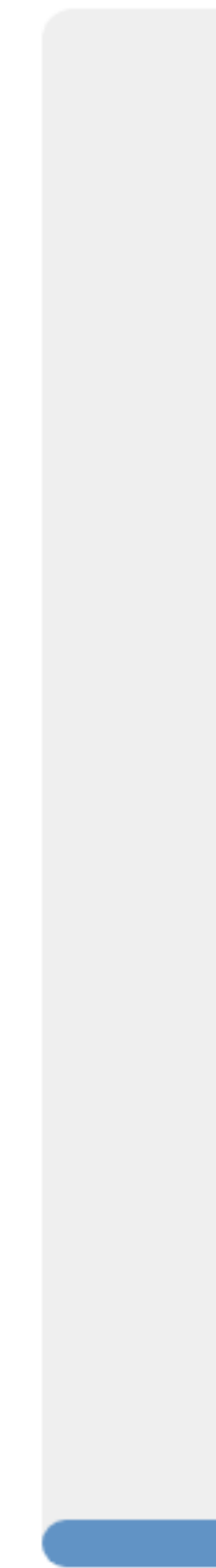
C

0



D

0



E

How can a deeper memory hierarchy help in performance?

- Assume that we have a processor running @ 4 GHz and a program with 20% of load/store instructions. If the instruction has no memory access, the CPI is just 1. Now, in addition to the DDR5, whose latency is 13.75 ns, we also got 2 levels SRAM caches where

- the 1st-level one at latency of 0.5ns and can capture 90% of the desired data/instructions.

- the 2nd-level at latency of 5 ns and can capture 60% of the desired data/instructions

What's the average CPI (pick the closest one)?

A. 5

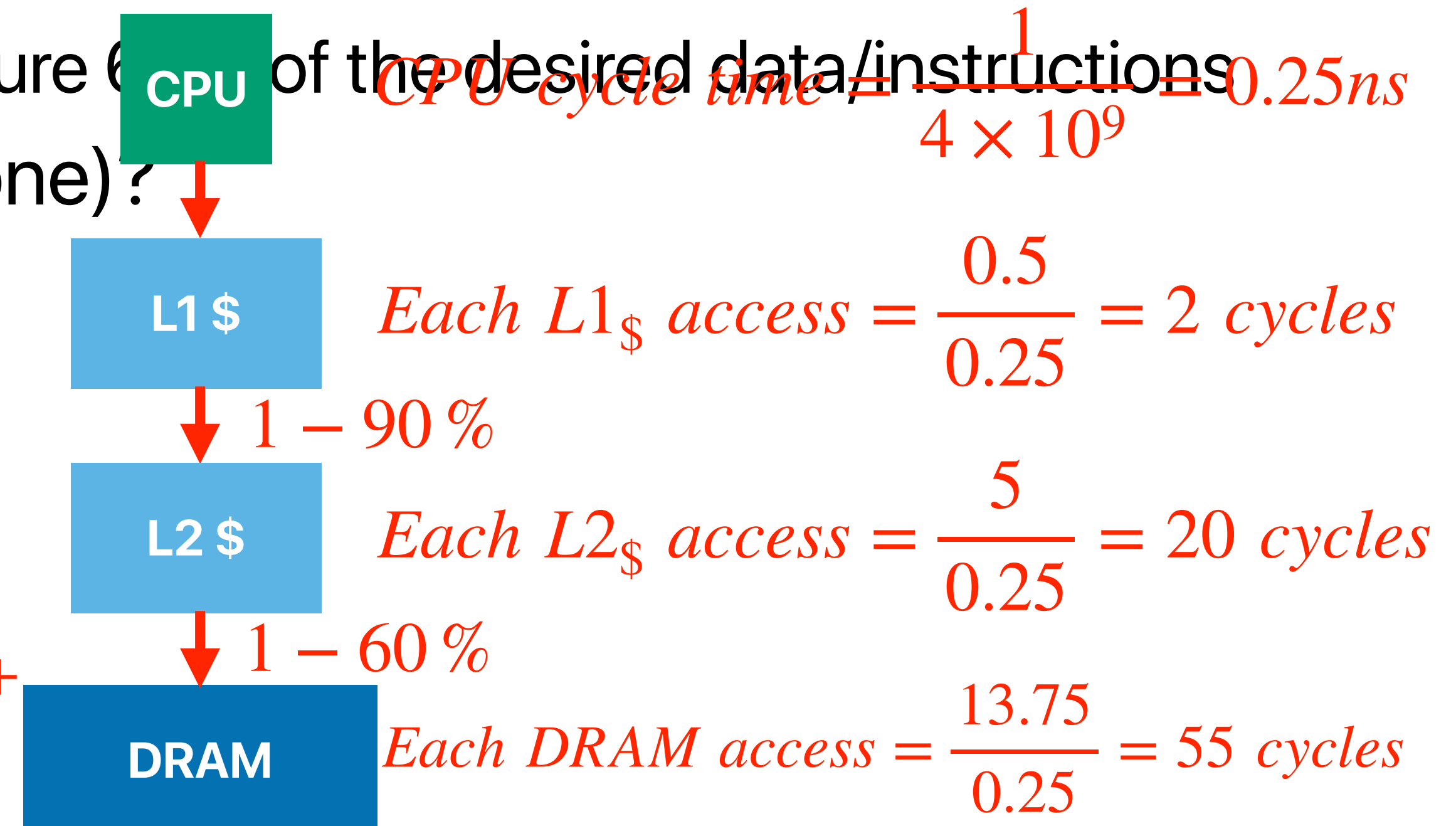
B. 10

C. 30

D. 50

E. 70

$$CPI_{average} = 1 + 100\% \times [2 + (1 - 90\%) \times (20 + (1 - 60\%) \times 55)] + 20\% \times [2 + (1 - 90\%) \times (20 + (1 - 60\%) \times 55)] = 8.44 \text{ cycles}$$



L1? L2? L3?

Can we really “predict” upcoming data accurately (e.g., 90%) with such small caches?

CPU-Z - ID : vfljgr	
CPU Mainboard Memory SPD Graphics Bench About	
Processor	
Name	AMD Ryzen 7 7700X
Code Name	Raphael
Max TDP	105 W
Package	AM5
Technology	5 nm
Core Voltage	1.2 V
Specification	
AMD Ryzen 7 7700X 8-Core Processor	
Ext. Family	19
Ext. Model	1
Revision	RPL/2
Instructions	
MMX(+), SSE, SSE2, SSE3, SSSE3, SSE4.1, SSE4.2, SSE4A, x86-64, AMD-V, AES, AVX, AVX2, AVX512, FMA3, SHA	
Clocks (Core #0)	
Core Speed	5188.99 MHz
Multiplier	x 52.0 (4 - 55.5)
Bus Speed	99.79 MHz
Rated FSB	
Cache	
L1 Data	8 x 32 KB
L1 Inst.	8 x 32 KB
Level 2	8 x 1024 KB
Level 3	32 MBytes
Selection Socket #1	
Cores	8
Threads	16
CPU-Z Ver. 2.09.0.x64 Tools Validate Close	

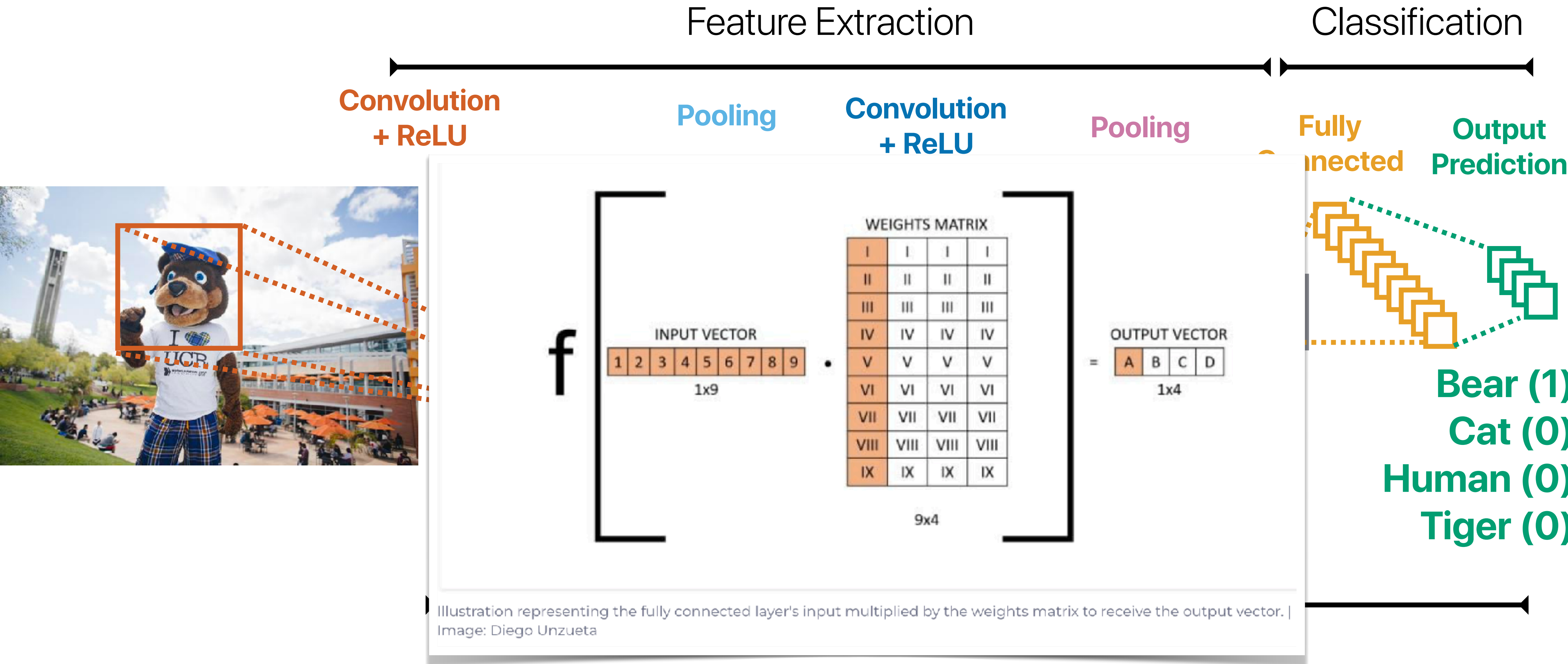
CPU-Z - ID : pk15b	
CPU Mainboard Memory SPD Graphics Bench About	
Processor	
Name	Intel Core i7 14700K
Code Name	Raptor Lake
Max TDP	125 W
Package	LGA1700
Technology	14 nm
Core Voltage	1.1 V
Specification	
Intel(R) Core(TM) i7-14700K	
Ext. Family	6
Ext. Model	67
Revision	B0
Instructions	
MMX, SSE, SSE2, SSE3, SSSE3, SSE4.1, SSE4.2, EM64T, VT-x, AES, AVX, AVX2, FMA3, SHA	
Clocks (Core #0)	
Core Speed	5287.07 MHz
Multiplier	x 53.0 (8 - 55)
Bus Speed	99.76 MHz
Rated FSB	
Cache	
L1 Data	8 x 48 KB + 12 x 32 KB
L1 Inst.	8 x 32 KB + 12 x 64 KB
Level 2	8 x 2 MB + 3 x 4 MB
Level 3	33 MBytes
Selection Socket #1	
Cores	8 + 12
Threads	28
CPU-Z Ver. 2.08.0.x64 Tools Validate Close	

Take-aways: inside out our memory hierarchy

- Memory access time is the most critical performance problem
 - One memory operation is as expensive as 50 arithmetic operations
 - Processor has to fetch instructions from memory
 - We have an average of 33% of data memory access instructions!
- Hierarchical caching with small amount of SRAMs will work if we can efficiently capture data and instructions

The predictability of your code

The Machine Learning Inference Pipeline





Locality of data

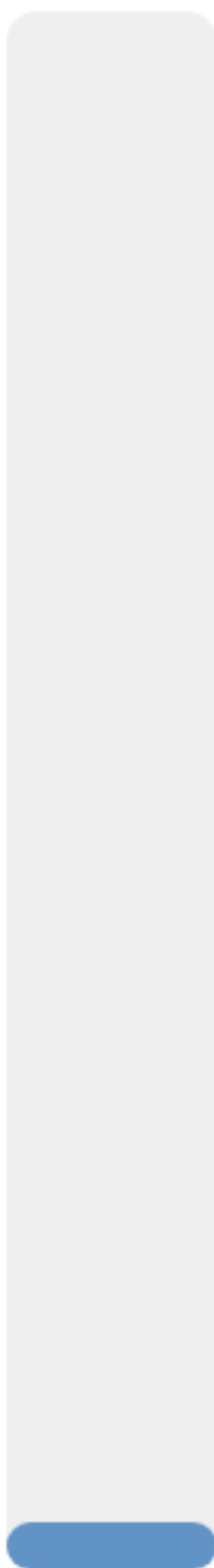
- Which description about locality of arrays `matrix` and `vector` in the following code is the **most accurate**?

```
for(uint64_t i = 0; i < m; i++) {  
    result = 0;  
    for(uint64_t j = 0; j < n; j++) {  
        result += matrix[i][j]*vector[j];  
    }  
    output[i] = result;  
}
```



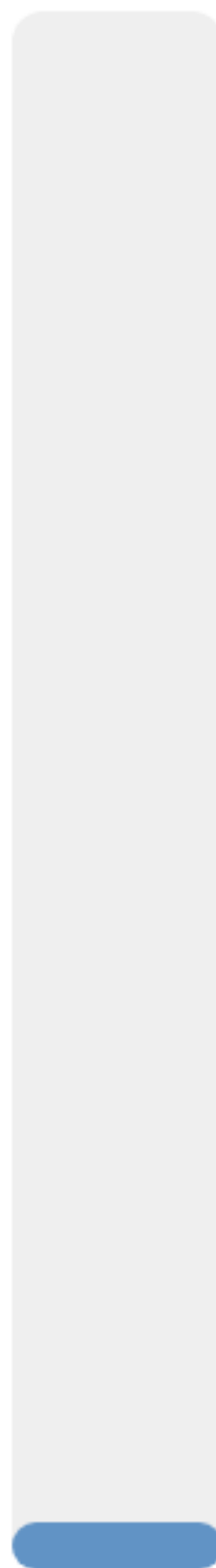
- A. Access of `matrix` has temporal locality, `vector` has spatial locality
- B. Both `matrix` and `vector` have temporal locality, and `vector` also has spatial locality
- C. Access of `matrix` has spatial locality, `vector` has temporal locality
- D. Both `matrix` and `vector` have spatial locality and temporal locality
- E. Both `matrix` and `vector` have spatial locality, and `vector` also has temporal locality

0



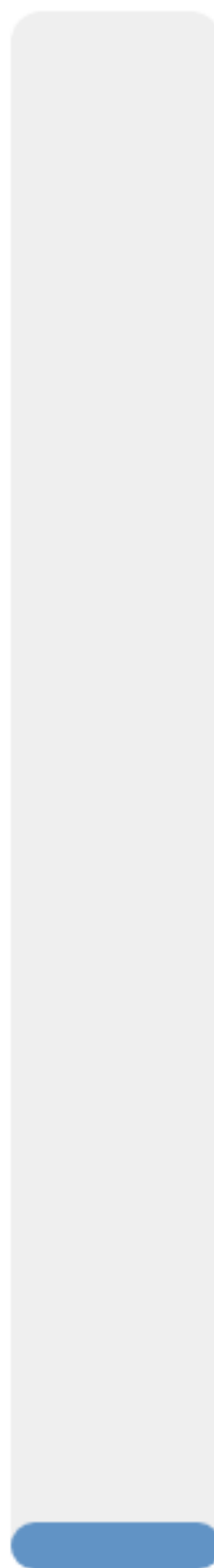
A

0



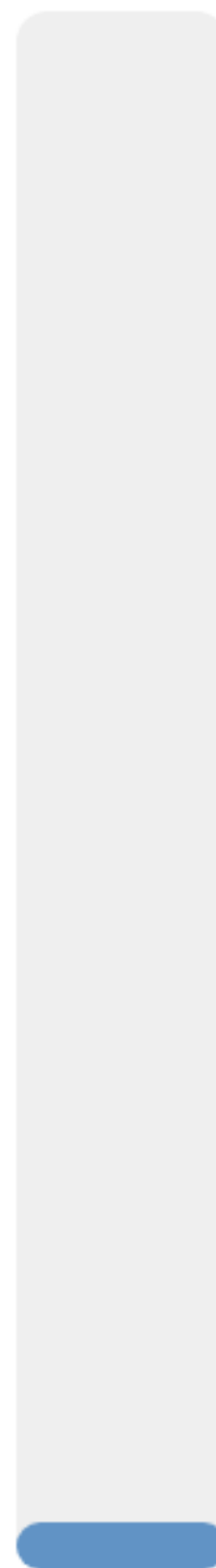
B

0



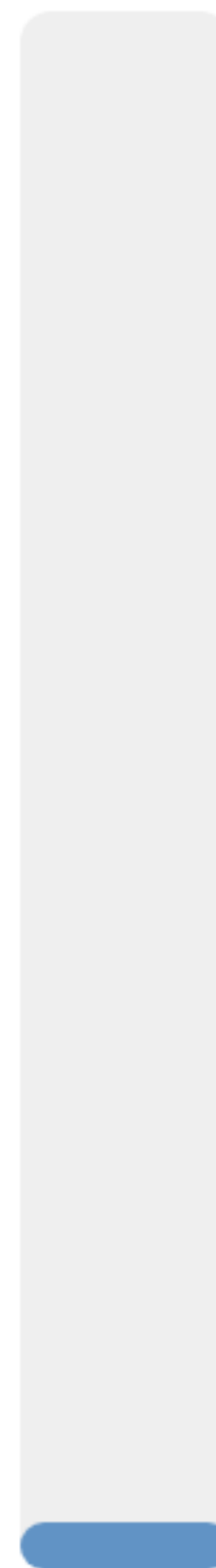
C

0



D

0



E



Locality of data

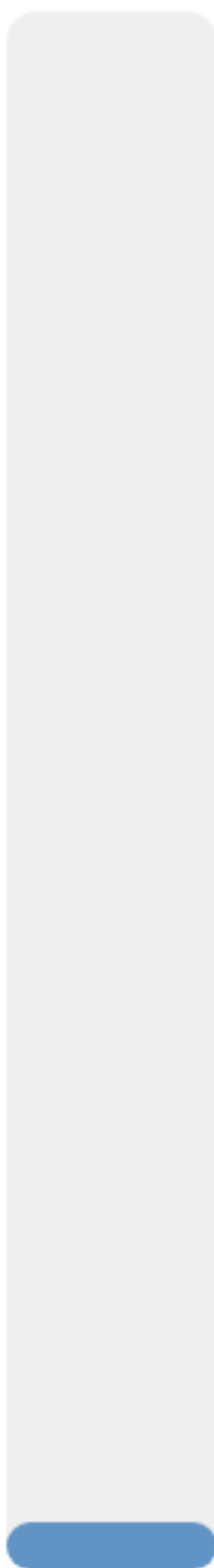
- Which description about locality of arrays `matrix` and `vector` in the following code is the **most accurate**?

```
for(uint64_t i = 0; i < m; i++) {  
    result = 0;  
    for(uint64_t j = 0; j < n; j++) {  
        result += matrix[i][j]*vector[j];  
    }  
    output[i] = result;  
}
```



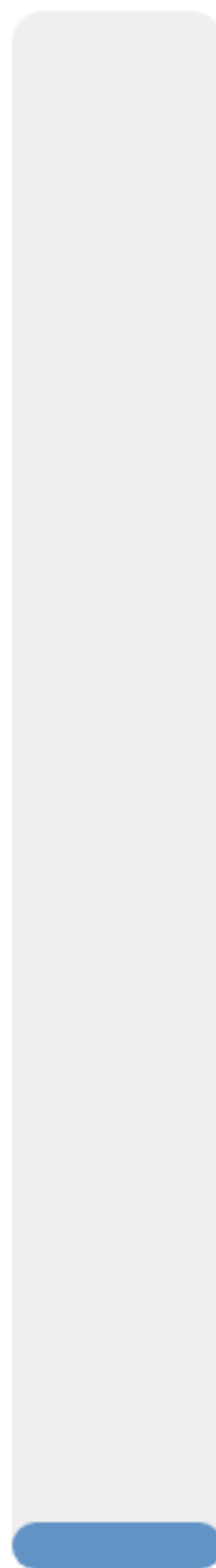
- A. Access of `matrix` has temporal locality, `vector` has spatial locality
- B. Both `matrix` and `vector` have temporal locality, and `vector` also has spatial locality
- C. Access of `matrix` has spatial locality, `vector` has temporal locality
- D. Both `matrix` and `vector` have spatial locality and temporal locality
- E. Both `matrix` and `vector` have spatial locality, and `vector` also has temporal locality

0



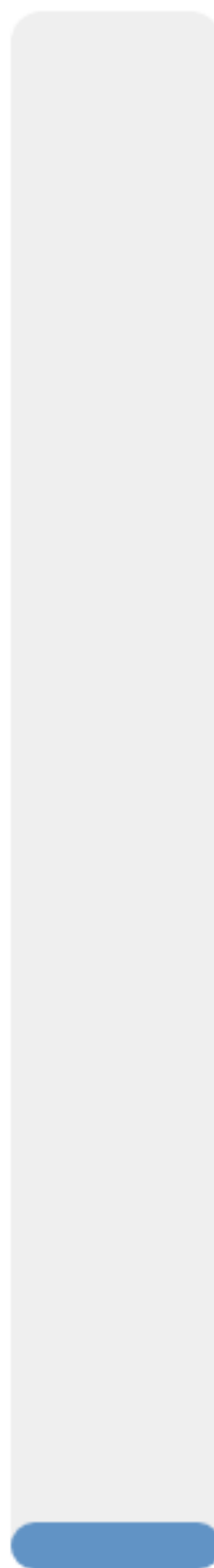
A

0



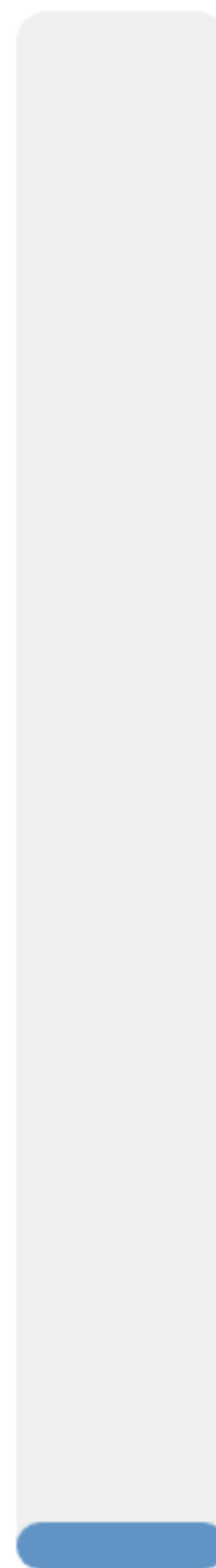
B

0



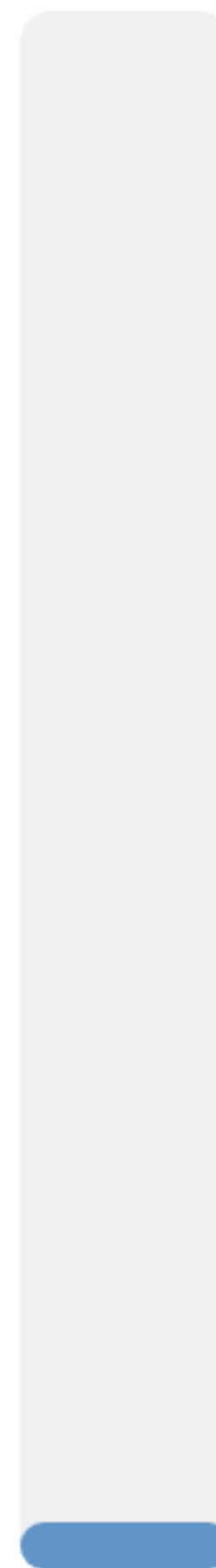
C

0



D

0



E

Data locality

- Which description about locality of arrays `matrix` and `vector` in the following code is the **most accurate**?

```
for(uint64_t i = 0; i < m; i++) {  
    result = 0;  
    for(uint64_t j = 0; j < n; j++) {  
        result += matrix[i][j]*vector[j];  
    }  
    output[i] = result;  
}
```

spatial locality:

`matrix[0][0], matrix[0][1], matrix[0][2], ...`

`vector[0], vector[1], ..., vector[n]`

temporal locality:

reuse of `vector[0], vector[1], ...,`

- A. Access of `matrix` has temporal locality, `vector` has spatial locality
- B. Both `matrix` and `vector` have temporal locality, and `vector` also has spatial locality
- C. Access of `matrix` has spatial locality, `vector` has temporal locality
- D. Both `matrix` and `vector` have spatial locality and temporal locality
- E. Both `matrix` and `vector` have spatial locality, and `vector` also has temporal locality



Data/instructions in the memory

Memory

2⁶⁴ Bytes

0x0000000000000000	i = 0; (i < m)
0x0000000000000008	result = 0; j = 0;
0x0000000000000010	(j < n)
0x0000000000000018	a = matrix[i][j]; b =
0x0000000000000020	vector[j]; temp = a*b;
0x0000000000000028	result = result + temp;
0x0000000000000030	
0x0000000000000038	output[i]=result;i++;
0x0000000000000040	
0x0000000000000048	vector[0]
0x0000000000000050	vector[1]
0x0000000000000058	vector[2]
0x0000000000000060	vector[3]
0x0000000000000068	vector[4]
0x0000000000000070	vector[5]
0x0000000000000078	vector[6]
0x0000000000000080	vector[7]
0x0000000000000088	vector[8]
0x0000000000000090	vector[9]
0x0000000000000098	
0x00000000000000A0	matrix[0][0]
0x00000000000000A8	matrix[0][1]
0x00000000000000B0	matrix[0][2]
0x00000000000000B8	matrix[0][3]
	matrix[0][4]
	matrix[0][5]

⋮

⋮

0xFFFFFFFFFFFFF98							
0xFFFFFFFFFFFFFA0							
0xFFFFFFFFFFFFFA8							
0xFFFFFFFFFFFFFB0							
0xFFFFFFFFFFFFFB8							
0xFFFFFFFFFFFFFC0							
0xFFFFFFFFFFFFFC8							
0xFFFFFFFFFFFFFD0							
0xFFFFFFFFFFFFFD8							
0xFFFFFFFFFFFFFE0							
0xFFFFFFFFFFFFFE8							
0xFFFFFFFFFFFFFF0							
0xFFFFFFFFFFFFFF8							

64-bit

Code also has locality

keep going to the
next instruction —
spatial locality

```
for(uint64_t i = 0; i < m; i++) {  
    result = 0;  
    for(uint64_t j = 0; j < n; j++) {  
        result += matrix[i][j]*vector[j];  
    }  
    output[i] = result;  
}
```

**repeat many times —
temporal locality!**

```
i = 0;  
while(i < m) {  
    result = 0;  
    j = 0;  
    while(j < n) {  
        a = matrix[i][j];  
        b = vector[j];  
        temp = a*b;  
        result = result + temp;  
    }  
    output[i] = result;  
    i++;  
}
```

Locality

- Spatial locality — application tends to visit nearby stuffs in the memory
 - Code — the current instruction, and then the next instruction

Most of time, your program is just visiting a very small amount of data/instructions within a given window

- Temporal locality — application revisit the same thing again and again
 - Code — loops, frequently invoked functions
 - Typically tens of static instructions — at most several KBs
 - Data — program can read/write the same data many times (e.g., vectors in matrix-vector product)

Take-aways: inside out our memory hierarchy

- Memory access time is the most critical performance problem
 - One memory operation is as expensive as 50 arithmetic operations
 - Processor has to fetch instructions from memory
 - We have an average of 33% of data memory access instructions!
- Hierarchical caching with small amount of SRAMs will work if we can efficiently capture data and instructions
- Caching is possible! Most of time, we only work on a small amount of data!
 - Spatial locality
 - Temporal locality

Outline

- Designing a cache that captures the predictability
- The A, B, Cs of caches
- Estimating how cache friendly is our code

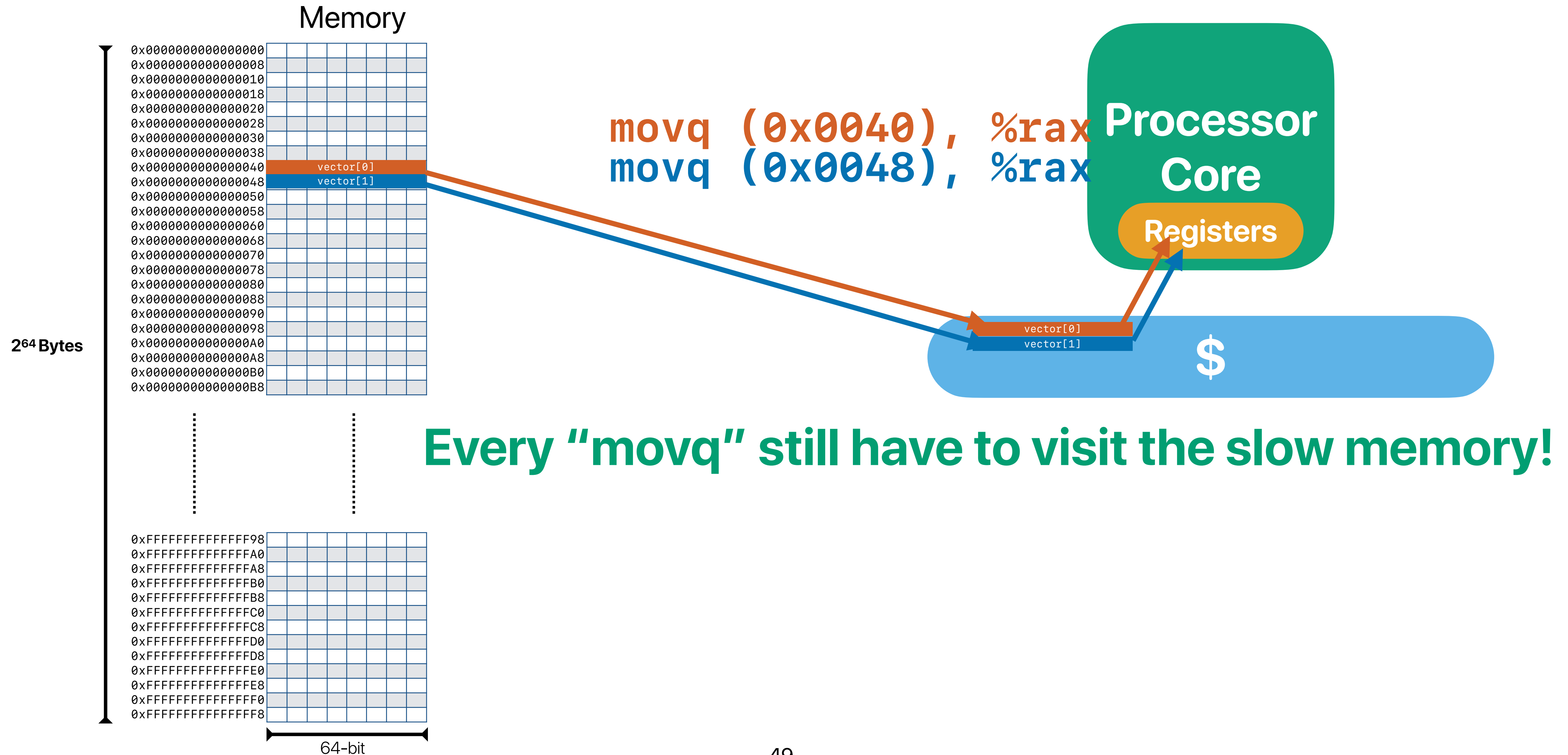
Designing a hardware to exploit locality

- Spatial locality — application tends to visit nearby stuffs in the memory

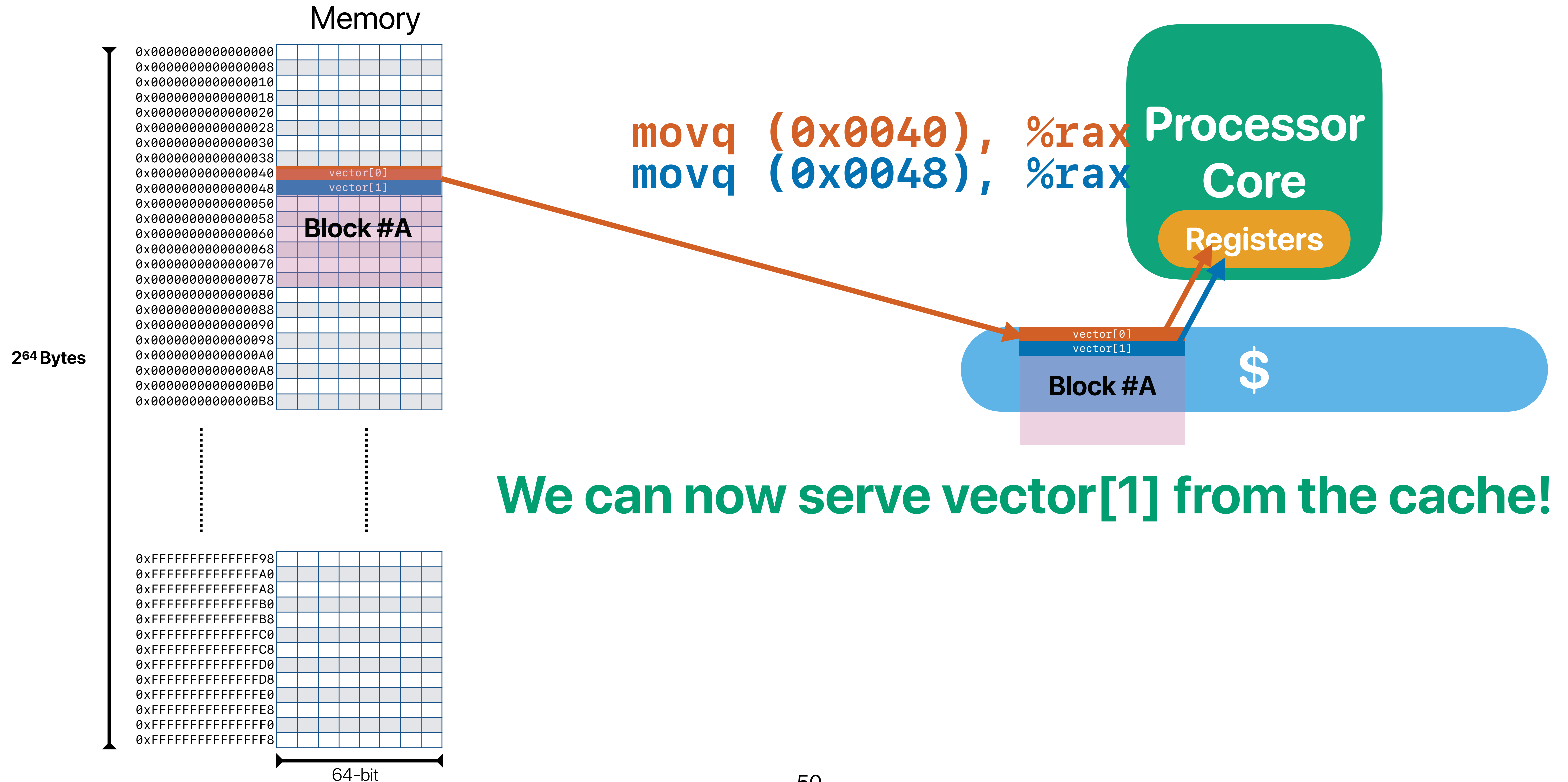
We need to "cache consecutive memory locations" every time — the cache should store a "block" of code/data

- Temporal locality — application revisit the same thing again and again
 - Code — loops, frequently invoked functions
 - Typically tens of static instructions — at most several KBs
 - Data — program can read/write the same data many times (e.g., vectors in matrix-vector product)

If we just cache what has requested



If we can cache a block of data



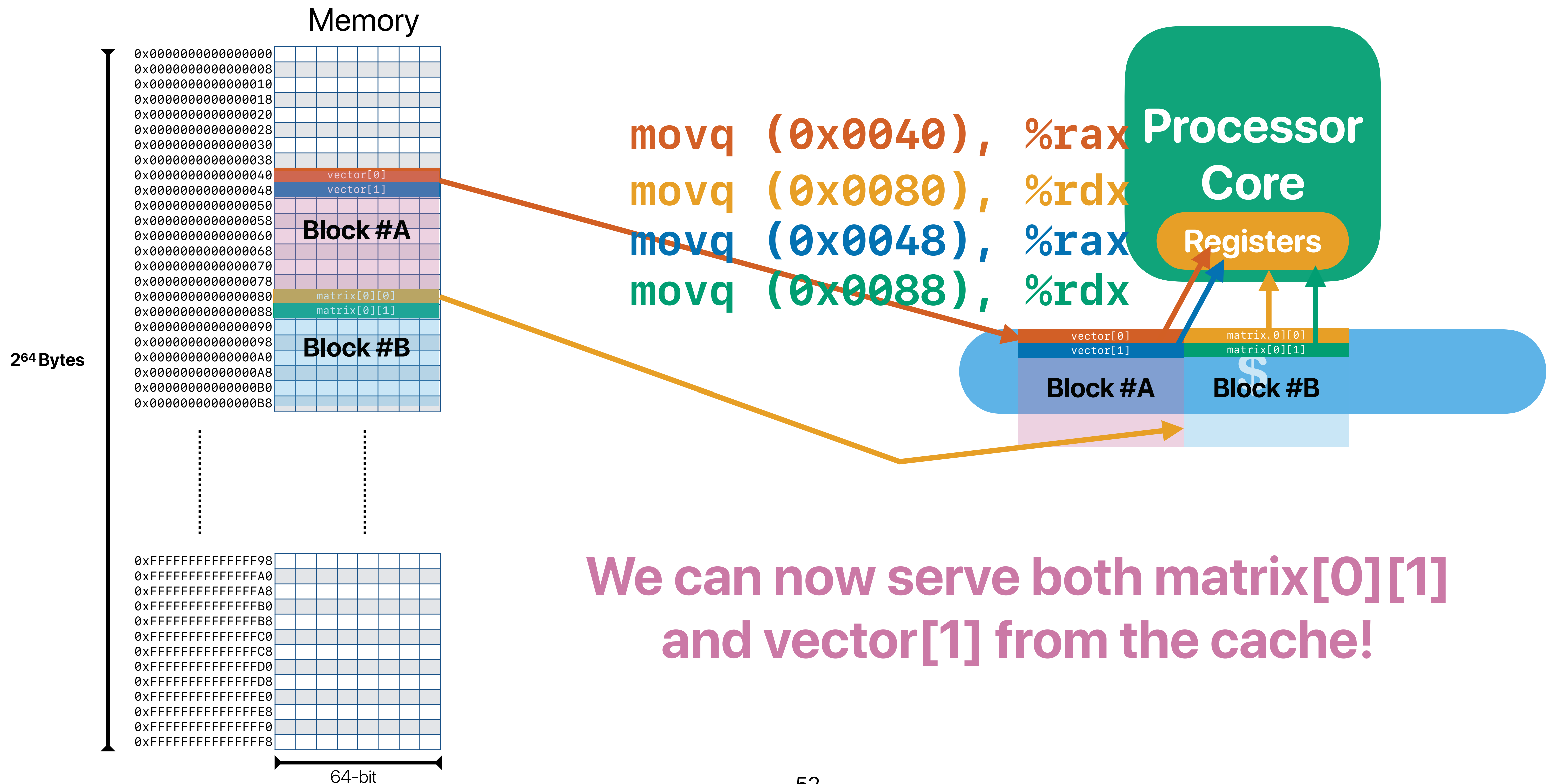
Recap: Locality

- Which description about locality of arrays `matrix` and `vector` in the following code is the **most accurate**?

```
for(uint64_t i = 0; i < m; i++) {  
    result = 0;  
    for(uint64_t j = 0; j < n; j++) {  
        result += matrix[i][j]*vector[j];  
    }  
    output[i] = result;  
}
```

**Simply caching one block
isn't enough**

We need to cache multiple blocks of data!



Designing a hardware to exploit locality

- Spatial locality — application tends to visit nearby stuffs in the memory

We need to "cache consecutive memory locations" every time
— the cache should store a "block" of code/data

- Temporal locality — application revisit the same thing again and again

We need to "cache frequently used memory blocks"

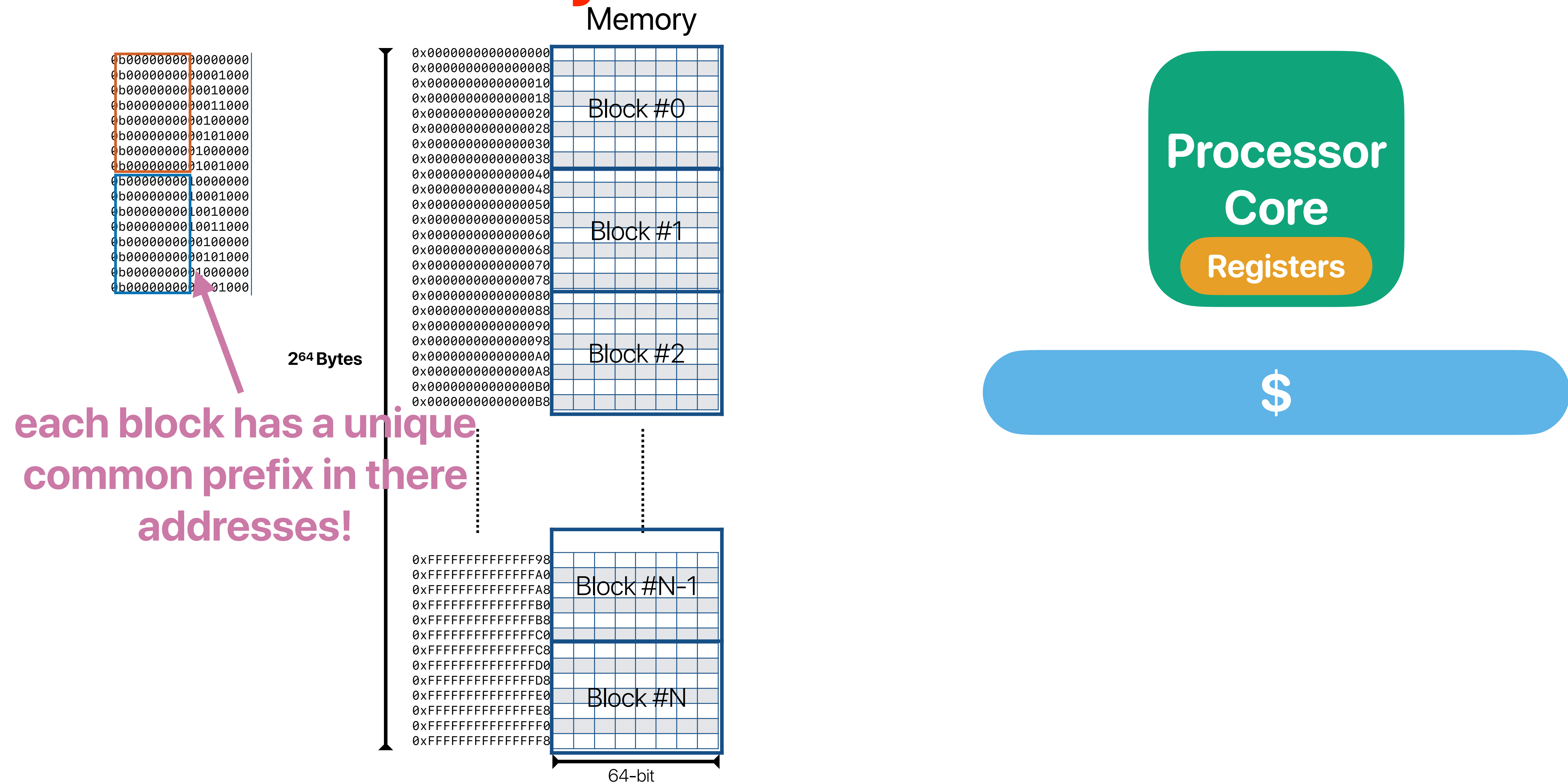
- **Code** — loops, frequently invoked functions
 - Typically tens of static instructions — at most several KBs
- **Data** — program can read/write the same data many times (e.g., vectors in matrix-vector product)
 - **the cache should store a few blocks**
 - **the cache must be able to distinguish blocks**



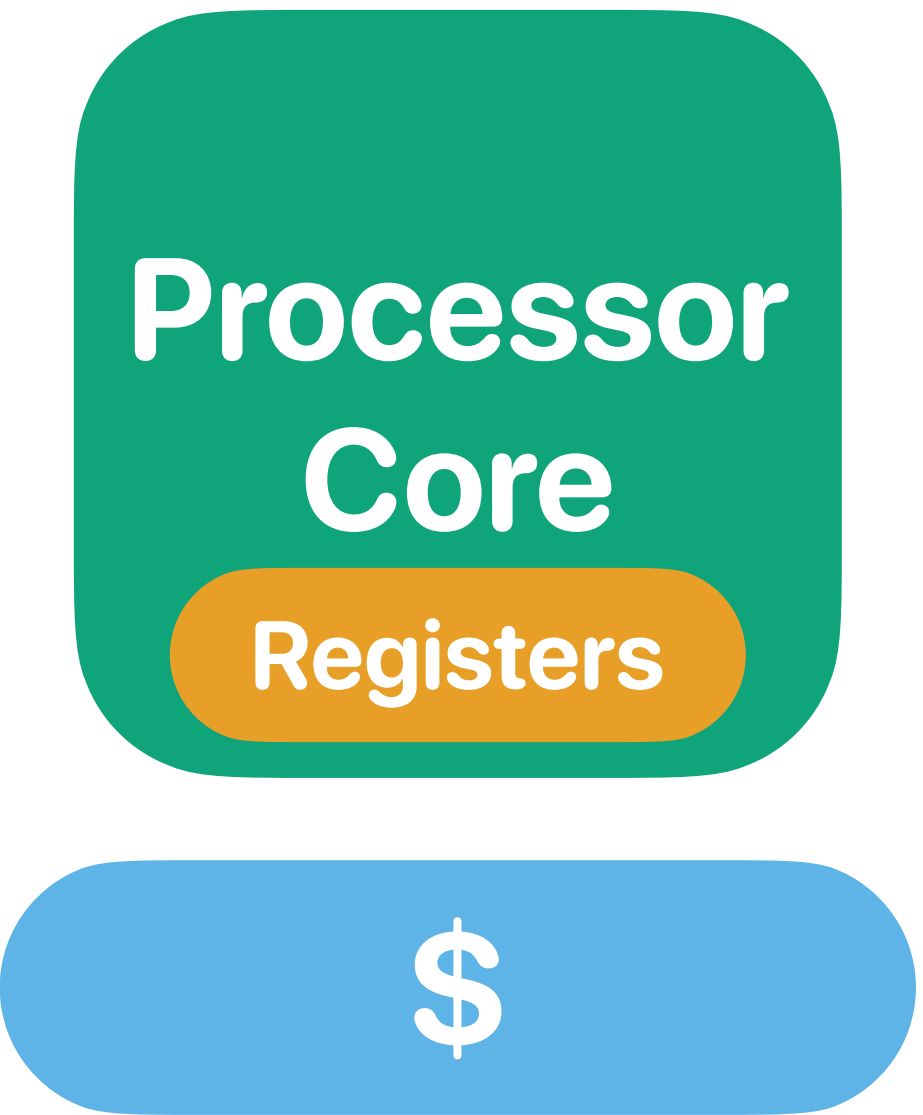
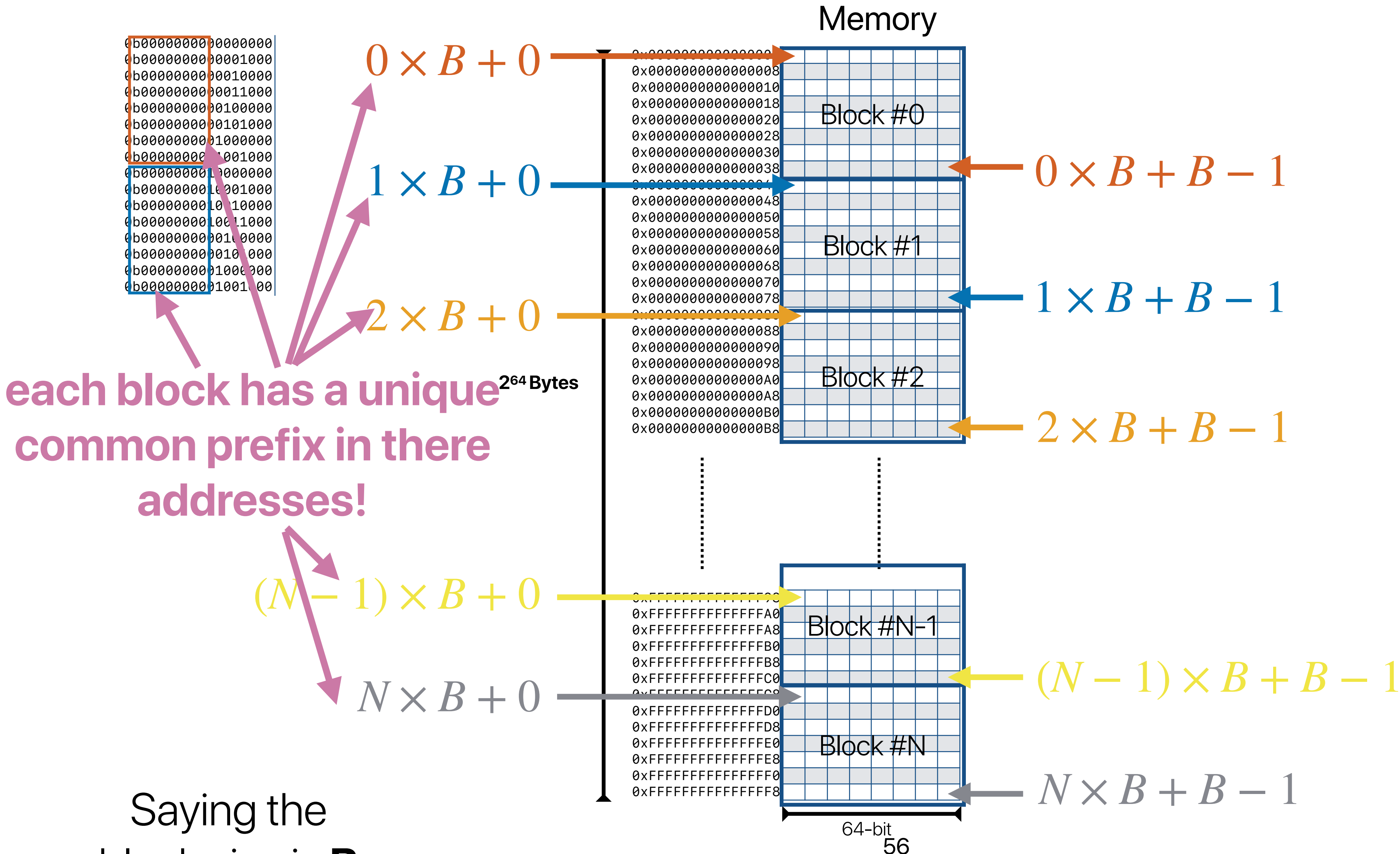
How to tell who is there?

?															
?															
0123456789ABCDEF															
This is CS 203:															
Advanced Computer Architecture!															
This is CS 203:															
Advanced Computer Architecture!															
This is CS 203:															
Advanced Computer Architecture!															
This is CS 203:															
Advanced Computer Architecture!															
This is CS 203:															

Partition memory addresses into fix-sized chunks



Partition memory addresses into fix-sized chunks



each block has a unique common prefix in there addresses!

Saying the block size is **B**

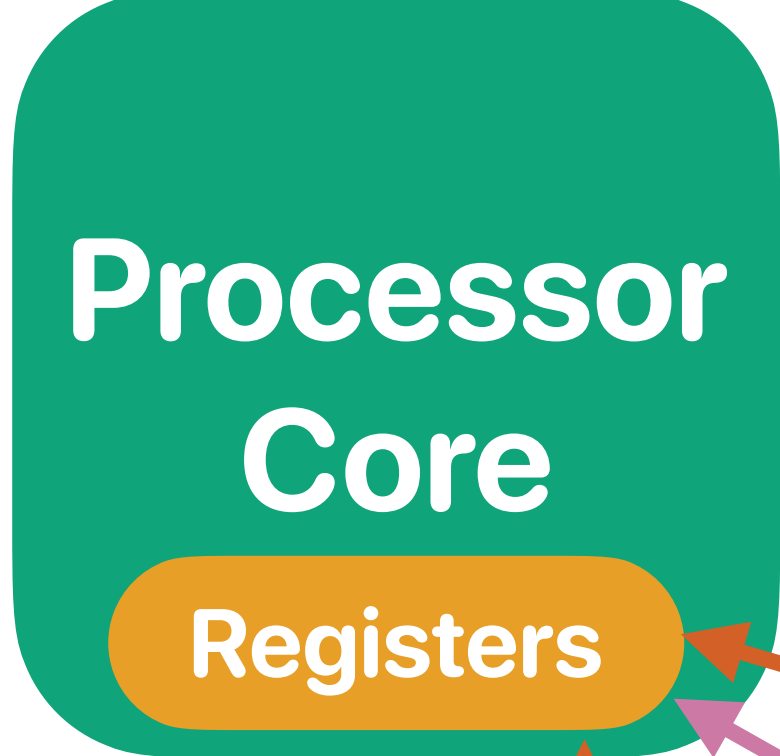
Registers

A 6x16 grid of hexadecimal characters (0-9, A-F) in various colors, illustrating the concept of a byte (8 bits).



0x000	This is CS 203:
0x001	
0x507	Advanced Compute
0x100	r Architecture!
0x210	
0x750	This is CS 203:
0x004	
0x507	Advanced Compute
0x267	r Architecture!
0x265	
0x00A	This is CS 203:
0x700	
0x705	Advanced Compute
0x214	r Architecture!
	This is CS 203:
	Advanced Compute
	r Architecture!
	This is CS 203:
	Advanced Compute
	r Architecture!
	This is CS 203:

How to tell w



block offset

tag

1w 0x0008

1w 0x4048

0x404 not found,
go to lower-level memory

Tell if the block here can be used
Tell if the block here is modified

Valid Bit

Dirty Bit

tag

data

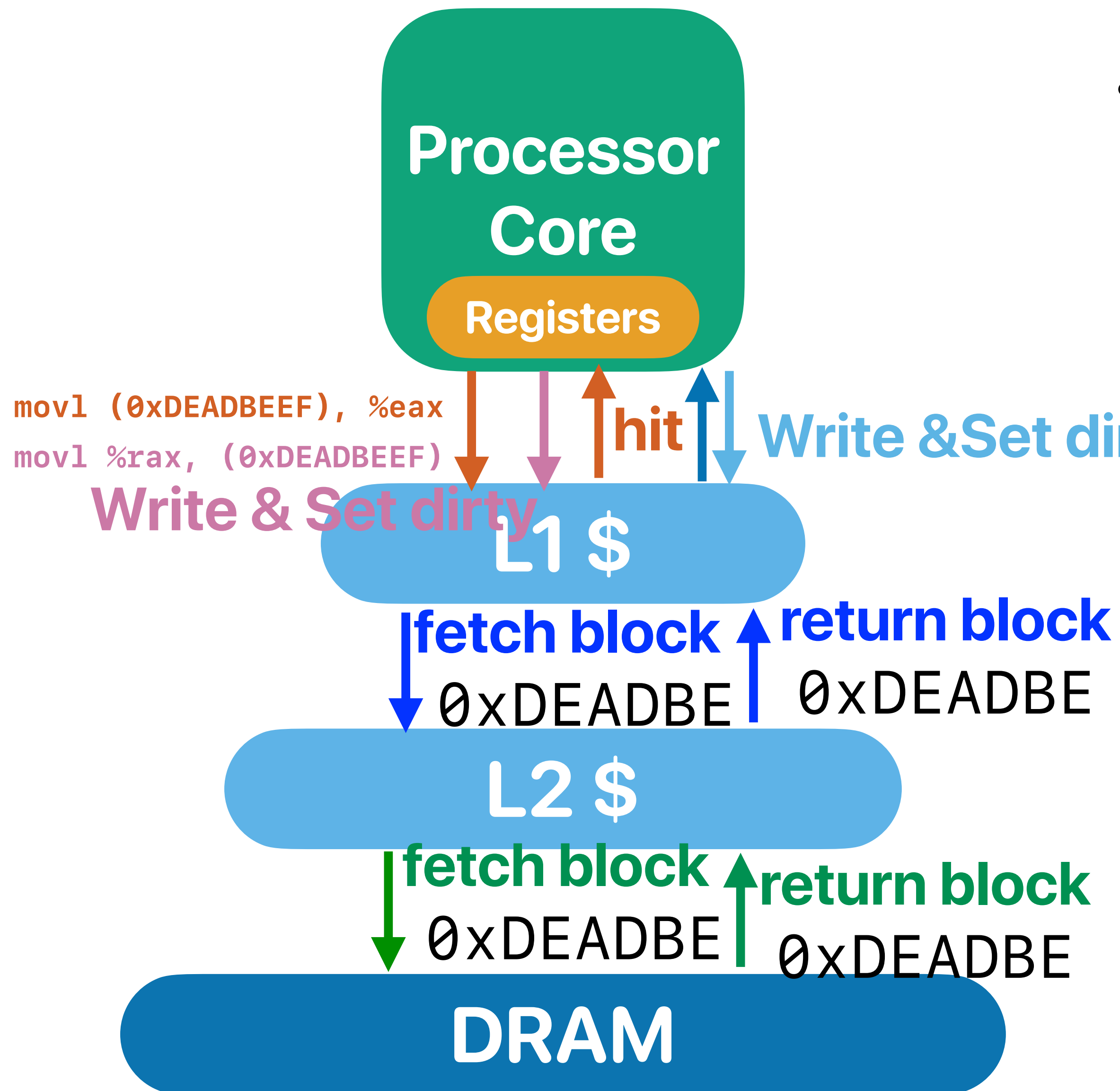
			0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
1	1		0x000		This is													
1	1		0x001		Advanced													
1	1		0x507		n Archite													
1	0		0x400		This is													
1	0		0x210		Advanced													
0	1		0x750		n Archite													
0	1		0x007		This is													
1	1		0x527		Advanced													
1	1		0x277		n Archite													
1	1		0x275		This is													
0	1		0x00A		Advanced													
0	1		0x720		This is													
0	1		0x720		Advanced													
0	1		0x705		n Archite													
0	1		0x217		This is													
1	1																	
1	1																	
0	1																	
0	1																	
1	1																	
1	1																	
1	0																	
0	1																	

Take-aways: designing caches

- Basic cache structures —
 - Caching in granularity of a block to capture spatial locality
 - Caching multiple blocks to keep frequently used data — temporal locality
 - Tags to distinguish cached blocks

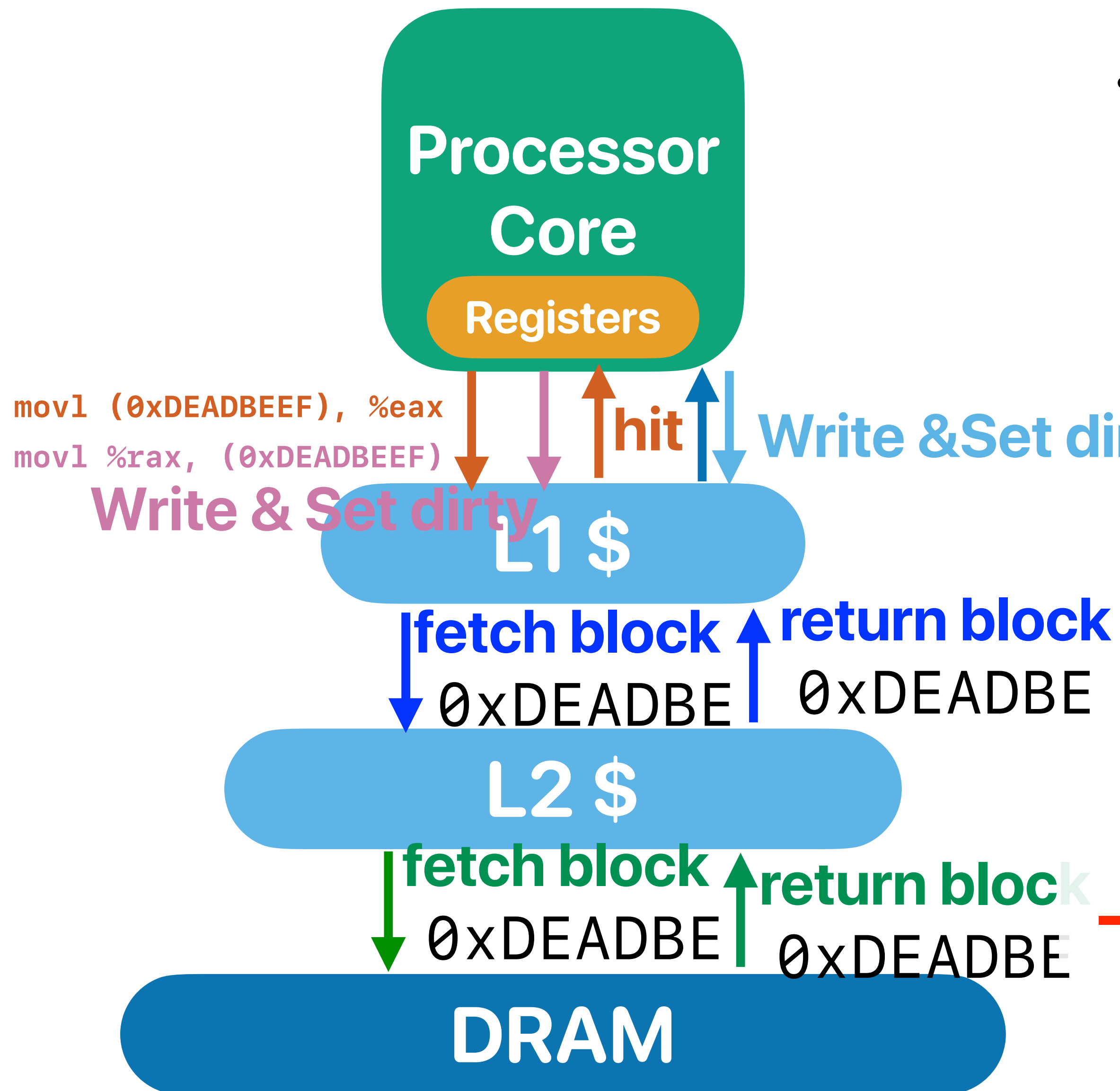
**Put everything all together:
How cache interacts with CPU**

Processor/cache interaction



- Processor sends memory access request to L1-\$
 - **if hit & it's a read**
 - **Read: return data**
 - **Write: Update "ONLY" in L1 and set DIRTY**
 - **Why don't we write to L2? — Too slow**
 - **if miss**
 - Fetch the requesting block from lower-level memory hierarchy and place in the cache
 - **Present the write "ONLY" in L1 and set DIRTY**
- What if we run out of \$ blocks?**

Processor/cache interaction

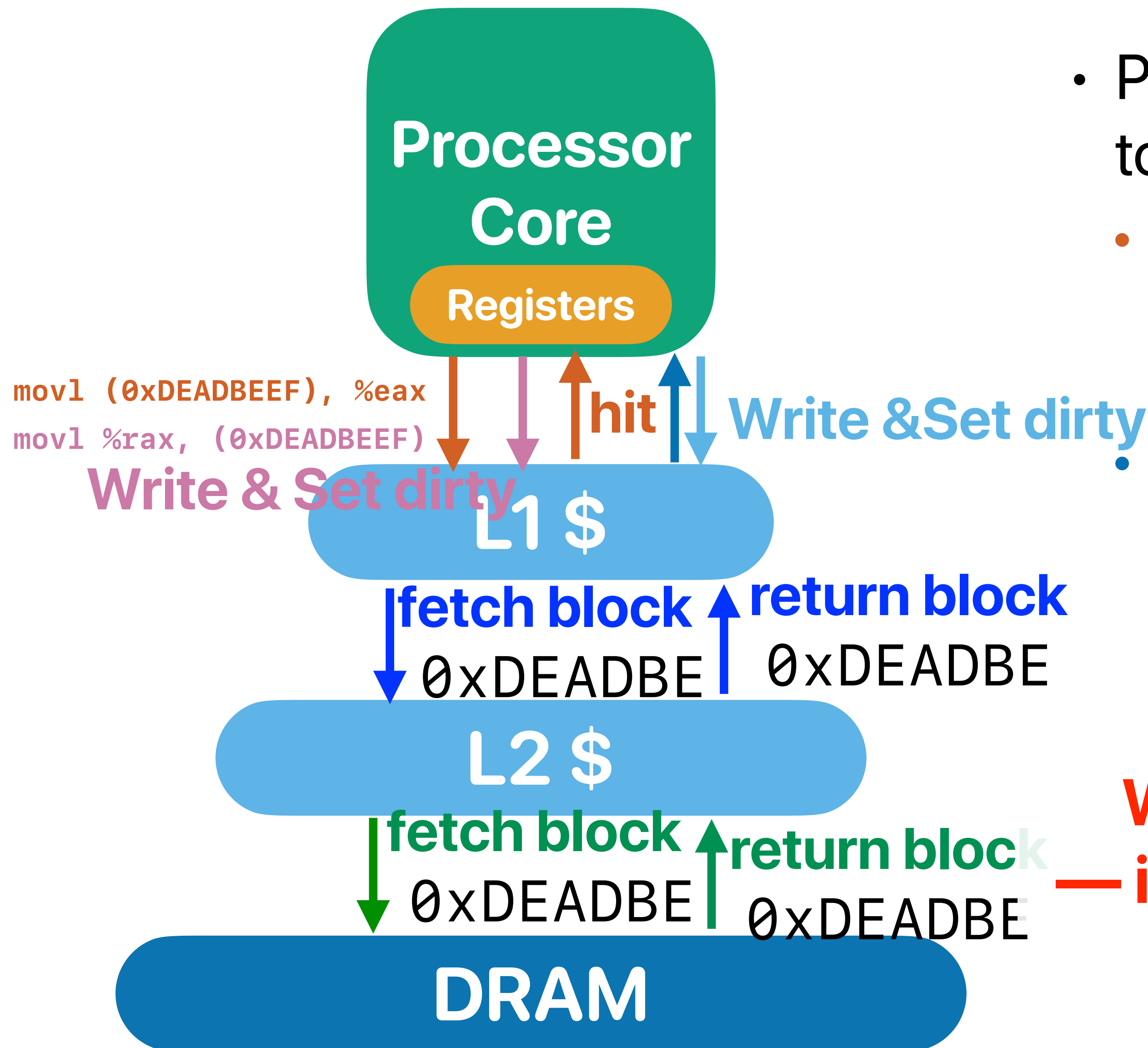


- Processor sends memory access request to L1-\$
- **if hit & it's a read**
 - **Read:** return data
- **Write: Update "ONLY" in L1 and set DIRTY**
- **if miss**
 - If there an empty block — place the data there
 - If NOT (most frequent case) — select a **victim block**
 - Least Recently Used (LRU) policy

What if the victim block is modified?
— ignoring the update is not acceptable!

- **Present the write "ONLY" in L1 and set DIRTY**

Processor/cache interaction

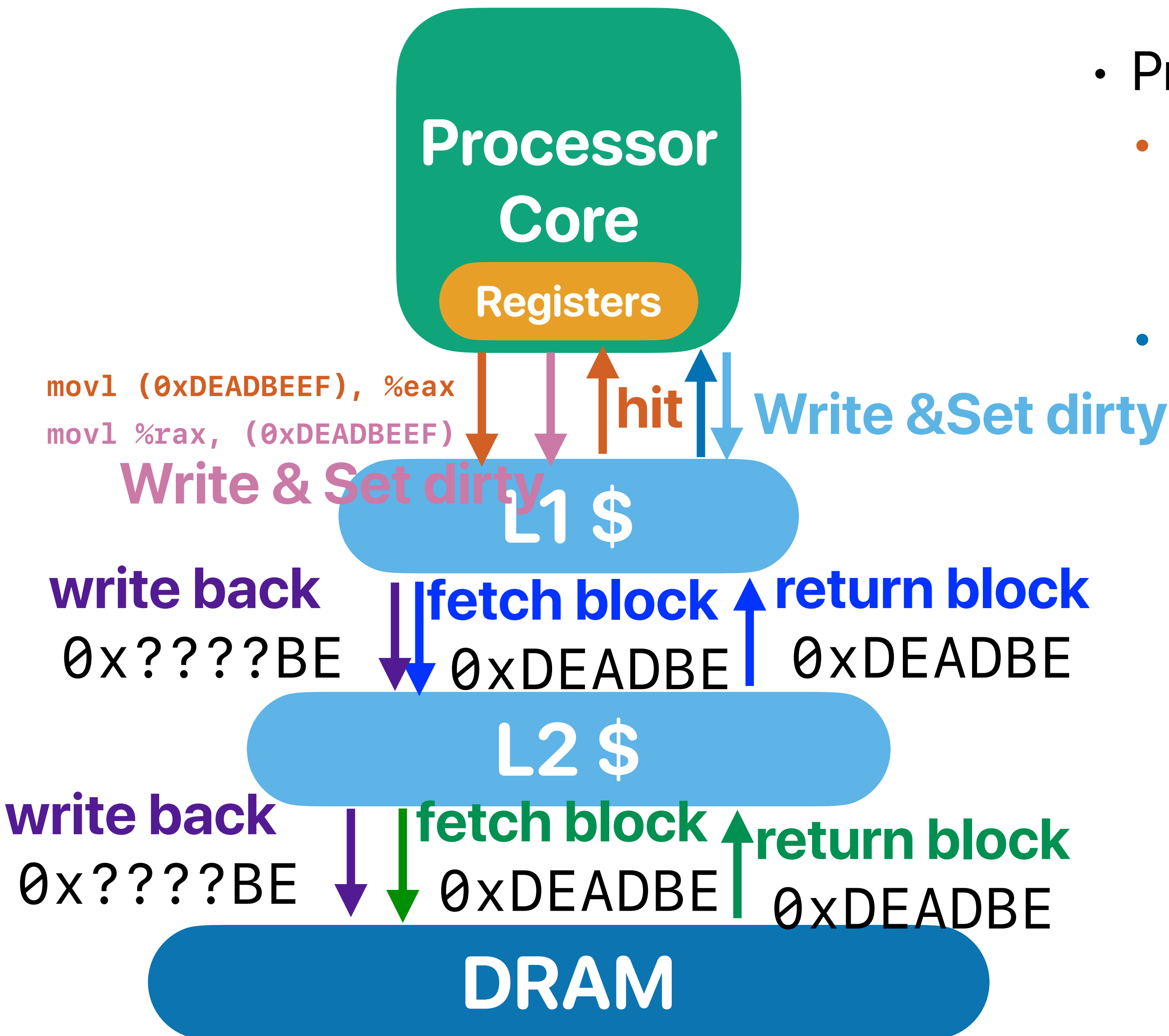


- Processor sends memory access request to L1-\$
- **if hit & it's a read**
 - **Read:** return data
- **Write: Update "ONLY" in L1 and set DIRTY**
- **if miss**
 - If there an empty block — place the data there
 - If NOT (most frequent case) — select a **victim block**
 - Least Recently Used (LRU) policy

What if the victim block is modified?
— ignoring the update is not acceptable!

- **Present the write "ONLY" in L1 and set DIRTY**

Processor/cache interaction

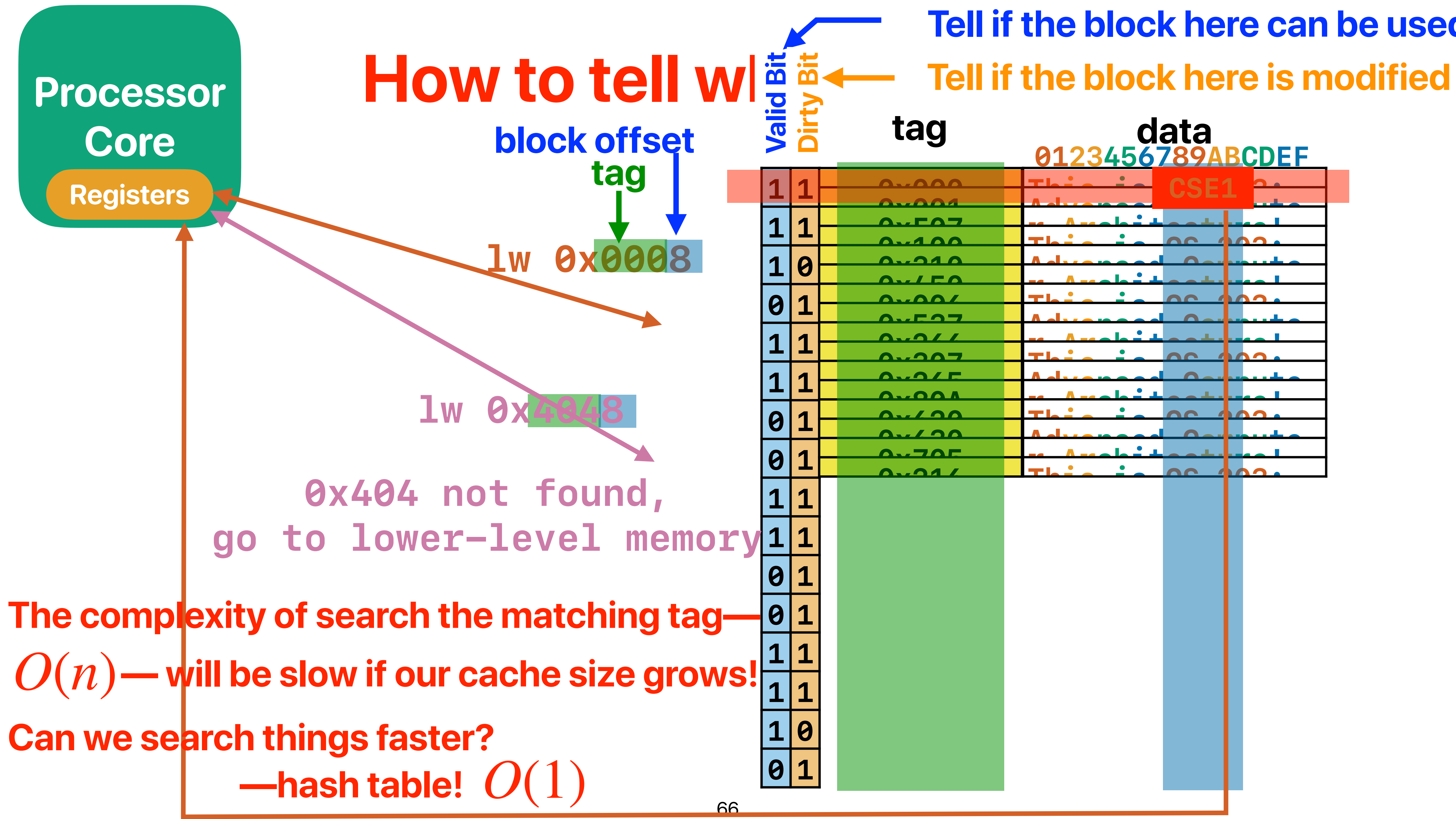


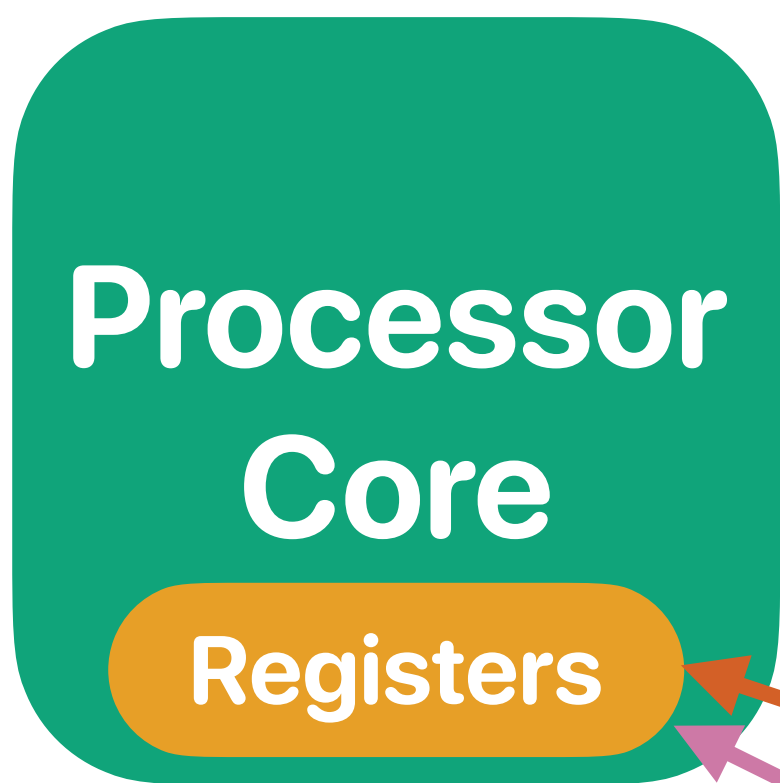
- Processor sends memory access request to L1-\$
 - **if hit & it's a read**
 - **Read:** return data
 - **Write:** Update "ONLY" in L1 and set DIRTY
 - **if miss**
 - If there an empty block — place the data there
 - If NOT (most frequent case) — select a **victim block**
 - Least Recently Used (LRU) policy
 - If the victim block is "dirty" & "valid"
 - **Write back** the block to lower-level memory hierarchy
 - If write-back or fetching causes any miss, repeat the same process
 - Fetch the requesting block from lower-level memory hierarchy and place in the cache
 - **Present the write "ONLY" in L1 and set DIRTY**

Take-aways: designing caches

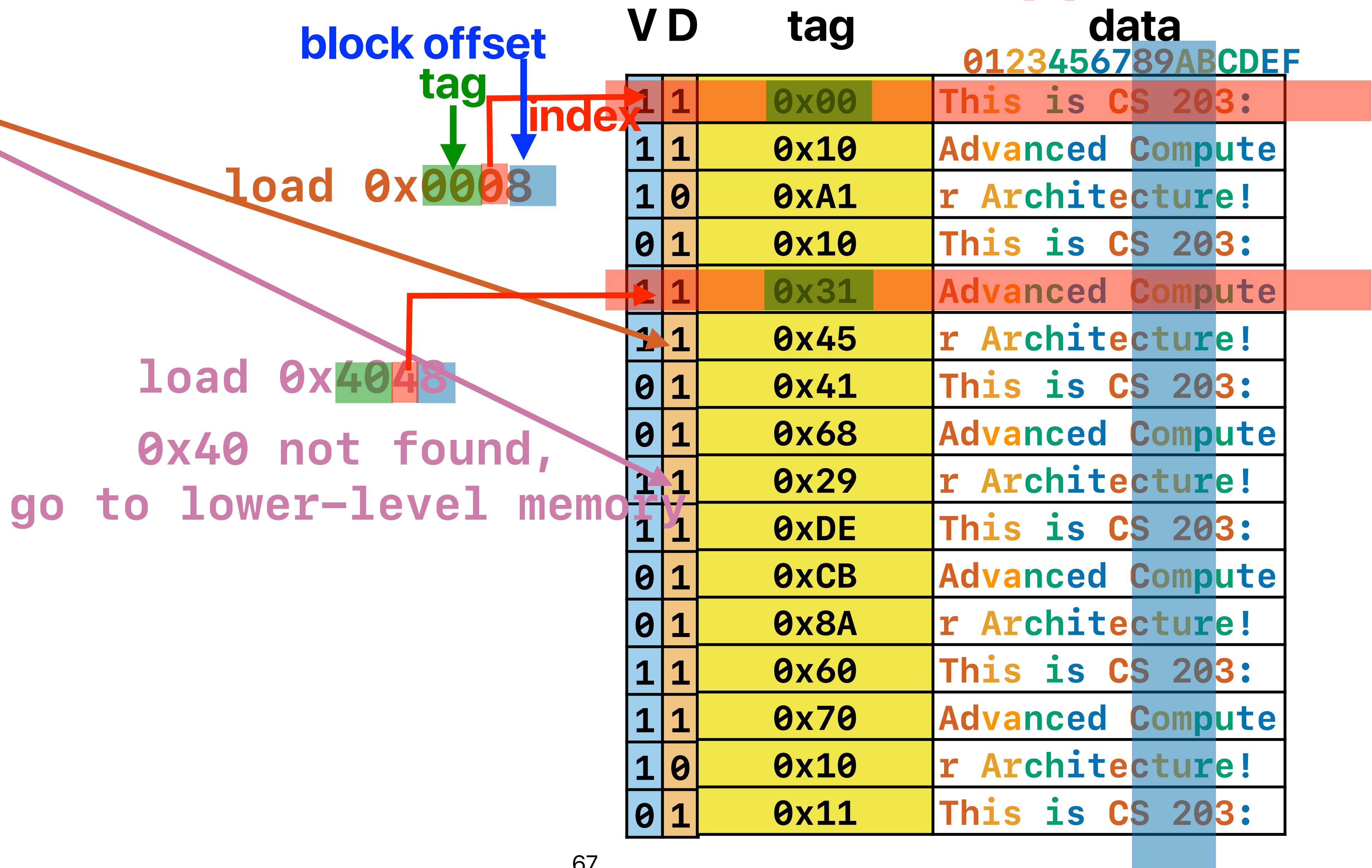
- Basic cache structures —
 - Caching in granularity of a block to capture spatial locality
 - Caching multiple blocks to keep frequently used data — temporal locality
 - Tags to distinguish cached blocks
- Hierarchical caching — data must be presented on the top level (L1) before the processor can use

How to tell w



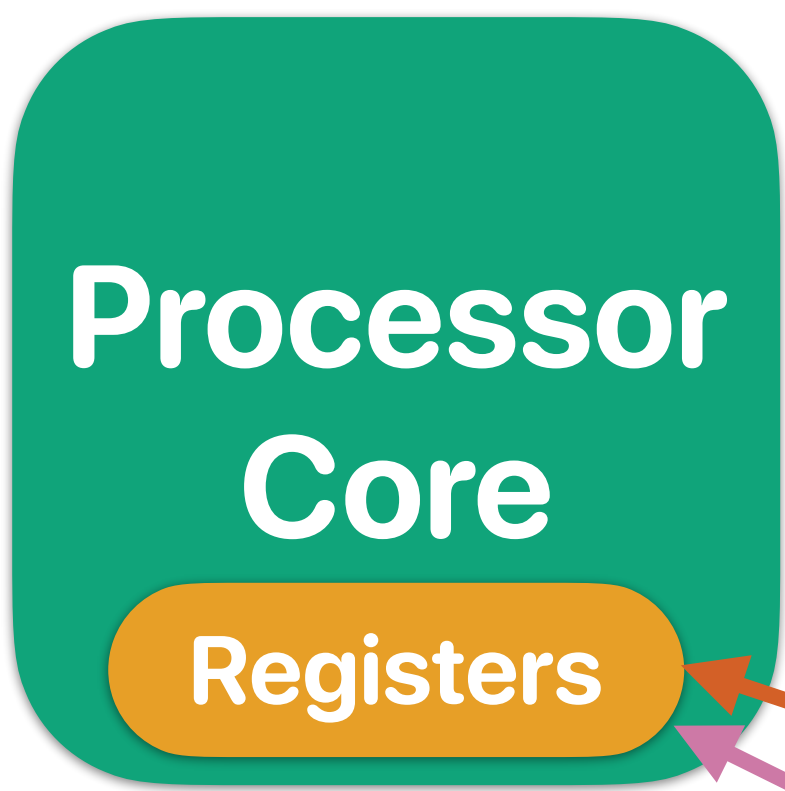


Hash-like structure — direct-mapped cache

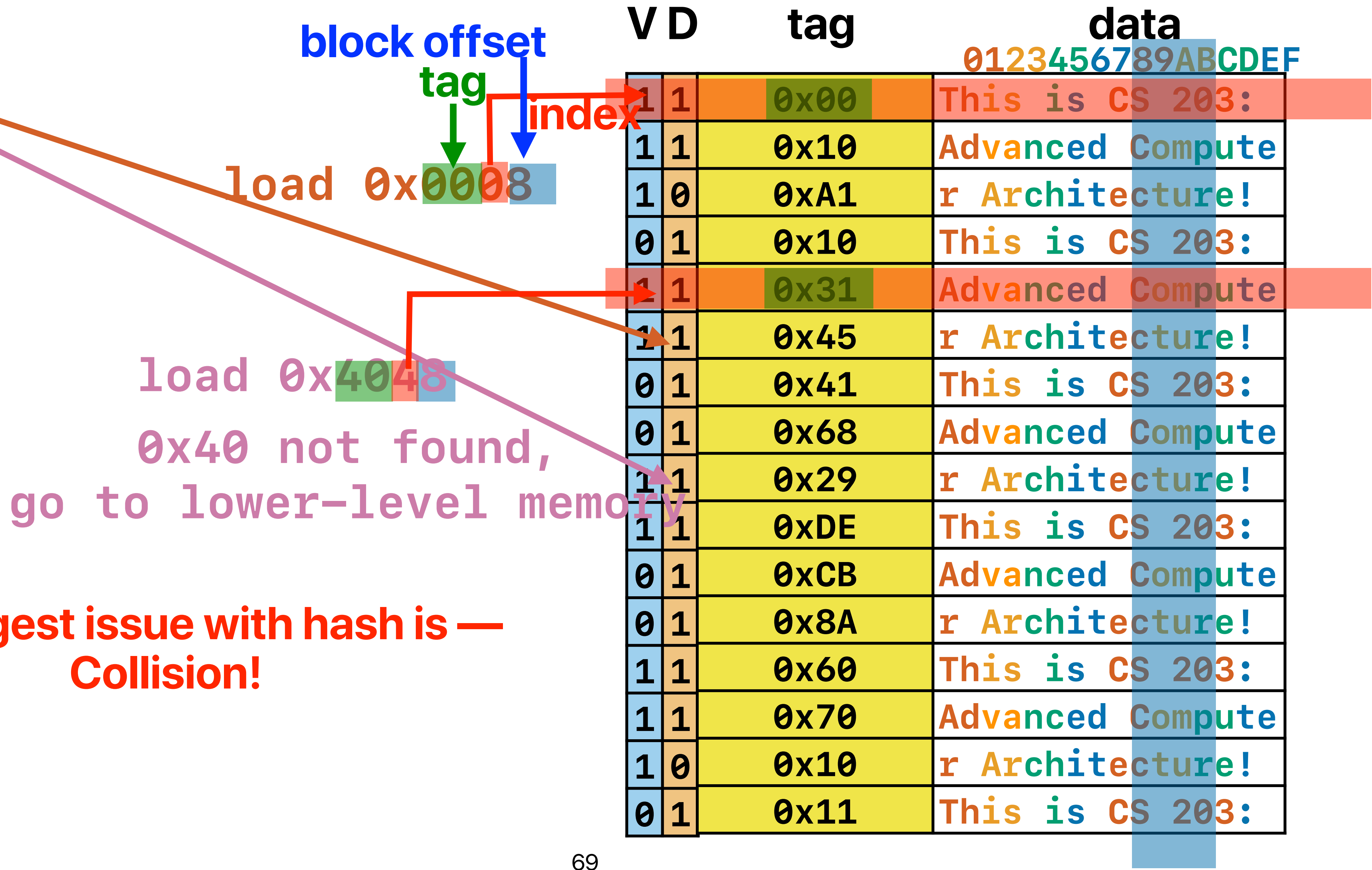


Take-aways: designing caches

- Basic cache structures —
 - Caching in granularity of a block to capture spatial locality
 - Caching multiple blocks to keep frequently used data — temporal locality
 - Tags to distinguish cached blocks
- Hierarchical caching — data must be presented on the top level (L1) before the processor can use
- Optimizing cache structures
 - Hash block into "sets" to reduce the search time

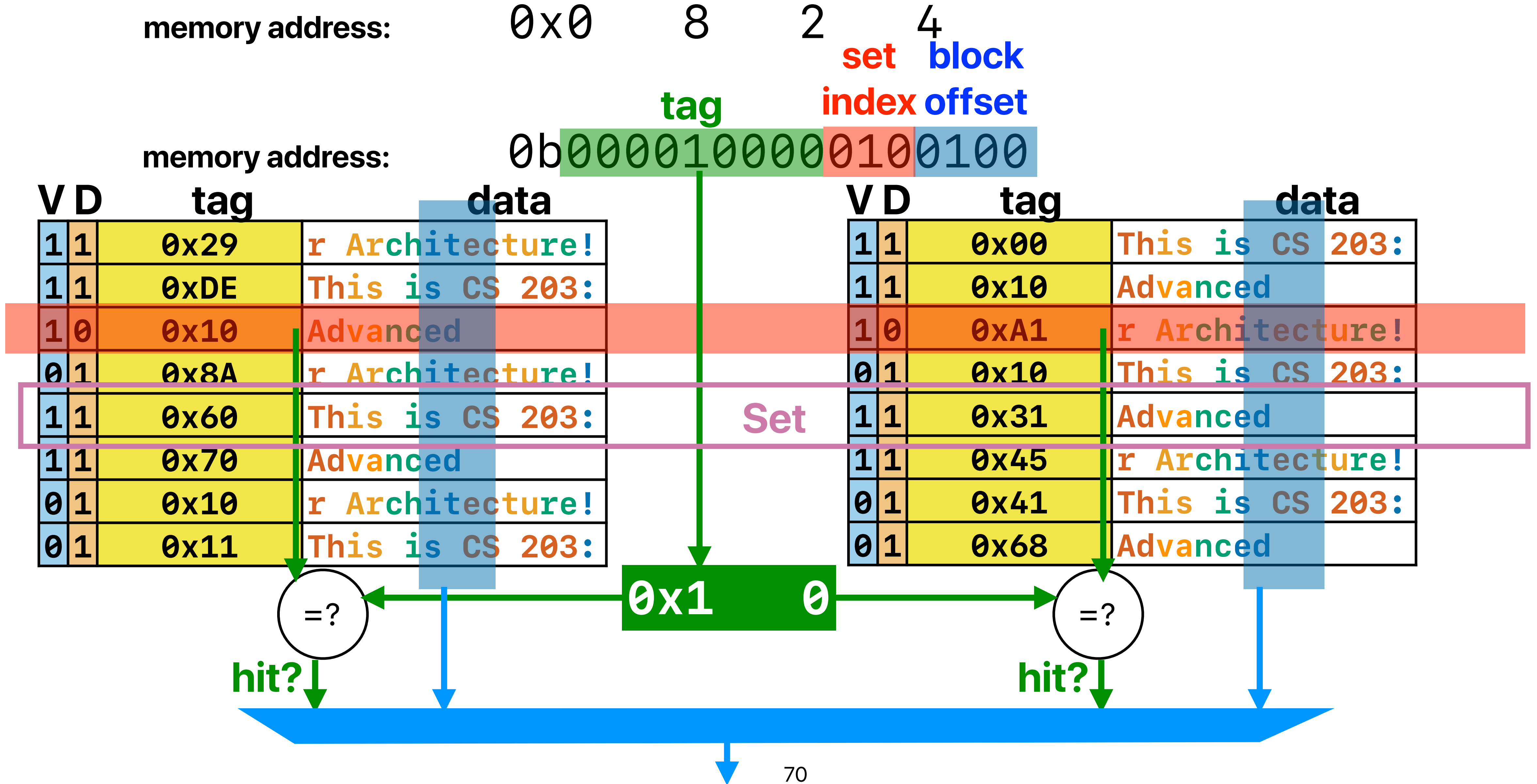


Hash-like structure — direct-mapped cache



The biggest issue with hash is — Collision!

Way-associative cache





Blocksize == Linesize

```
/sys/devices/system/cpu/cpu0/cache/index0/physical_line_partition:1
/sys/devices/system/cpu/cpu0/cache/index0/number_of_sets:64
/sys/devices/system/cpu/cpu0/cache/index0/ways_of_associativity:12
/sys/devices/system/cpu/cpu0/cache/index0/id:0
/sys/devices/system/cpu/cpu0/cache/index0/shared_cpu_list:0-1
/sys/devices/system/cpu/cpu0/cache/index0/type:Data
/sys/devices/system/cpu/cpu0/cache/index0/size:48K
/sys/devices/system/cpu/cpu0/cache/index0/level:1
/sys/devices/system/cpu/cpu0/cache/index0/coherency_line_size:64
/sys/devices/system/cpu/cpu0/cache/index0/shared_cpu_map:000003
```

Blocksize == Linesize

```
/sys/devices/system/cpu/cpu0/cache/index0/physical_line_partition:1  
/sys/devices/system/cpu/cpu0/cache/index0/number of sets:64  
/sys/devices/system/cpu/cpu0/cache/index0/ways of associativity:12  
/sys/devices/system/cpu/cpu0/cache/index0/id:0  
/sys/devices/system/cpu/cpu0/cache/index0/shared_cpu_list:0-1  
/sys/devices/system/cpu/cpu0/cache/index0/type:Data  
/sys/devices/system/cpu/cpu0/cache/index0/size:48K  
/sys/devices/system/cpu/cpu0/cache/index0/level:1  
/sys/devices/system/cpu/cpu0/cache/index0/coherency_line_size:64  
/sys/devices/system/cpu/cpu0/cache/index0/shared_cpu_map:000003
```

Take-aways: designing caches

- Basic cache structures —
 - Caching in granularity of a block to capture spatial locality
 - Caching multiple blocks to keep frequently used data — temporal locality
 - Tags to distinguish cached blocks
- Hierarchical caching — data must be presented on the top level (L1) before the processor can use
- Optimizing cache structures
 - Hash block into "sets" to reduce the search time
 - Set-associativity to reduce the "collision" problem

Way-associative cache

memory address: $0x0$ 8 2 4

set block

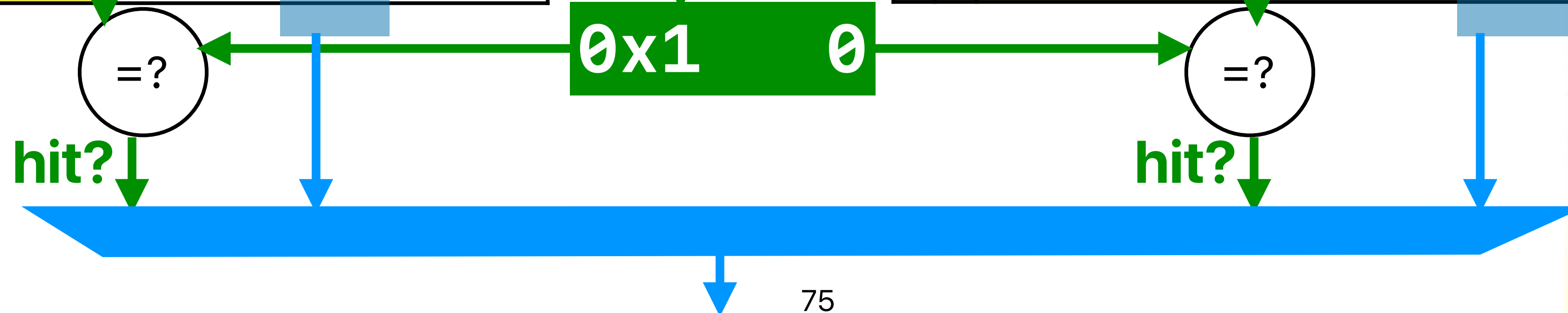
tag index offset

memory address: $0b00001000000100100$

V	D	tag	data
1	1	$0x29$	r Architecture!
1	1	$0xDE$	This is CS 203:
1	0	$0x10$	Advanced
0	1	$0x8A$	r Architecture!
1	1	$0x60$	This is CS 203:
1	1	$0x70$	Advanced
0	1	$0x10$	r Architecture!
0	1	$0x11$	This is CS 203:

V	D	tag	data
1	1	$0x00$	This is CS 203:
1	1	$0x10$	Advanced
1	0	$0xA1$	r Architecture!
0	1	$0x10$	This is CS 203:
1	1	$0x31$	Advanced
1	1	$0x45$	r Architecture!
0	1	$0x41$	This is CS 203:
0	1	$0x68$	Advanced

Set



The A, B, Cs of your cache

$$C = ABS$$

- **C**: Capacity in data arrays
- **A**: Way-Associativity — how many blocks within a set
 - N-way: N blocks in a set, $A = N$
 - 1 for direct-mapped cache
- **B**: Block Size (Linesize)
 - How many bytes in a block
- **S**: Number of Sets:
 - A set contains blocks sharing the same index
 - 1 for fully associate cache



Corollary of $C = ABS$

memory address: 0b 

- number of bits in **block** offset — $\lg(\mathbf{B})$
- number of bits in **set** index: $\lg(\mathbf{S})$
- tag bits: $\text{address_length} - \lg(\mathbf{S}) - \lg(\mathbf{B})$
 - address_length is N bits for N-bit machines (e.g., 64-bit for 64-bit machines)
- $(\text{address} / \text{block_size}) \% \mathbf{S} = \text{set index}$



NVIDIA Tegra Orin

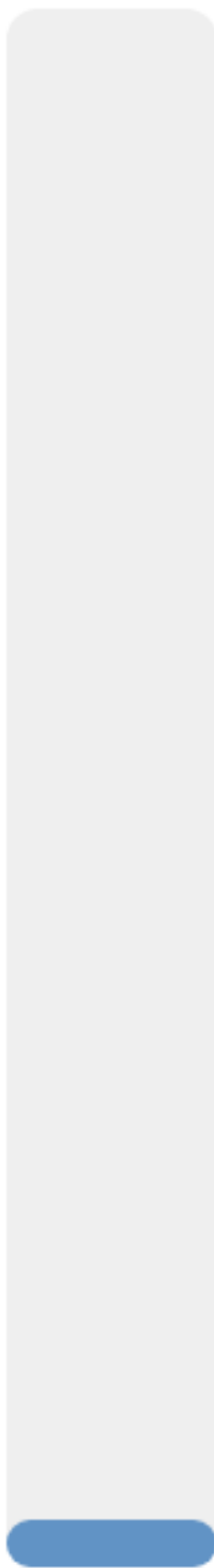
- L1 data (D-L1) cache configuration of NVIDIA Tegra Orin
 - Size 64KB, 4-way set associativity, 64B block
 - Assume 64-bit memory address

Which of the following is correct?

- A. Tag is 49 bits
- B. Index is 8 bits
- C. Offset is 7 bits
- D. The cache has 1024 sets
- E. None of the above

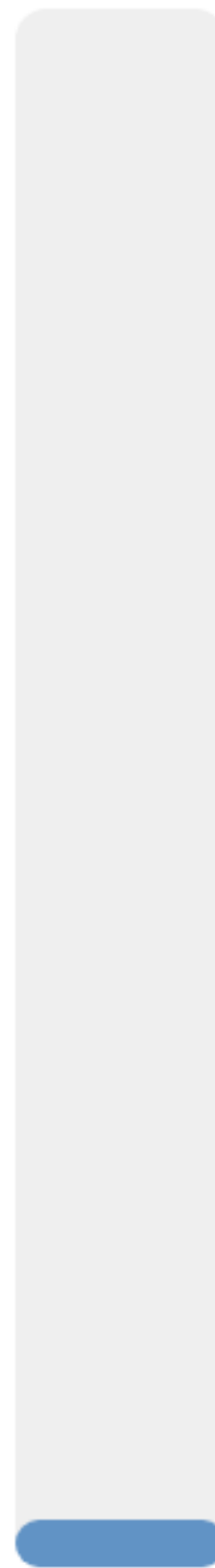
A screenshot of a Pollev poll interface. It shows five horizontal input fields labeled A, B, C, D, and E, stacked vertically. The interface is clean and modern, with a light blue header and a white body. The poll is titled 'Which of the following is correct?' and the options are A. Tag is 49 bits, B. Index is 8 bits, C. Offset is 7 bits, D. The cache has 1024 sets, and E. None of the above.

0



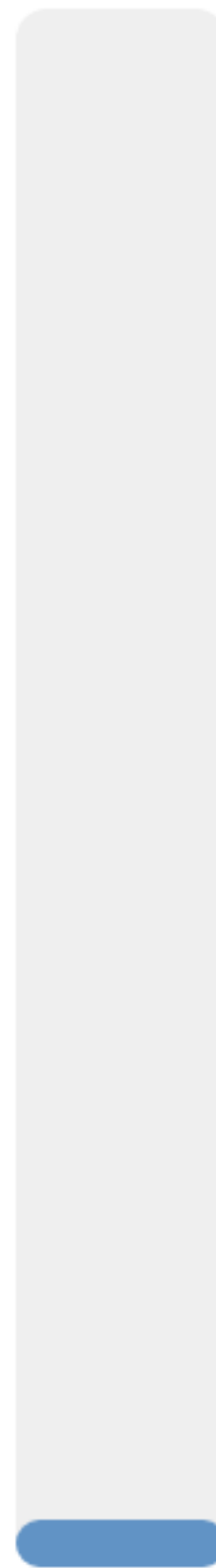
A

0



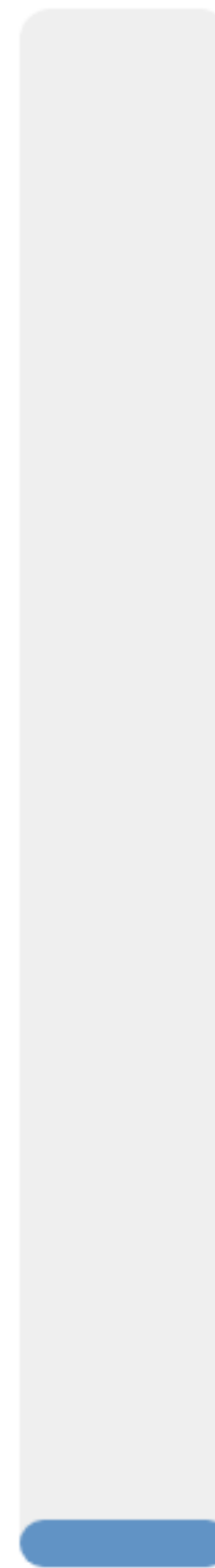
B

0



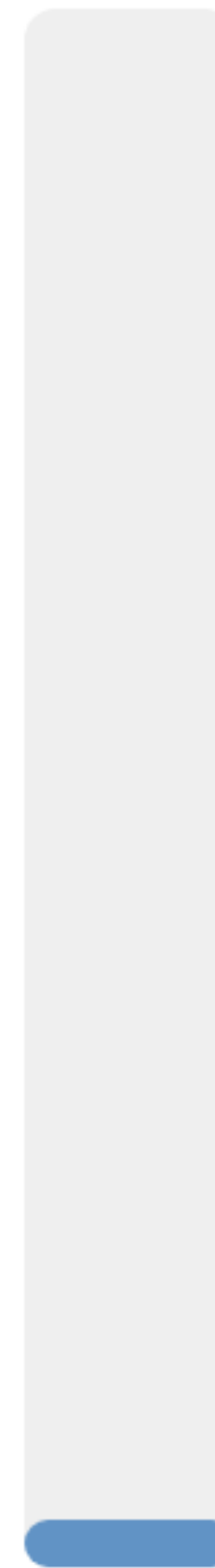
C

0



D

0



E



NVIDIA Tegra Orin

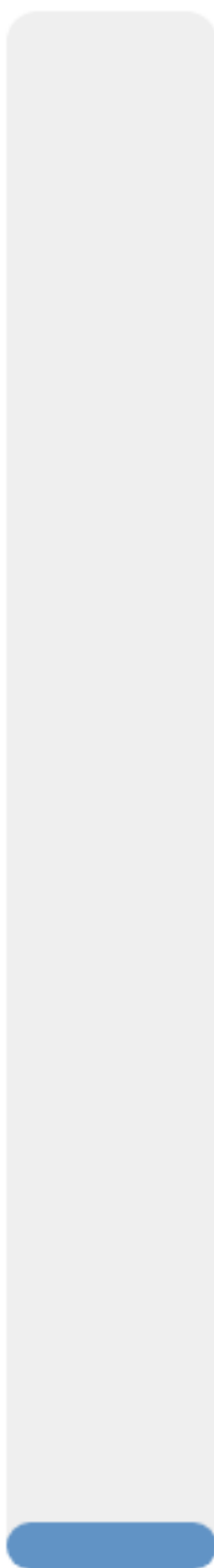
- L1 data (D-L1) cache configuration of NVIDIA Tegra Orin
 - Size 64KB, 4-way set associativity, 64B block
 - Assume 64-bit memory address

Which of the following is correct?

- A. Tag is 49 bits
- B. Index is 8 bits
- C. Offset is 7 bits
- D. The cache has 1024 sets
- E. None of the above

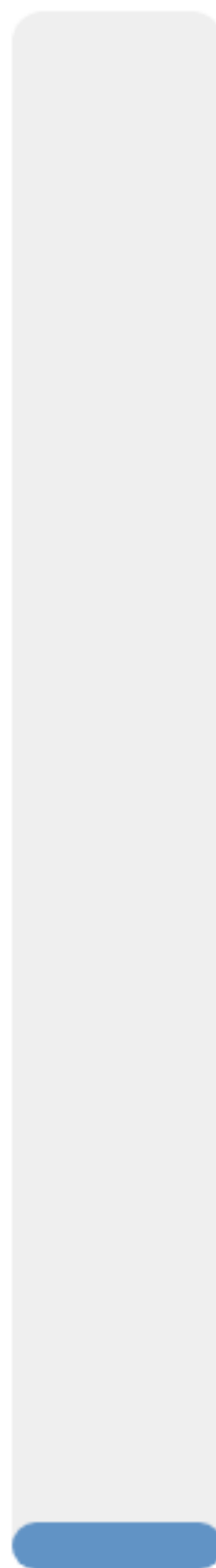
A screenshot of a Pollev poll interface. It shows five horizontal input fields labeled A, B, C, D, and E, stacked vertically. The fields are currently empty, suggesting a multiple-choice question where one or more options can be selected. The interface is clean and modern, with a light blue header and footer.

0



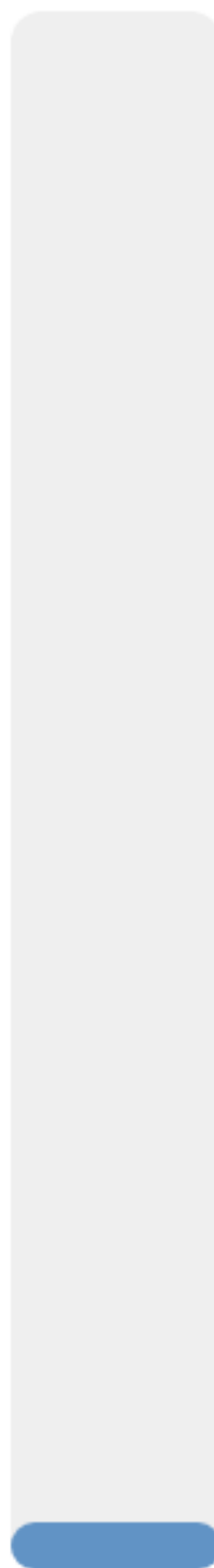
A

0



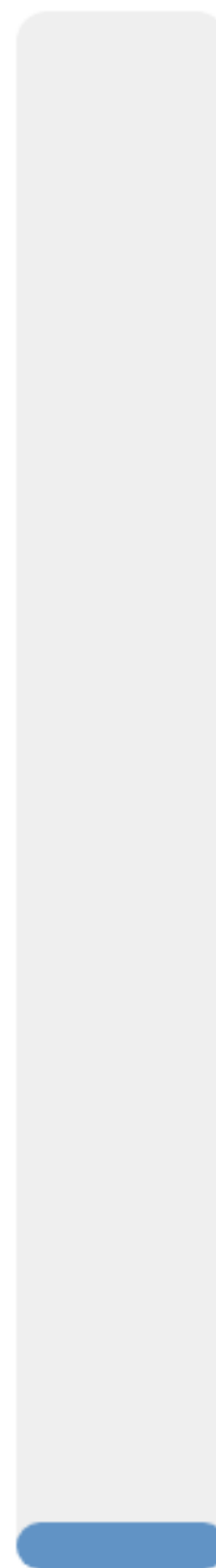
B

0



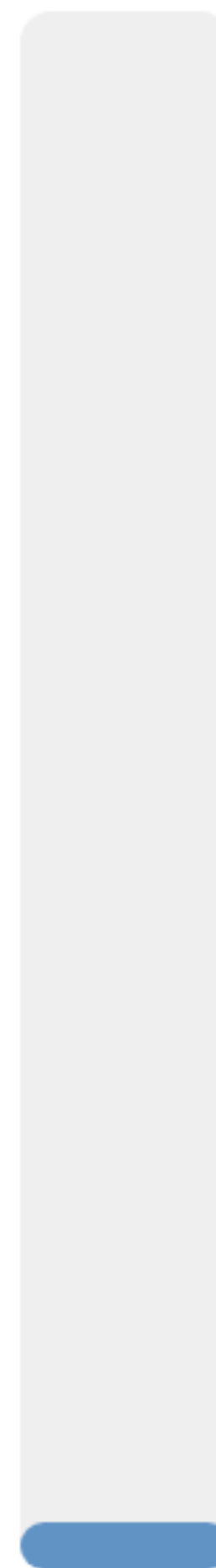
C

0



D

0



E

NVIDIA Tegra Orin

- L1 data (D-L1) cache configuration of NVIDIA Tegra Orin
 - Size 64KB, 4-way set associativity, 64B block
 - Assume 64-bit memory address

Which of the following is correct?

A. Tag is 49 bits

B. Index is 8 bits

C. Offset is 7 bits

D. The cache has 1024 sets

E. None of the above

$$C = A \times B \times S$$

$$64 \times 1024 = 4 \times 64 \times S$$

$$S = 256$$

$$\text{Offset} = \log_2(64) = 6$$

$$\text{Index} = \log_2(256) = 8$$

$$\text{Tag} = 64 - 8 - 6 = 50$$

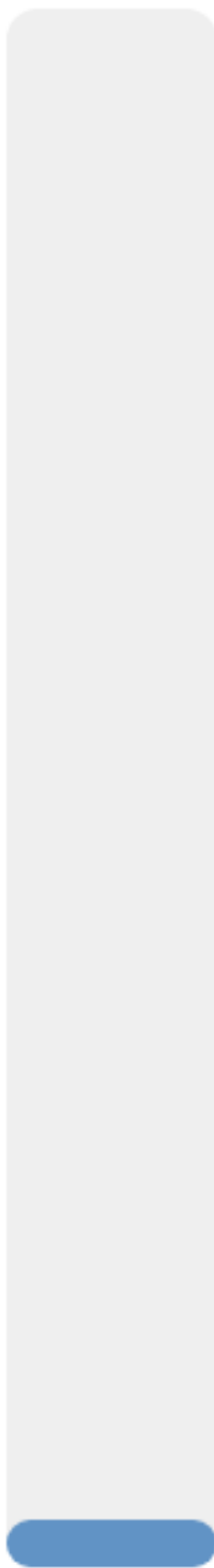


intel Core i7 14700K

- L1 data (D-L1) cache configuration of Core i7 14700K's P-Core
 - Size 48KB, 12-way set associativity, 64B block
 - Assume 64-bit memory address
 - Which of the following is **NOT** correct?
 - A. Tag is 52 bits
 - B. Index is 6 bits
 - C. Offset is 6 bits
 - D. The cache has 128 sets
 - E. All of the above are correct

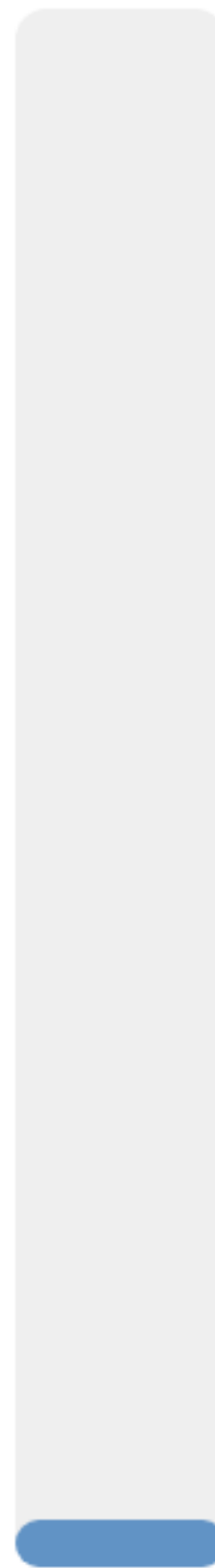
A screenshot of a Pollev poll interface. It shows five horizontal input fields labeled A, B, C, D, and E, stacked vertically. The fields are currently empty, suggesting a multiple-choice question where users select one or more options.

0



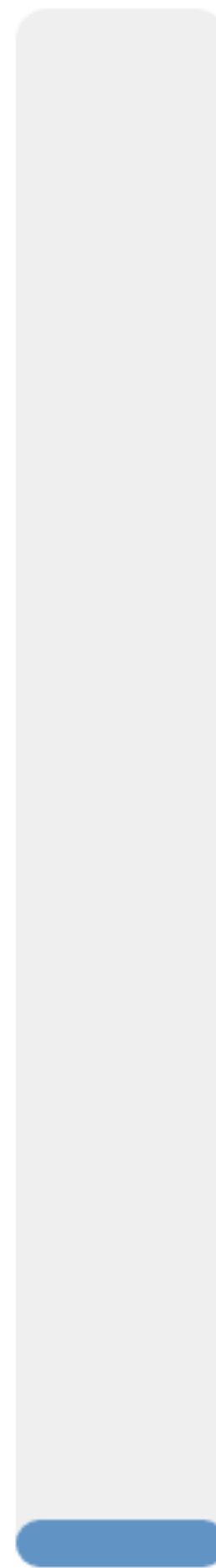
A

0



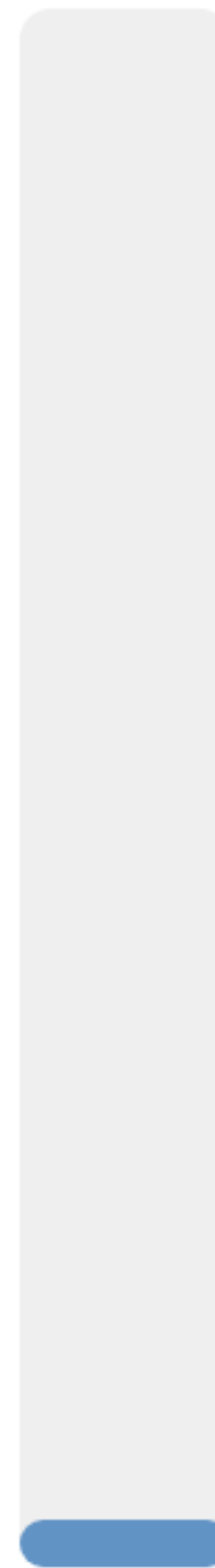
B

0



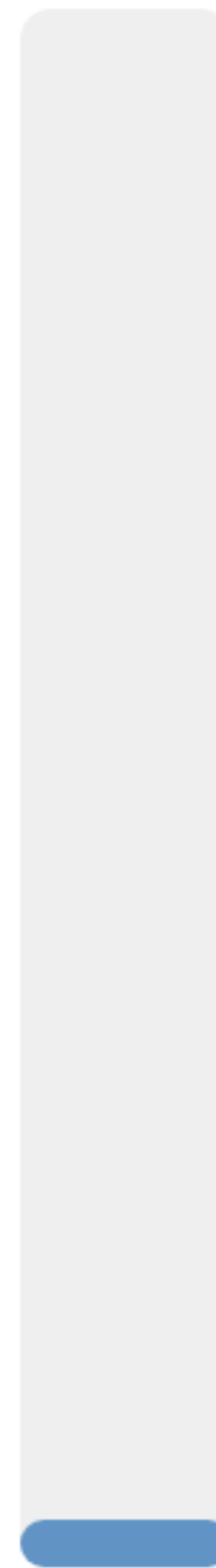
C

0



D

0



E

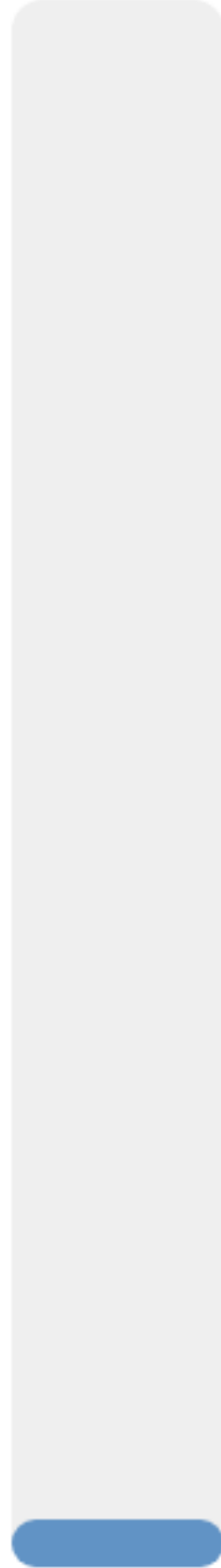


intel Core i7 14700K

- L1 data (D-L1) cache configuration of Core i7 14700K's P-Core
 - Size 48KB, 12-way set associativity, 64B block
 - Assume 64-bit memory address
 - Which of the following is **NOT** correct?
 - A. Tag is 52 bits
 - B. Index is 6 bits
 - C. Offset is 6 bits
 - D. The cache has 128 sets
 - E. All of the above are correct

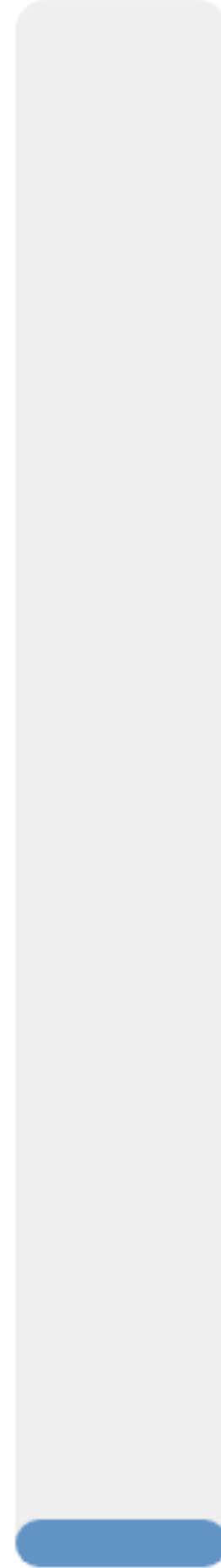
A screenshot of a Pollev poll interface. It shows five horizontal input fields, each with a letter (A, B, C, D, E) on the left. The fields are empty, suggesting a multiple-choice question where users select an answer.

0



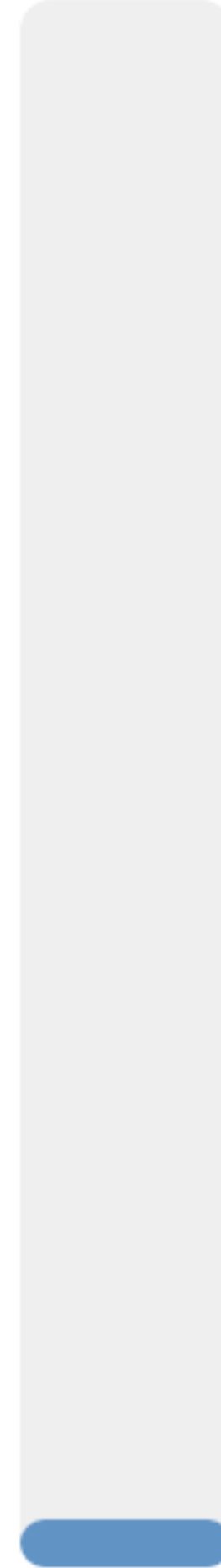
A

0



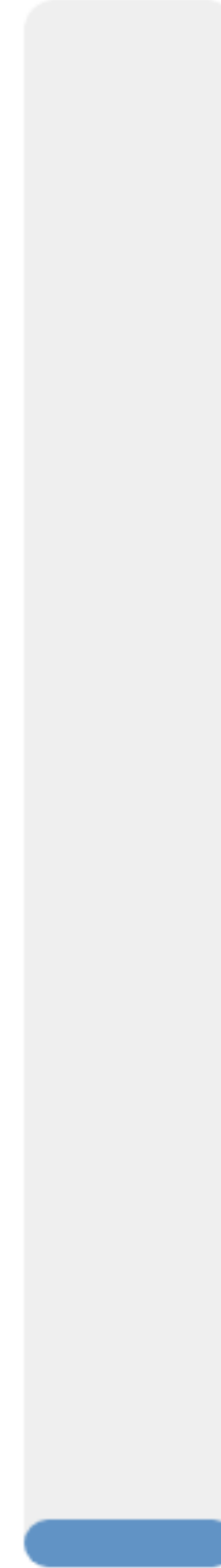
B

0



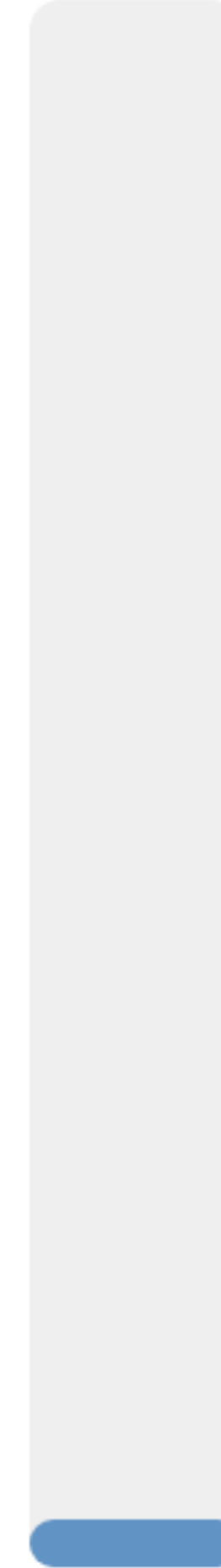
C

0



D

0



E

intel Core i7 14700K

- L1 data (D-L1) cache configuration of Core i7 14700K's P-Core
 - Size 48KB, 12-way set associativity, 64B block
 - Assume 64-bit memory address
 - Which of the following is **NOT** correct?

A. Tag is 52 bits

B. Index is 6 bits

C. Offset is 6 bits

D. The cache has 128 sets

E. All of the above are correct

$$C = A \times B \times S$$

$$48 \times 1024 = 12 \times 64 \times S$$

$$S = 64$$

$$Offset = \log_2(64) = 6$$

$$Index = \log_2(12) = 3.58 \approx 4$$

$$Tag = 64 - 6 - 4 = 52$$

Take-aways: designing caches

- Basic cache structures —
 - Caching in granularity of a block to capture spatial locality
 - Caching multiple blocks to keep frequently used data — temporal locality
 - Tags to distinguish cached blocks
- Hierarchical caching — data must be presented on the top level (L1) before the processor can use
- Optimizing cache structures
 - Hash block into “sets” to reduce the search time
 - Set-associativity to reduce the “collision” problem
- $C = A B S$
 - C: capacity
 - A: Associativity
 - S: Number of sets
 - $\lg(S)$: Number of bits in set index
 - $\lg(B)$: Number of bits in block offset

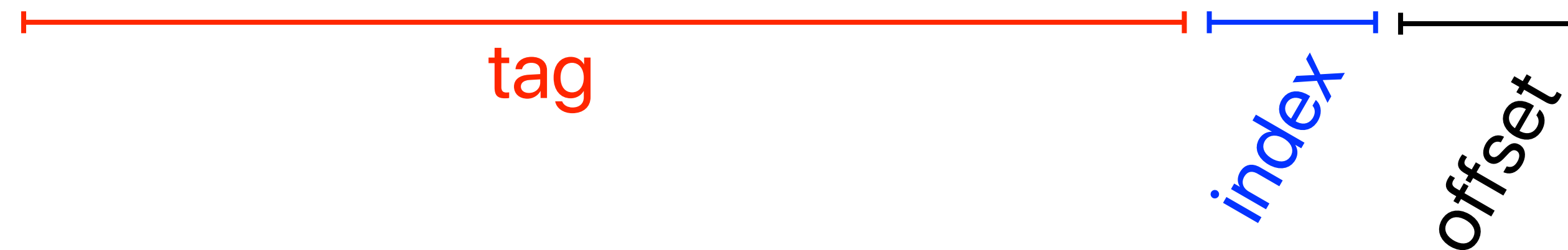
Simulate the cache!

Simulate a direct-mapped cache

- A direct mapped (1-way) cache with 256 bytes total capacity, a block size of 16 bytes

- # of blocks = $\frac{256}{16} = 16$
- $\lg(16) = 4$: 4 bits are used for the index
- $\lg(16) = 4$: 4 bits are used for the byte offset
- The tag is $64 - (4 + 4) = 56$ bits

- For example: 0x 8 0 0 0 0 0 8 0
= 0b1000 0000 0000 0000 0000 0000 0000 1000 0000





Matrix vector revisited

```
for(uint64_t i = 0; i < m; i++) {  
    result = 0;  
    for(uint64_t j = 0; j < n; j++) {  
        result += matrix[i][j]*vector[j];  
    }  
    output[i] = result;  
}
```

Matrix vector revisited

	tag index		tag index	
	Address (Hex)		Address (Binary)	
for(uint64_t i = 0; i < m; i++) {	&a[0][0]	0x558FE0A1D330	0b10101011000111111100000101000011101001100110000	
result = 0;	&b[0]	0x558FE0A1DC30	0b10101011000111111100000101000011101110000110000	
for(uint64_t j = 0; j < n; j++) {	&a[0][1]	0x558FE0A1D338	0b10101011000111111100000101000011101001100111000	
result += matrix[i][j]*vector[j];	&b[1]	0x558FE0A1DC38	0b10101011000111111100000101000011101110000111000	
}	&a[0][2]	0x558FE0A1D340	0b10101011000111111100000101000011101001101000000	
output[i] = result;	&b[2]	0x558FE0A1DC40	0b10101011000111111100000101000011101110001000000	
}	&a[0][3]	0x558FE0A1D348	0b10101011000111111100000101000011101001101001000	
	&b[3]	0x558FE0A1DC48	0b101010110001111111000001010000111011100001001000	
	&a[0][4]	0x558FE0A1D350	0b10101011000111111100000101000011101001101010000	
	&b[4]	0x558FE0A1DC50	0b101010110001111111000001010000111011100001010000	
	&a[0][5]	0x558FE0A1D358	0b10101011000111111100000101000011101001101011000	
	&b[5]	0x558FE0A1DC58	0b101010110001111111000001010000111011100001011000	
	&a[0][6]	0x558FE0A1D360	0b10101011000111111100000101000011101001101100000	
	&b[6]	0x558FE0A1DC60	0b101010110001111111000001010000111011100001100000	
	&a[0][7]	0x558FE0A1D368	0b10101011000111111100000101000011101001101101000	
	&b[7]	0x558FE0A1DC68	0b101010110001111111000001010000111011100001101000	
	&a[0][8]	0x558FE0A1D370	0b10101011000111111100000101000011101001101110000	
	&b[8]	0x558FE0A1DC70	0b101010110001111111000001010000111011100001110000	
	&a[0][9]	0x558FE0A1D378	0b10101011000111111100000101000011101001101111000	
	&b[9]	0x558FE0A1DC78	0b101010110001111111000001010000111011100001111000	

Simulate a direct-mapped cache

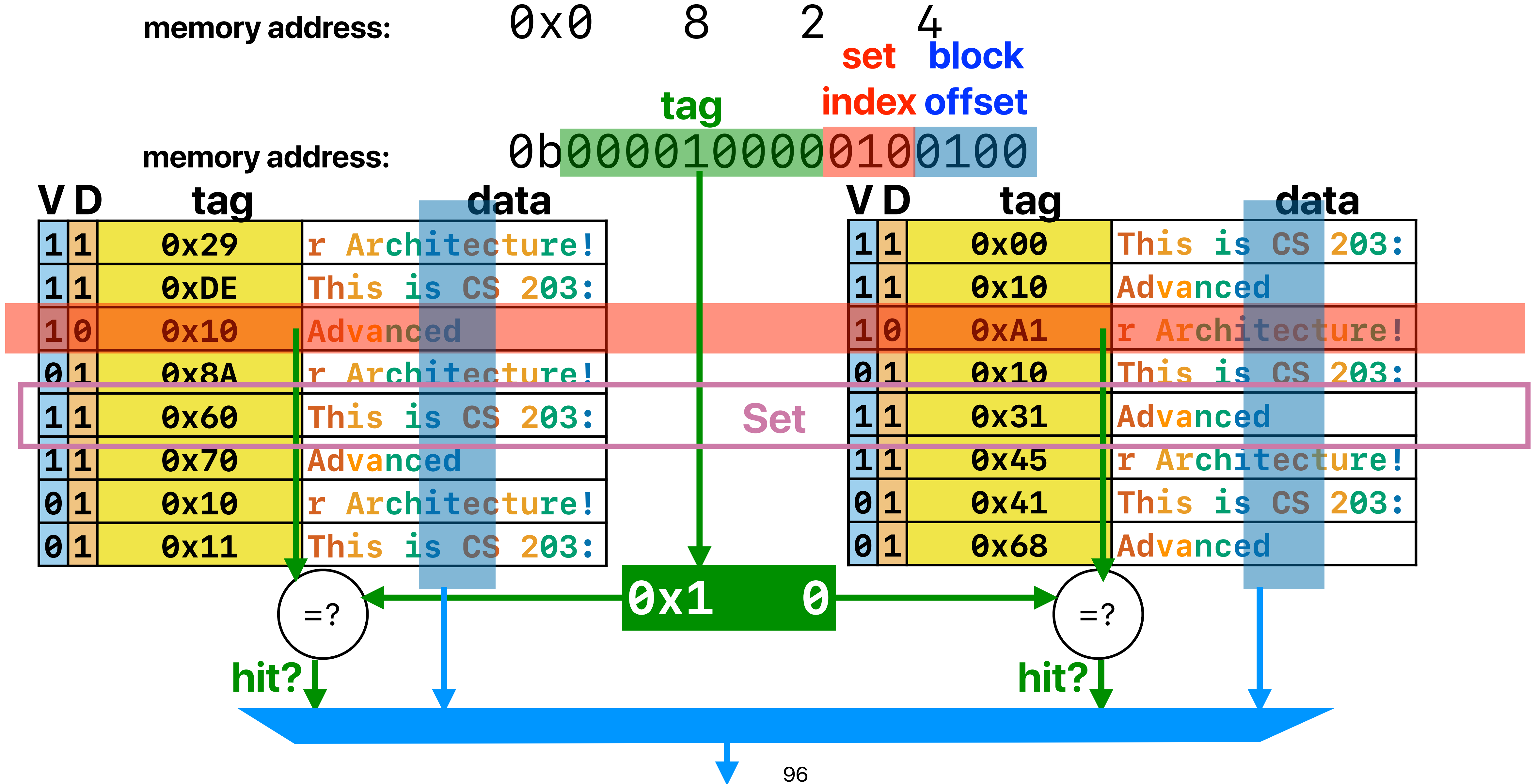
tag index

	V	D	Tag	Data
0	0	0		
1	0	0		
2	0	0		
3	1	0	0x558FE0A1DC	b[0], b[1]
4	1	0	0x558FE0A1DC	b[2], b[3]
5	0	0		
6	0	0		
7	0	0		
8	0	0		
9	0	0		
10	0	0		
11	0	0		
12	0	0		
13	0	0		
14	0	0		
15	0	0		

This cache doesn't work!!!
— collisions!

Address (Hex)		
&a[0][0]	0x558FE0A1D330	miss
&b[0]	0x558FE0A1DC30	miss
&a[0][1]	0x558FE0A1D338	miss
&b[1]	0x558FE0A1DC38	miss
&a[0][2]	0x558FE0A1D340	miss
&b[2]	0x558FE0A1DC40	miss
&a[0][3]	0x558FE0A1D348	miss
&b[3]	0x558FE0A1DC48	miss
&a[0][4]	0x558FE0A1D350	miss
&b[4]	0x558FE0A1DC50	miss
&a[0][5]	0x558FE0A1D358	miss
&b[5]	0x558FE0A1DC58	miss
&a[0][6]	0x558FE0A1D360	miss
&b[6]	0x558FE0A1DC60	miss
&a[0][7]	0x558FE0A1D368	miss
&b[7]	0x558FE0A1DC68	miss
&a[0][8]	0x558FE0A1D370	miss
&b[8]	0x558FE0A1DC70	miss
&a[0][9]	0x558FE0A1D378	
&b[9]	0x558FE0A1DC78	

Way-associative cache



Now, 2-way, same-sized cache

- A 2-way cache with 256 bytes total capacity, a block size of 16 bytes

- # of blocks = $\frac{256}{16} = 16$
- # of sets = $\frac{16}{2} = 8$ (2-way: 2 blocks in a set)
- $\lg(8) = 3$: 3 bits are used for the index
- $\lg(16) = 4$: 4 bits are used for the byte offset
- The tag is $64 - (4 + 4) = 56$ bits

- For example: 0x 8 0 0 0 0 0 0 8 0
= 0b1000 0000 0000 0000 0000 0000 1000 0000





Matrix vector revisited

tag index

tag

index

```
for(uint64_t i = 0; i < m; i++) {
    result = 0;
    for(uint64_t j = 0; j < n; j++) {
        result += matrix[i][j]*vector[j];
    }
    output[i] = result;
}
```

Address (Hex)		Address (Binary)	
&a[0][0]	0x558FE0A1D330	0b1010101100011111110000010100001110100110	0110000
&b[0]	0x558FE0A1DC30	0b1010101100011111110000010100001110111000	0110000
&a[0][1]	0x558FE0A1D338	0b1010101100011111110000010100001110100110	0111000
&b[1]	0x558FE0A1DC38	0b1010101100011111110000010100001110111000	0111000
&a[0][2]	0x558FE0A1D340	0b1010101100011111110000010100001110100110	1000000
&b[2]	0x558FE0A1DC40	0b1010101100011111110000010100001110111000	1000000
&a[0][3]	0x558FE0A1D348	0b1010101100011111110000010100001110100110	1001000
&b[3]	0x558FE0A1DC48	0b1010101100011111110000010100001110111000	1001000
&a[0][4]	0x558FE0A1D350	0b1010101100011111110000010100001110100110	1010000
&b[4]	0x558FE0A1DC50	0b1010101100011111110000010100001110111000	1010000
&a[0][5]	0x558FE0A1D358	0b1010101100011111110000010100001110100110	1011000
&b[5]	0x558FE0A1DC58	0b1010101100011111110000010100001110111000	1011000
&a[0][6]	0x558FE0A1D360	0b1010101100011111110000010100001110100110	1100000
&b[6]	0x558FE0A1DC60	0b1010101100011111110000010100001110111000	1100000
&a[0][7]	0x558FE0A1D368	0b1010101100011111110000010100001110100110	1101000
&b[7]	0x558FE0A1DC68	0b1010101100011111110000010100001110111000	1101000
&a[0][8]	0x558FE0A1D370	0b1010101100011111110000010100001110100110	1110000
&b[8]	0x558FE0A1DC70	0b1010101100011111110000010100001110111000	1110000
&a[0][9]	0x558FE0A1D378	0b1010101100011111110000010100001110100110	1111000
&b[9]	0x558FE0A1DC78	0b1010101100011111110000010100001110111000	1111000

Simulate a 2-way cache

V	D	Tag	Data	V	D	Tag	Data
0	0			0	0		
0	0			0	0		
0	0			0	0		
1	0	0xAB1FC143A6	a[0][0], a[0][1]	1	0	0xAB1FC143B8	b[0], b[1]
1	0	0xAB1FC143A6	a[0][2], a[0][3]	1	0	0xAB1FC143B8	b[2], b[3]
0	0			0	0		
0	0			0	0		
0	0			0	0		

	Address (Hex)	Tag	Index	
&a[0][0]	0x558FE0A1D330	0xAB1FC143A6	0x3	miss
&b[0]	0x558FE0A1DC30	0xAB1FC143B8	0x3	miss
&a[0][1]	0x558FE0A1D338	0xAB1FC143A6	0x3	hit
&b[1]	0x558FE0A1DC38	0xAB1FC143B8	0x3	hit
&a[0][2]	0x558FE0A1D340	0xAB1FC143A6	0x4	miss
&b[2]	0x558FE0A1DC40	0xAB1FC143B8	0x4	miss
&a[0][3]	0x558FE0A1D348	0xAB1FC143A6	0x4	hit
&b[3]	0x558FE0A1DC48	0xAB1FC143B8	0x4	hit
&a[0][4]	0x558FE0A1D350	0xAB1FC143A6	0x5	miss
&b[4]	0x558FE0A1DC50	0xAB1FC143B8	0x5	miss
&a[0][5]	0x558FE0A1D358	0xAB1FC143A6	0x5	hit
&b[5]	0x558FE0A1DC58	0xAB1FC143B8	0x5	hit
&a[0][6]	0x558FE0A1D360	0xAB1FC143A6	0x6	miss
&b[6]	0x558FE0A1DC60	0xAB1FC143B8	0x6	miss
&a[0][7]	0x558FE0A1D368	0xAB1FC143A6	0x6	hit
&b[7]	0x558FE0A1DC68	0xAB1FC143B8	0x6	hit
&a[0][8]	0x558FE0A1D370	0xAB1FC143A6	0x7	miss
&b[8]	0x558FE0A1DC70	0xAB1FC143B8	0x7	miss
&a[0][9]	0x558FE0A1D378	0xAB1FC143A6	0x7	hit
&b[9]	0x558FE0A1DC78	0xAB1FC143B8	0x7	hit



NVIDIA Tegra Orin

- D-L1 Cache configuration of NVIDIA Tegra Orin
 - Size 64KB, 4-way set associativity, 64B block, LRU policy, write-allocate, write-back, and assuming 64-bit address.

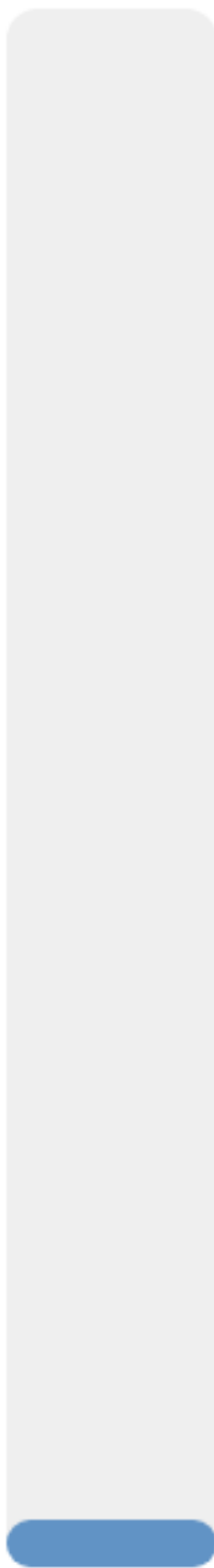
```
double a[16384], b[16384], c[16384], d[16384], e[16384];
/* a = 0x10000, b = 0x20000, c = 0x30000, d = 0x40000, e = 0x50000 */
for(i = 0; i < 512; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
    //load a[i], b[i], c[i], d[i] and then store to e[i]
}
```

What's the data cache miss rate for this code?

- A. 12.5%
- B. 56.25%
- C. 66.67%
- D. 68.75%
- E. 100%

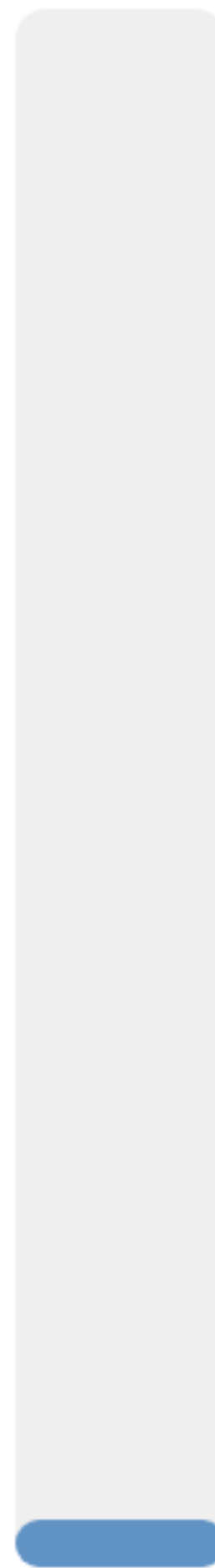


0



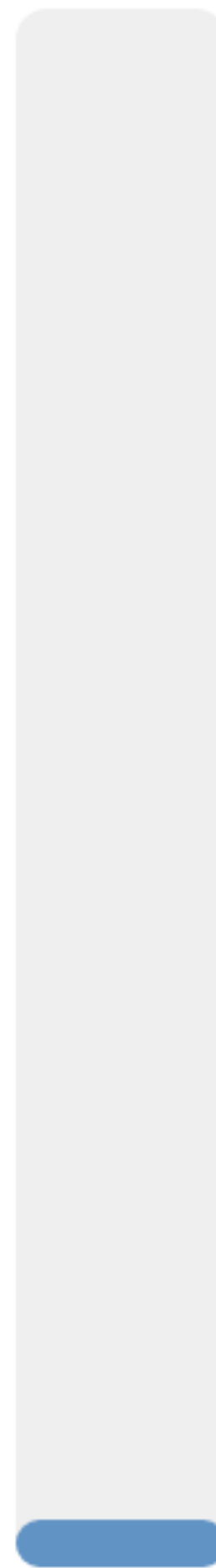
A

0



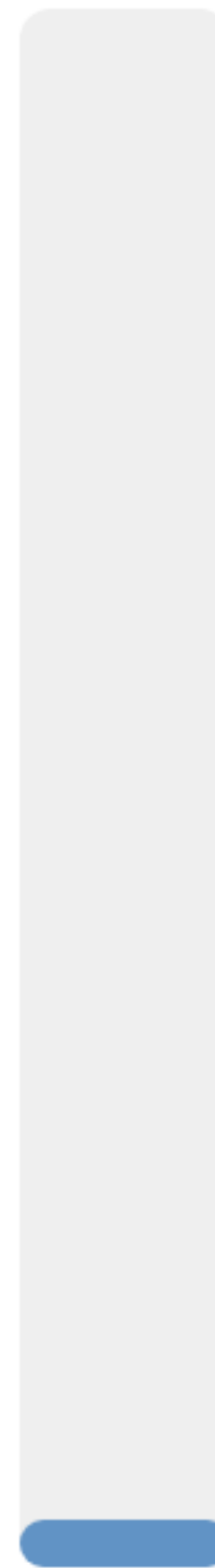
B

0



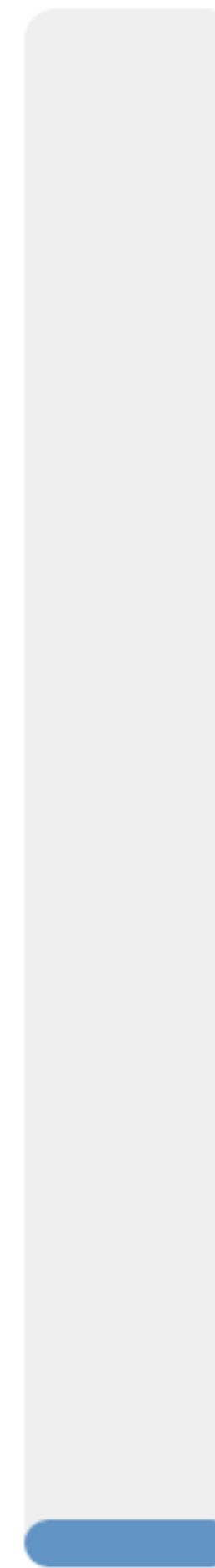
C

0



D

0



E



NVIDIA Tegra Orin

- D-L1 Cache configuration of NVIDIA Tegra Orin
 - Size 64KB, 4-way set associativity, 64B block, LRU policy, write-allocate, write-back, and assuming 64-bit address.

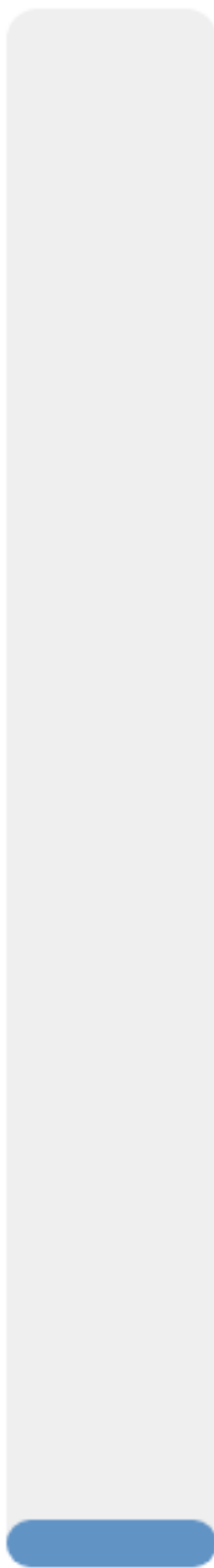
```
double a[16384], b[16384], c[16384], d[16384], e[16384];
/* a = 0x10000, b = 0x20000, c = 0x30000, d = 0x40000, e = 0x50000 */
for(i = 0; i < 512; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
    //load a[i], b[i], c[i], d[i] and then store to e[i]
}
```

What's the data cache miss rate for this code?

- A. 12.5%
- B. 56.25%
- C. 66.67%
- D. 68.75%
- E. 100%

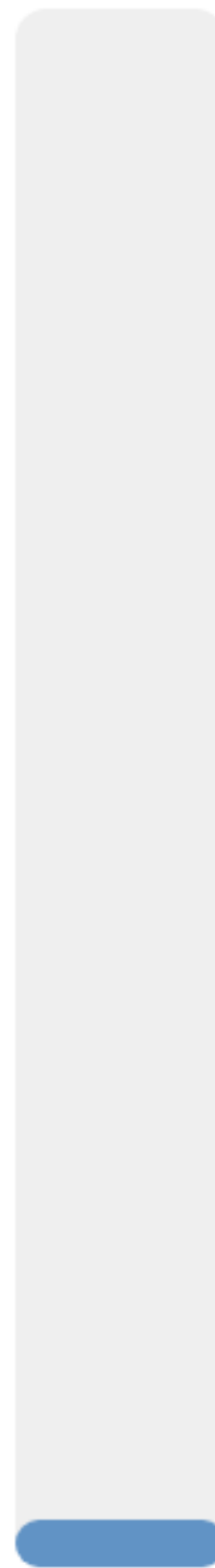


0



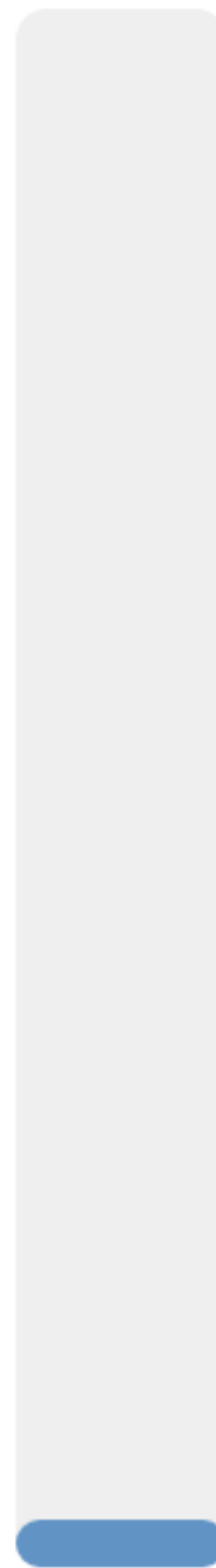
A

0



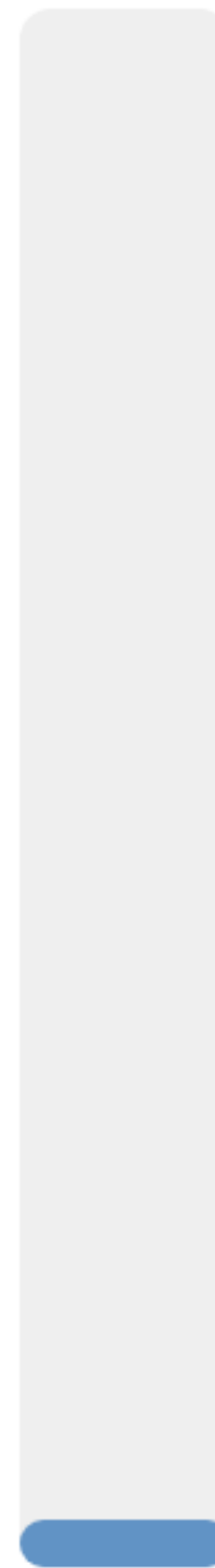
B

0



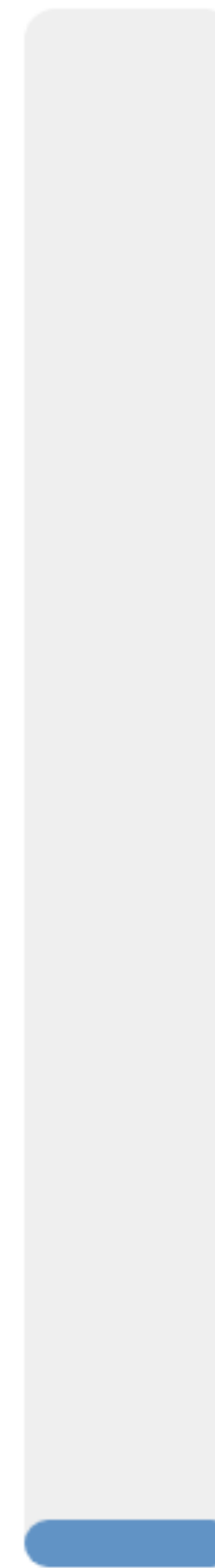
C

0



D

0



E

NVIDIA Tegra Orin 100% miss rate!

- Size 64KB, 4-way set associativity, 64B block, LRU policy, write-allocate, write-back, and assuming 64-bit address.

```
double a[16384], b[16384], c[16384], d[16384], e[16384];
/* a = 0x10000, b = 0x20000, c = 0x30000, d = 0x40000, e = 0x50000 */
for(i = 0; i < 512; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
    //load a[i], b[i], c[i], d[i] and then store to e[i]
```

C = ABS
32KB = 4 * 64 * S
S = 256
offset = lg(64) = 6 bits
index = lg(256) = 8 bits
tag = the rest bits

	Address (Hex)	Address in binary			Tag	Index	Hit? Miss?	Replace?
	a[0]	0x10000	0b000010000000000000000000	0b1	0b0		Miss	
	b[0]	0x20000	0b001000000000000000000000	0b0	0b0		Miss	
	c[0]	0x30000	0b001100000000000000000000	0b1	0b0		Miss	
	d[0]	0x40000	0b010000000000000000000000	0b10	0b0		Miss	
	e[0]	0x50000	0b010100000000000000000000	0b11	0b0		Miss	a[0-7]
	a[1]	0x10008	0b000010000000000000000001	0b1	0b0		Miss	b[0-7]
	b[1]	0x20008	0b001000000000000000000001	0b0	0b0		Miss	c[0-7]
	c[1]	0x30008	0b001100000000000000000001	0b1	0b0		Miss	d[0-7]
	d[1]	0x40008	0b010000000000000000000001	0b10	0b0		Miss	e[0-7]
	e[1]	0x50008	0b010100000000000000000001	0b11	0b0		Miss	a[0-7]
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮

Estimating code performance on caches (cont.)

NVIDIA Tegra Orin

100% miss rate!

- Size 64KB, 4-way set associativity, 64B block, LRU policy, write-allocate, write-back, and assuming 64-bit address.

```
double a[16384], b[16384], c[16384], d[16384], e[16384];
/* a = 0x10000, b = 0x20000, c = 0x30000, d = 0x40000, e = 0x50000 */
for(i = 0; i < 512; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
    //load a[i], b[i], c[i], d[i] and then store to e[i]
```

C = ABS
64KB = 4 * 64 * S
S = 256
offset = lg(64) = 6 bits
index = lg(256) = 8 bits
tag = the rest bits

	Address (Hex)	Address in binary			Tag	Index	Hit? Miss?	Replace?
a[0]	0x10000	0b000100	00000000	000000	0x4	0x0	Miss	
b[0]	0x20000	0b001000	00000000	000000	0x8	0x0	Miss	
c[0]	0x30000	0b001100	00000000	000000	0xC	0x0	Miss	
d[0]	0x40000	0b010000	00000000	000000	0x10	0x0	Miss	
e[0]	0x50000	0b010100	00000000	000000	0x14	0x0	Miss	a[0-7]
a[1]	0x10008	0b000100	00000000	001000	0x4	0x0	Miss	b[0-7]
b[1]	0x20008	0b001000	00000000	001000	0x8	0x0	Miss	c[0-7]
c[1]	0x30008	0b001100	00000000	001000	0xC	0x0	Miss	d[0-7]
d[1]	0x40008	0b010000	00000000	001000	0x10	0x0	Miss	e[0-7]
e[1]	0x50008	0b010100	00000000	001000	0x14	0x0	Miss	a[0-7]
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮

NVIDIA Tegra Orin

- D-L1 Cache configuration of NVIDIA Tegra Orin
 - Size 64KB, 4-way set associativity, 64B block, LRU policy, write-allocate, write-back, and assuming 64-bit address.

```
double a[16384], b[16384], c[16384], d[16384], e[16384];
/* a = 0x10000, b = 0x20000, c = 0x30000, d = 0x40000, e = 0x50000 */
for(i = 0; i < 512; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
    //load a[i], b[i], c[i], d[i] and then store to e[i]
}
```

What's the data cache miss rate for this code?

- A. 12.5%
- B. 56.25%
- C. 66.67%
- D. 68.75%
- E. 100%





intel Core i7 14700K

- D-L1 Cache configuration of intel Core i7 14700K's P-Core
- Size 48KB, 12-way set associativity, 64B block, LRU policy, write-allocate, write-back, and assuming 64-bit address.

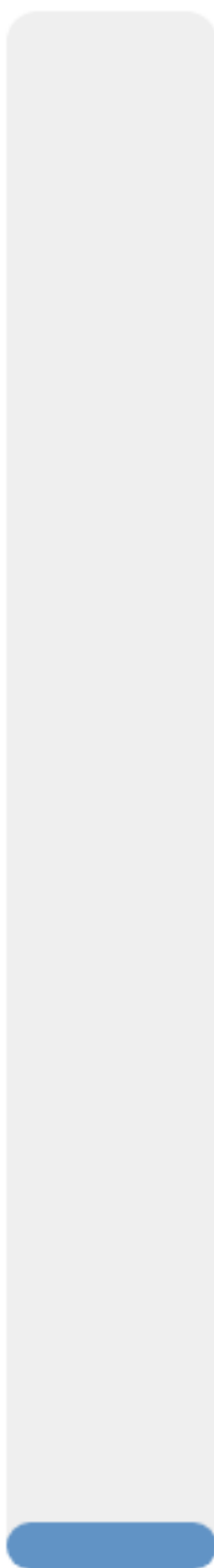
```
double a[16384], b[16384], c[16384], d[16384], e[16384];
/* a = 0x10000, b = 0x20000, c = 0x30000, d = 0x40000, e = 0x50000 */
for(i = 0; i < 512; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
    //load a[i], b[i], c[i], d[i] and then store to e[i]
}
```

What's the data cache miss rate for this code?

- A. 12.5%
- B. 56.25%
- C. 66.67%
- D. 68.75%
- E. 100%

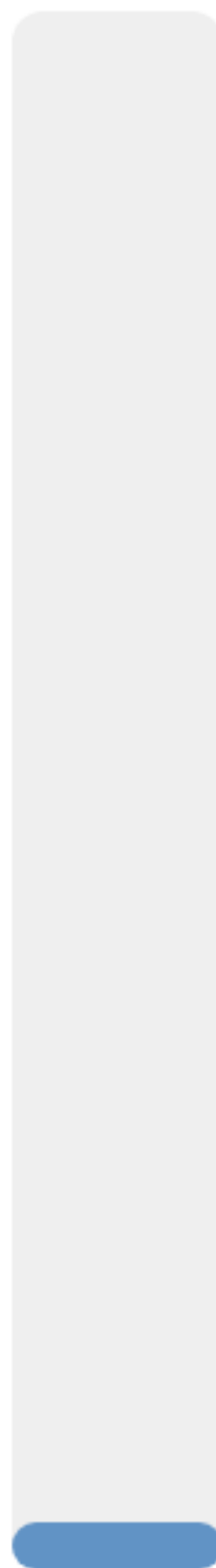
A screenshot of a Pollev.com poll interface. It shows five empty input boxes labeled A, B, C, D, and E, arranged vertically. The interface is part of a presentation, as indicated by the footer text.

0



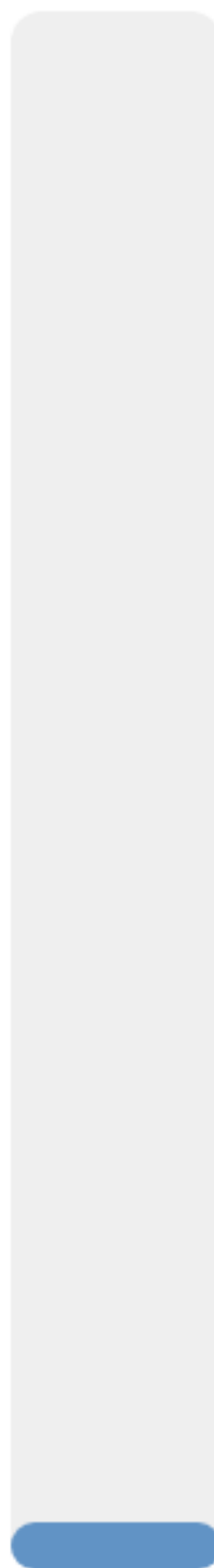
A

0



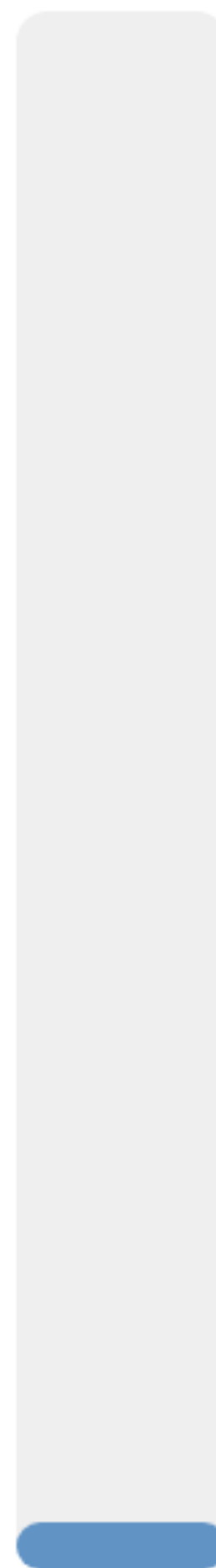
B

0



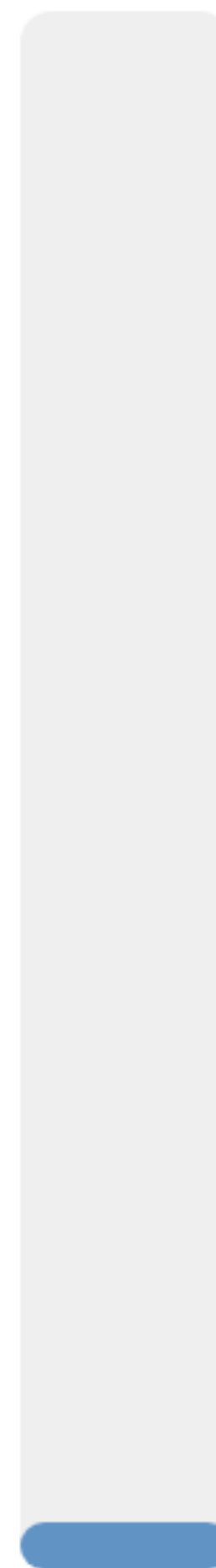
C

0



D

0



E



intel Core i7 14700K

- D-L1 Cache configuration of intel Core i7 14700K's P-Core
- Size 48KB, 12-way set associativity, 64B block, LRU policy, write-allocate, write-back, and assuming 64-bit address.

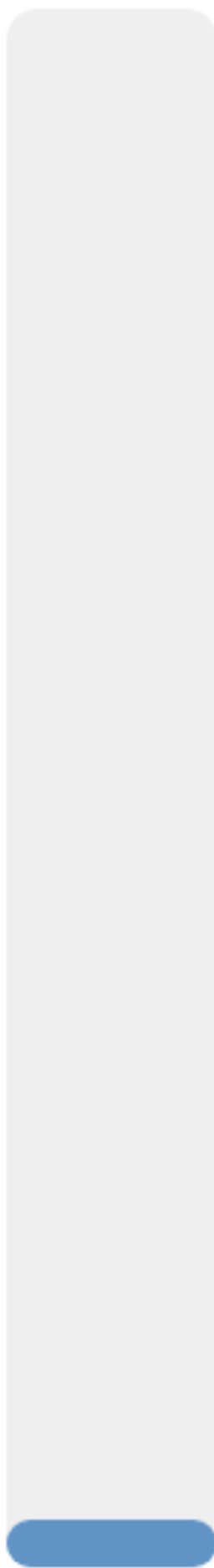
```
double a[16384], b[16384], c[16384], d[16384], e[16384];
/* a = 0x10000, b = 0x20000, c = 0x30000, d = 0x40000, e = 0x50000 */
for(i = 0; i < 512; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
    //load a[i], b[i], c[i], d[i] and then store to e[i]
}
```

What's the data cache miss rate for this code?

- A. 12.5%
- B. 56.25%
- C. 66.67%
- D. 68.75%
- E. 100%

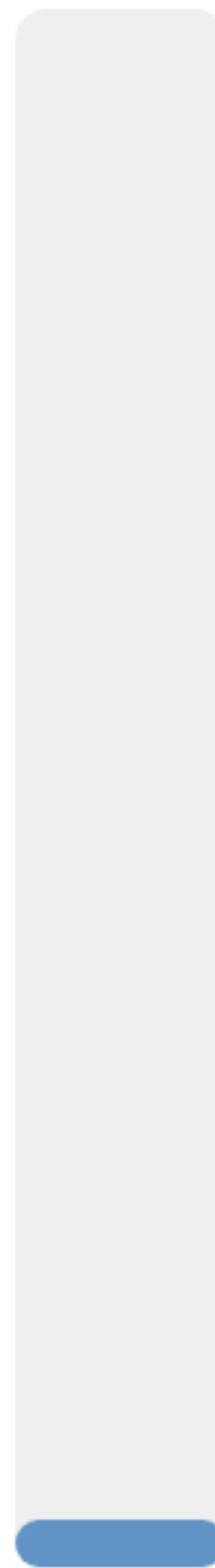
A screenshot of a Pollev poll interface. It shows five empty input boxes, each with a letter (A, B, C, D, E) to its left. The interface is clean and modern, with a light blue header and a white background for the poll area.

0



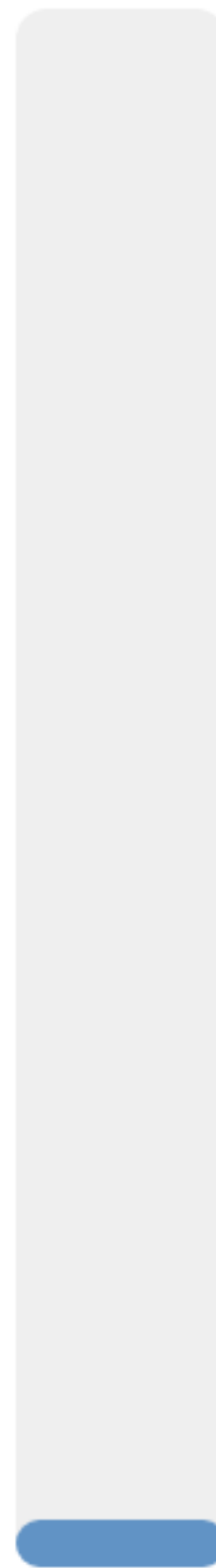
A

0



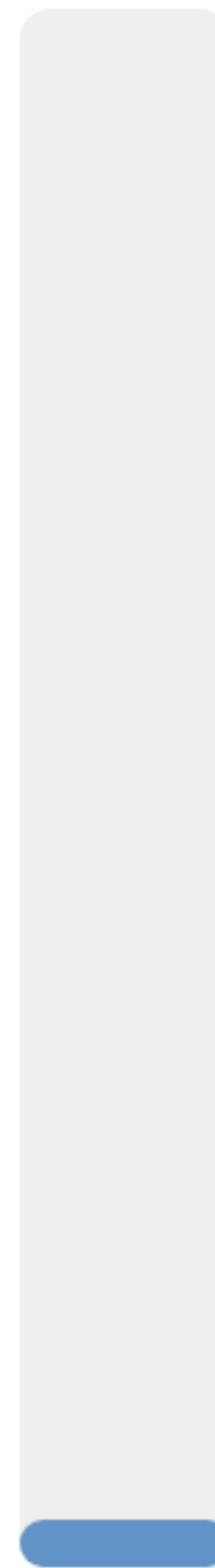
B

0



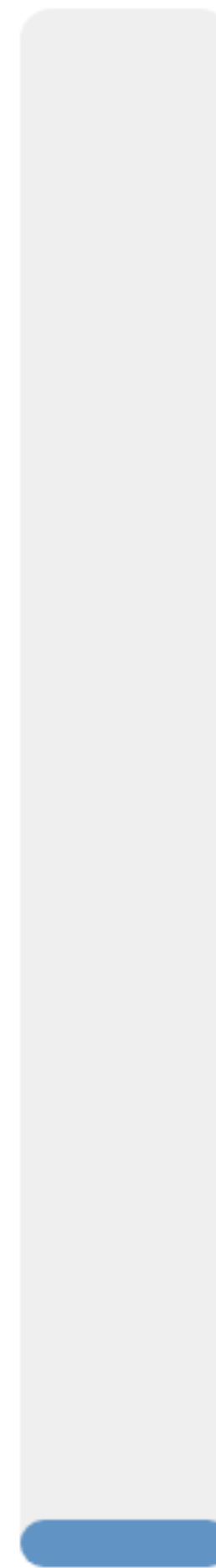
C

0



D

0



E

intel Core i7 14700K

- Size 48KB, 12-way set associativity, 64B block, LRU policy, write-allocate, write-back, and assuming 64-bit address.

```
double a[16384], b[16384], c[16384], d[16384], e[16384];
/* a = 0x10000, b = 0x20000, c = 0x30000, d = 0x40000, e = 0x50000 */
for(i = 0; i < 512; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
    //load a[i], b[i], c[i], d[i] and then store to e[i]
```

C = ABS
48KB = 12 * 64 * S
S = 64
offset = lg(64) = 6 bits
index = lg(64) = 6 bits
tag = the rest bits

	Address (Hex)	Address in binary	Tag	Index	Hit? Miss?	Replace?
a[0]	0x10000	0b0001000000000000000000	0x10	0x0	Miss	
b[0]	0x20000	0b0010000000000000000000	0x20	0x0	Miss	
c[0]	0x30000	0b0011000000000000000000	0x30	0x0	Miss	
d[0]	0x40000	0b0100000000000000000000	0x40	0x0	Miss	
e[0]	0x50000	0b0101000000000000000000	0x50	0x0	Miss	
a[1]	0x10008	0b0001000000000000001000	0x10	0x0	Hit	
b[1]	0x20008	0b0010000000000000001000	0x20	0x0	Hit	
c[1]	0x30008	0b0011000000000000001000	0x30	0x0	Hit	
d[1]	0x40008	0b0100000000000000001000	0x40	0x0	Hit	
e[1]	0x50008	0b0101000000000000001000	0x50	0x0	Hit	
⋮	⋮	⋮	⋮	⋮	⋮	⋮

intel Core i7 14700K (cont.)

- Size 48KB, 12-way set associativity, 64B block, LRU policy, write-allocate, write-back, and assuming 64-bit address.

```
double a[16384], b[16384], c[16384], d[16384], e[16384];
/* a = 0x10000, b = 0x20000, c = 0x30000, d = 0x40000, e = 0x50000 */
for(i = 0; i < 512; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
    //load a[i], b[i], c[i], d[i] and then store to e[i]
```

C = ABS
48KB = 12 * 64 * S
S = 64
offset = lg(64) = 6 bits
index = lg(64) = 6 bits
tag = the rest bits

	Address (Hex)	Address in binary	Tag	Index	Hit? Miss?	Replace?
a[7]	0x10038	0b00010000000000111000	0x10	0x0	Hit	
b[7]	0x20038	0b00100000000000111000	0x20	0x0	Hit	
c[7]	0x30038	0b00110000000000111000	0x30	0x0	Hit	
d[7]	0x40038	0b01000000000000111000	0x40	0x0	Hit	
e[7]	0x50038	0b01010000000000111000	0x50	0x0	Hit	
a[8]	0x10040	0b00010000000001000000	0x10	0x1	Miss	
b[8]	0x20040	0b00100000000001000000	0x20	0x1	Miss	
c[8]	0x30040	0b00110000000001000000	0x30	0x1	Miss	
d[8]	0x40040	0b01000000000001000000	0x40	0x1	Miss	
e[8]	0x50040	0b01010000000001000000	0x50	0x1	Miss	
a[9]	0x10048	0b00010000000001001000	0x10	0x1	Hit	
b[9]	0x20048	0b00100000000001001000	0x20	0x1	Hit	
c[9]	0x30048	0b00110000000001001000	0x30	0x1	Hit	
d[9]	0x40048	0b01000000000001001000	0x40	0x1	Hit	

$$\frac{5 \times \frac{512}{8}}{5 \times 512} = \frac{1}{8} = 12.5 \%$$

Miss when the array index is a multiply of 8!



Computer Science & Engineering

203

つづく

